

PYTHON PARA TODOS

De Cero a Programador



Python

Aprende a programar

Jose Ojeda Rojas

Publicado en Febrero de 2026

1ª edición

PYTHON PASO A PASO

@JOSECODETECH

Todas las marcas mencionadas en esta obra, ajenas al autor, son marcas registradas por sus propios autores y se mencionan con fines academicos y orientativos.

No esta permitida la reproduccion total o parcial de esta obra sin el consentimiento expreso del autor.

Copyright © 2026 Jose Ojeda Rojas. Todos los derechos reservados.

Contenido

Prólogo: Tu Viaje hacia la Programación	9
¿Por qué aprender a programar hoy en día?	9
¿Por qué Python es el lenguaje ideal para empezar?	10
¿Para quién es este libro y cómo sacarle el máximo provecho?	11
Capítulo 1: Pensar como un Programador	12
1.1 ¿Qué es programar? La comunicación con la máquina	12
1.2 El concepto de Algoritmo: Tu receta para el éxito	13
1.3 Descomposición: El arte de vencer problemas complejos	14
1.4 Reconocimiento de Patrones: Encontrando lo que se repite	15
1.5 Ejercicio mental: Diseñando un algoritmo cotidiano.....	16
Capítulo 2: Tu Taller de Trabajo	19
2.1 El papel del Intérprete: Cómo tu ordenador entiende Python	19
2.2 Instalación de Python paso a paso	20
2.3 Tu herramienta de escritura: Instalación y configuración de Visual Studio Code (VS Code)	22
2.4 Tu primer programa: "¡Hola, Mundo!" y el ciclo de edición-ejecución	24
Capítulo 3: Almacenando Información - Variables y Tipos de Datos Básicos	26
3.1 ¿Qué es una variable? Las "cajas con etiqueta" para tus datos.....	26
3.2 Tipos de Datos Esenciales.....	29
3.3 Operaciones básicas: Matemáticas y uniendo texto.....	31
3.4 Conversión de Tipos (Casting): La metamorfosis de los datos	35
3.5 Ejercicio práctico: Calculadora de Índice de Masa Corporal (IMC) ...	37
Capítulo 4: Interactuando con el Mundo - Entrada y Salida	39

4.1 Mostrando resultados: La función <code>print()</code> a fondo.....	40
4.2 Preguntando al usuario: La función <code>input()</code>	42
4.3 Formato de texto dinámico: Las F-strings (la forma moderna)	45
4.4 Ejercicio práctico: Calculadora de Propinas Interactiva.....	48
Capítulo 5: Tomando Decisiones (Lógica Condicional)	51
5.1 El guardián de la puerta: La sentencia <code>if</code>	51
5.2 La alternativa: <code>else</code> (Si no...).....	53
5.3 Más de dos opciones: La sentencia <code>elif</code> (Si no, si...)	55
5.4 Los Jueces del Código: Operadores de Comparación	58
5.5 Combinando condiciones: Operadores Lógicos (<code>and</code> , <code>or</code> , <code>not</code>).....	61
5.6 Ejercicio Práctico: El Guardián del Bosque Mágico	64
Capítulo 6: El poder de la repetición (Bucles).....	68
6.1 El bucle <code>while</code> : Repetir mientras sea verdad	68
6.2 El bucle <code>for</code> : Repetir para cada elemento	70
1. Recorriendo texto (Strings).....	71
2. Repetir un número de veces: La función <code>range()</code>	71
¿Cuándo usar <code>while</code> y cuándo usar <code>for</code>?.....	72
6.3 Ejercicio Práctico: El Juego de las Adivinanzas	74
Guía paso a paso (Descomposición del problema):	74
¡Inténtalo tú primero!.....	75
Solución propuesta.....	75
Capítulo 7: Listas (Las súper-variables).....	78
7.1 ¿Cómo se crea una lista?	78
7.2 Accediendo a los elementos (El truco del índice 0)	79
7.3 Modificando listas (Son mutables)	80

7.4 Recorriendo listas con bucles for.....	81
7.5 Modificando listas: Métodos clave (.append, .remove)	84
Capítulo 8: El libro de los secretos (Diccionarios).....	87
8.1 ¿Cómo se crea un diccionario?	87
8.2 Accediendo a los datos (La llave maestra)	88
8.3 Modificando el diccionario: Añadir, cambiar y borrar	90
1. Añadir y Modificar datos	90
2. Borrar datos (La palabra clave del)	91
8.4 Recorriendo diccionarios con bucles for	92
Capítulo 9: Funciones (Tus propias herramientas).....	95
9.1 Definiendo nuestra primera función (def)	95
9.2 Funciones con entradas (Parámetros y Argumentos).....	97
9.3 El valor de retorno (return): La entrega del pedido	99
9.4 Ejercicio Práctico: Conversor de Moneda	102
9.5 El Ámbito (Scope): ¿Dónde vive mi variable?.....	104
9.6 Funciones anónimas: Las pequeñas lambda	107
9.7 Ejercicio práctico: De Monolito a Modular.....	111
Capítulo 10: Organizando Proyectos Grandes - Módulos y Paquetes	115
10.1 ¿Qué es un módulo? Divide y vencerás.....	115
10.2 Importando código: La sentencia import	116
10.3 La Biblioteca Estándar de Python: El tesoro incluido.....	123
10.4 Creando tus propios módulos "profesionales"	126
10.5 ¿Qué es un paquete? Carpetas con superpoderes	130
10.6 Introducción a pip y PyPI: El supermercado global.....	133

Capítulo 11: Cuando las Cosas Salen Mal - Manejo de Errores y Excepciones	136
11.1 Los errores son inevitables: Conoce a tu enemigo	136
11.2 El bloque try-except: Tu red de seguridad	139
11.3 Capturando excepciones específicas: Un plan para cada desastre ...	142
11.4 El bloque finally: Lo que sucede sí o sí	145
11.5 Lanzando tus propias excepciones con raise	149
Capítulo 12: Manejando Archivos - Memoria a Largo Plazo	152
12.1 RAM vs. Disco: El cerebro y la libreta	152
12.2 Abriendo y cerrando la "libreta"	153
12.3 Leyendo archivos: Recuperando la memoria	156
12.4 Escribiendo y añadiendo datos: ¡Cuidado con el borrador!	160
12.5 Rutas de archivos: El mapa del tesoro (y el lío de Windows vs Mac)	163
Capítulo 13: Clases y Objetos - El molde y la galleta	167
13.1 La analogía fundamental: El Plano vs. La Casa	167
13.2 Definiendo tu primera Clase: El plano del Robot	169
13.3 Creando Objetos: ¡La fábrica empieza a producir!	173
13.4 Usando el objeto: La notación del punto (.)	174
13.5 El Constructor <code>__init__</code> : El momento del nacimiento	177
Capítulo 14: Herencia - No reinventes la rueda	181
14.1 La clase Padre y la clase Hija	181
14.2 Extendiendo y Sobrescribiendo (La verdadera potencia)	182
14.3 El comando vital: <code>super()</code>	184
Capítulo 15: Encapsulamiento - Secretos y seguridad	187
15.1 La filosofía de Python: "Todos somos adultos aquí"	187

15.2 El aviso amable: Un guion bajo (_variable)	188
15.3 La barrera fuerte: Doble guion bajo (__variable).....	188
Capítulo 16: Librerías útiles - No reinventes la rueda (Parte 2)	192
16.1 Ejemplo 1: Visualización de Datos con matplotlib	193
16.2 Ejemplo 2: Hablando con Internet usando requests.....	195
Capítulo Final: El Camino por Delante	198
1. El camino del Desarrollador Web 🌐	198
2. El camino del Científico de Datos y la IA 🧠 📊	199
3. El camino de la Automatización y Scripting 🤖	199
4. El camino del Desarrollador de Videojuegos 🎮	200
Un último consejo de amigo	200
Sobre el autor	202

Prólogo: Tu Viaje hacia la Programación

Bienvenido. Estás a punto de embarcarte en un viaje que transformará tu forma de interactuar con la tecnología. Este libro no es solo un manual técnico; es una guía diseñada para enseñarte a pensar de una manera nueva: a pensar como un programador.

¿Por qué aprender a programar hoy en día?

Vivimos en un mundo definido por el software. Desde la aplicación que usas para pedir comida hasta los sistemas que gestionan el tráfico aéreo, todo funciona gracias al código. Aprender a programar te otorga un superpoder en este mundo moderno: la capacidad de dejar de ser un mero consumidor de tecnología para convertirte en un creador.

No importa si eres un estudiante, un profesional en busca de nuevas oportunidades, un emprendedor o simplemente una persona curiosa. La programación te permite automatizar tareas tediosas, analizar datos complejos, construir herramientas que resuelvan problemas reales y dar vida a tus ideas. Es una habilidad fundamental para el siglo XXI, una herramienta que amplifica tu capacidad de crear e innovar en cualquier campo.

¿Por qué Python es el lenguaje ideal para empezar?

El mundo de la programación está lleno de lenguajes diferentes, pero hemos elegido Python como tu guía por varias razones fundamentales:

1. **Legibilidad:** Python fue diseñado para ser fácil de leer y escribir. Su sintaxis es limpia y se asemeja al inglés, lo que te permite concentrarte en la lógica de la resolución de problemas en lugar de luchar contra reglas gramaticales complejas.
2. **Versatilidad:** Es un lenguaje "todoterreno". Se utiliza en el desarrollo web, la inteligencia artificial, el análisis de datos, la automatización de sistemas, el desarrollo de juegos y mucho más. Aprender Python te abre las puertas a una multitud de campos profesionales.
3. **Comunidad y Ecosistema:** Python cuenta con una de las comunidades de desarrolladores más grandes y activas del mundo. Esto significa que encontrarás una cantidad inmensa de recursos, tutoriales, librerías y personas dispuestas a ayudarte cuando te atasques.
4. **Demanda Laboral:** Las habilidades en Python están entre las más demandadas en el mercado laboral actual.

¿Para quién es este libro y cómo sacarle el máximo provecho?

Este libro ha sido escrito para **principiantes absolutos**. No asumimos ningún conocimiento previo de programación. Si sabes usar un ordenador y tienes ganas de aprender, tienes todo lo necesario.

Nuestro enfoque es **práctico y progresivo**. No te abrumaremos con teoría abstracta desde el principio. En su lugar, introduciremos los conceptos paso a paso, siempre acompañados de ejemplos claros y ejercicios prácticos que te retarán a aplicar lo aprendido inmediatamente.

Para aprovechar al máximo este libro, te recomendamos:

1. **No solo leas, escribe:** La programación es una habilidad práctica, como tocar un instrumento. No puedes aprender a tocar el piano solo leyendo un libro sobre teoría musical. **Escribe cada línea de código** que encuentres en los ejemplos. No los copies y pegues; teclearlos te ayudará a interiorizar la sintaxis.
2. **Experimenta:** Una vez que un ejemplo funcione, ¡rómpele! Cambia valores, modifica el orden de las líneas, intenta añadir nuevas funcionalidades. La mejor forma de aprender es preguntarte "¿Qué pasaría si...?" y probarlo.
3. **Haz los ejercicios:** Al final de cada sección clave, encontrarás ejercicios propuestos. Son fundamentales para consolidar tus conocimientos. No pases al siguiente tema hasta que te sientas cómodo resolviéndolos.
4. **Usa el repositorio de GitHub:** Todo el código de este libro, tanto los ejemplos como las soluciones a los ejercicios, estará disponible en un repositorio público en GitHub. Úsalo como referencia, pero intenta siempre resolver los problemas por ti mismo primero.

Aprender a programar es un desafío, pero también es increíblemente gratificante. Habrá momentos de frustración cuando tu código no funcione,
PYTHON PASO A PASO [@JOSECODETECH](#)

pero también momentos de pura euforia cuando finalmente lo haga. Este libro es tu mapa de ruta; síguelo con paciencia, curiosidad y persistencia, y te aseguramos que llegarás a tu destino.

¿Estás listo? Empecemos.

Capítulo 1: Pensar como un Programador

Antes de aprender la sintaxis de Python, de saber dónde poner un paréntesis o unas comillas, necesitamos entender qué es lo que realmente hacemos cuando nos sentamos frente al ordenador a "programar".

Mucha gente piensa que programar es "escribir código". Pero el código es solo el resultado final. El verdadero trabajo ocurre antes, en tu cabeza.

Programar es, fundamentalmente, resolver problemas. El código es simplemente el lenguaje que usamos para explicarle la solución a una máquina.

1.1 ¿Qué es programar? La comunicación con la máquina

Imagina que tienes un asistente increíblemente rápido y poderoso, capaz de realizar cálculos complejos en nanosegundos. Pero este asistente tiene un problema: es absolutamente literal y no tiene sentido común. Si le dices "ve a la tienda y compra leche, y si tienen huevos, compra seis", este asistente volverá con 6 botellas de leche porque tenían huevos. ¿Por qué? Porque tu instrucción fue ambigua para una mente literal.

Las computadoras son ese asistente. No entienden de matices, intenciones o contextos. Solo entienden instrucciones precisas, inambiguas y ordenadas.

Programar es el acto de traducir una idea o una solución humana a un conjunto de instrucciones tan precisas que una máquina no tenga más remedio que ejecutarlas exactamente como tú pretendías.

1.2 El concepto de Algoritmo: Tu receta para el éxito

Ya hemos visto que necesitamos dar instrucciones precisas. ¿Cómo llamamos a un conjunto de instrucciones precisas para resolver un problema? **Algoritmo**.

Esta palabra, que suena tan técnica y compleja, es en realidad uno de los conceptos más sencillos de entender. Usas algoritmos todos los días, incluso sin darte cuenta.

Un algoritmo es una secuencia ordenada, finita y no ambigua de pasos que llevan a la solución de un problema.

Desglosemos esta definición:

- **Secuencia ordenada:** Los pasos deben seguirse en un orden específico. No puedes ponerte los zapatos antes que los calcetines.
- **Finita:** El proceso debe tener un final. No puede quedarse en un bucle infinito de pasos.
- **No ambigua:** Cada paso debe ser perfectamente claro y tener una sola interpretación posible.

La mejor analogía para un algoritmo es una receta de cocina muy detallada. No te dice "cocina el pollo hasta que esté listo", sino "hornea a 200 °C durante 45 minutos".

1.3 Descomposición: El arte de vencer problemas complejos

Muchos principiantes se sienten abrumados cuando se enfrentan a un problema de programación complejo. ¿Por dónde empiezo? ¿Cómo voy a ser capaz de crear un programa tan grande?

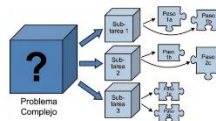
La respuesta es una técnica fundamental llamada **Descomposición**.

Descomponer es el acto de tomar un problema grande y complejo y dividirlo en una serie de problemas más pequeños y manejables. Es el antiguo principio de "divide y vencerás" aplicado a la informática.

Imagina que tienes que construir una casa. No puedes simplemente decir "construye una casa". Tienes que descomponer el problema en partes:

1. Excavar los cimientos.
2. Construir la estructura.
3. Instalar la fontanería.
4. Poner el tejado.
5. Pintar las paredes.

Cada una de estas sub-tareas es mucho más fácil de planificar y ejecutar que el proyecto completo. En programación, hacemos exactamente lo mismo. Un programa complejo se divide en funciones, módulos y clases más pequeñas, que podemos construir, probar y entender por separado.



1.4 Reconocimiento de Patrones: Encontrando lo que se repite

Una vez que hemos descompuesto un problema grande en piezas más pequeñas, a menudo notamos algo interesante: muchas de esas piezas se parecen entre sí.

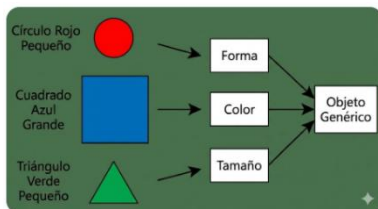
El **Reconocimiento de Patrones** es la habilidad de identificar similitudes o características comunes entre diferentes problemas o dentro de un mismo problema. Es crucial porque nos permite reutilizar soluciones.

Imagina que tienes que calcular el área de cien círculos diferentes.

- Podrías escribir el cálculo para el primer círculo: $3.14159 * \text{radio1} * \text{radio1}$.
- Luego para el segundo: $3.14159 * \text{radio2} * \text{radio2}$.
- Y así cien veces.

Eso es ineficiente y propenso a errores. Un programador reconoce el patrón: "todos los cálculos son iguales, solo cambia el valor del radio". Entonces, en lugar de escribir la misma fórmula cien veces, creamos una sola función que acepte el radio como entrada y devuelva el área, y usamos esa función para todos los círculos.

El reconocimiento de patrones es lo que nos lleva a crear funciones, usar bucles y diseñar estructuras de datos eficientes.



1.5 Ejercicio mental: Diseñando un algoritmo cotidiano

Ahora te toca a ti. Vamos a aplicar los conceptos de descomposición y diseño de algoritmos a una tarea que probablemente has hecho cientos de veces: **hacer un café**.

Este ejercicio no requiere ordenador. Su objetivo es entrenar tu mente para pensar en pasos secuenciales, lógicos y precisos.

El Problema: Tienes una cafetera de goteo estándar, café molido, agua y electricidad. Quieres preparar una taza de café. Diseña el algoritmo.

Instrucciones:

1. **Descompón el problema:** ¿Cuáles son las grandes etapas del proceso?
2. **Detalla los pasos:** Escribe las instrucciones para cada etapa. Sé tan preciso como sea posible. Recuerda que la "máquina" (tu amigo imaginario literal) no tiene sentido común. No digas "pon agua", di "abre el depósito y vierte agua hasta la marca de 2 tazas".
3. **Identifica patrones:** ¿Hay pasos que se repitan o que sean muy similares?
4. **Piensa en las condiciones:** ¿Qué pasa si no hay agua? ¿O si no hay café? ¿Tu algoritmo tiene en cuenta estos casos?

Propuesta de Solución (¡Inténtalo tú primero!):

Aquí tienes un ejemplo de cómo podría verse este algoritmo. Observa el nivel de detalle.

Algoritmo: Preparar Café (Versión 1.0)

1. Inicio del proceso.

2. Preparar la cafetera:

- Abre la tapa del depósito de agua.
- Toma la jarra de la cafetera.
- Llena la jarra con agua fría del grifo hasta la marca deseada (ej. 2 tazas).
- Vierte el agua de la jarra en el depósito de la cafetera.
- Coloca la jarra vacía de nuevo en su sitio, debajo del portafiltros.

3. Preparar el café:

- Abre el compartimento del portafiltros.
- ¿Hay un filtro viejo?
 - Si SÍ: Retíralo y deséchalo en la basura.
- Coloca un filtro de papel nuevo en el portafiltros.
- Toma la cuchara dosificadora.
- Por cada taza de agua que pusiste:
 - Llena la cuchara con café molido.
 - Vierte el café en el filtro.
- Cierra el compartimento del portafiltros.

4. Iniciar la preparación:

- Asegúrate de que la cafetera esté enchufada a la toma de corriente.
- Pulsa el botón de encendido ("ON").

5. Esperar y servir:

- Espera hasta que todo el agua haya pasado por el filtro y goteado en la jarra (el sonido de goteo se detendrá).
- Retira la jarra con cuidado.
- Sirve el café en una taza.
- ¿Deseas azúcar o leche?
 - Si Sí: Añade al gusto y remueve.
- Vuelve a colocar la jarra en la cafetera para mantener el resto caliente.

6. Fin del proceso. ¡Disfruta tu café!

Este ejercicio puede parecer trivial, pero es la esencia misma de la programación. Antes de escribir código, debes tener claro en tu mente el algoritmo que ese código va a implementar.

Hemos completado el Capítulo 1. Hemos aprendido qué es programar, qué son los algoritmos, la descomposición y el reconocimiento de patrones, y lo hemos aplicado en un ejercicio mental.

Capítulo 2: Tu Taller de Trabajo

Hasta ahora, hemos estado "programando" en nuestra cabeza y en papel. Pero para que una máquina ejecute nuestras instrucciones, necesitamos un entorno donde podamos escribir ese código y un "traductor" que lo convierta en algo que el ordenador entienda.

En este capítulo, vamos a configurar nuestro taller de programación: instalaremos Python (el traductor o intérprete) y Visual Studio Code (nuestro banco de trabajo para escribir código). Al final, escribiremos nuestro primer programa real.

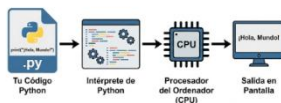
2.1 El papel del Intérprete: Cómo tu ordenador entiende Python

Recuerda lo que vimos en el capítulo anterior: las computadoras solo entienden código máquina (ceros y unos). Python, sin embargo, es un lenguaje de "alto nivel", diseñado para ser leído y escrito por humanos.

¿Cómo salvamos esa brecha? Con un **Intérprete**.

Un intérprete es un programa especial que hace de intermediario. Lee el código que tú escribes en Python y lo traduce a instrucciones que el procesador de tu ordenador puede ejecutar, todo esto sobre la marcha, línea por línea.

Cuando decimos "instalar Python", lo que realmente estamos haciendo es instalar este intérprete y un conjunto de herramientas básicas que lo acompañan.



2.2 Instalación de Python paso a paso

Vamos a instalar Python en tu ordenador. El proceso es sencillo, pero es importante seguir los pasos con cuidado.

Para usuarios de Windows:

1. Ve al sitio web oficial de Python: python.org.
2. Pasa el ratón sobre el menú "Downloads" y haz clic en el botón grande para descargar la última versión (por ejemplo, "Python 3.x.x").
3. Una vez descargado el archivo instalador (.exe), ejecútalo.
4. ¡MUY IMPORTANTE! En la primera pantalla del instalador, asegúrate de marcar la casilla que dice **"Add Python 3.x to PATH"** (o "Agregar Python 3.x al PATH"). Esto es crucial para que puedas ejecutar Python desde cualquier lugar en tu ordenador.
5. Haz clic en "Install Now" y espera a que termine el proceso.

Para usuarios de macOS:

1. Ve a python.org, menú "Downloads", y descarga la última versión para macOS.
2. Abre el paquete instalador (.pkg) descargado.
3. Sigue las instrucciones en pantalla, haciendo clic en "Continuar" y "Aceptar" en los pasos de licencia e instalación. Es un proceso estándar de instalación en Mac.
- 4.

Para usuarios de Linux:

Es muy probable que tu distribución ya venga con Python instalado. Para comprobarlo, abre una terminal y escribe:

```
python3 --version
```

Si te muestra una versión (como Python 3.10.6), ya lo tienes. Si no, puedes instalarlo usando el gestor de paquetes de tu distribución. Por ejemplo, en Ubuntu/Debian:

```
sudo apt update  
sudo apt install python3
```

Verificación de la instalación (para todos los sistemas)

Una vez finalizada la instalación, vamos a comprobar que todo funciona correctamente.

1. Abre una terminal (Símbolo del sistema o PowerShell en Windows, Terminal en macOS/Linux).
2. Escribe el siguiente comando y pulsa Enter:

```
python --version
```

(Nota: En algunos sistemas, especialmente macOS y Linux, es posible que tengas que escribir `python3 --version` en su lugar).

Si todo ha ido bien, deberías ver algo como esto:

Python 3.11.4

(La versión exacta puede variar, lo importante es que empiece por 3).

¡Felicidades! Ya tienes el "cerebro" de Python instalado en tu máquina. Ahora necesitamos una herramienta cómoda para escribir el código.

2.3 Tu herramienta de escritura: Instalación y configuración de Visual Studio Code (VS Code)

Podrías escribir código Python en el Bloc de Notas de Windows o en TextEdit de Mac, pero sería como intentar construir un mueble solo con una navaja suiza: posible, pero doloroso e ineficiente.

Para programar cómodamente, necesitamos un **Editor de Código** o un **Entorno de Desarrollo Integrado (IDE)**. Estas herramientas están diseñadas específicamente para programadores y ofrecen características como:

- **Resaltado de sintaxis:** Colorea las diferentes partes del código para que sea más fácil de leer.
- **Autocompletado:** Te sugiere comandos mientras escribes, ahorrándote tiempo y errores.
- **Detección de errores:** Te avisa si has cometido un error tipográfico antes incluso de ejecutar el programa.

Nosotros usaremos **Visual Studio Code (VS Code)**. Es gratuito, ligero, muy potente y es el editor más popular del mundo actualmente.

Paso 1: Descargar e instalar VS Code

1. Ve al sitio web oficial: code.visualstudio.com.
2. Detectará automáticamente tu sistema operativo. Haz clic en el botón de descarga grande.
3. Ejecuta el instalador descargado y sigue los pasos. Las opciones por defecto suelen estar bien.

Paso 2: Instalar la extensión de Python

VS Code es un editor "generalista". Para que se convierta en una herramienta potente para Python, necesitamos instalarle una "extensión" oficial.

1. Abre Visual Studio Code.
2. En la barra lateral izquierda, verás una serie de iconos. Haz clic en el que parece unos bloques de Tetris (o usa el atajo Ctrl+Shift+X en Windows/Linux, Cmd+Shift+X en Mac). Este es el menú de **Extensiones**.
3. En la barra de búsqueda que aparece, escribe "Python".
4. La primera opción debería ser la extensión oficial de Microsoft (tendrá decenas de millones de descargas). Haz clic en ella.
5. Haz clic en el botón azul pequeño que dice **"Install"**.



2.4 Tu primer programa: "¡Hola, Mundo!" y el ciclo de edición-ejecución

Ahora que tenemos el intérprete (Python) y el editor (VS Code) listos, es hora de hacer que trabajen juntos. Vamos a crear el programa más clásico de todos: el "¡Hola, Mundo!".

El objetivo de este programa es simplemente mostrar una frase en la pantalla. Es la prueba de fuego para confirmar que todo tu entorno de desarrollo está funcionando correctamente.

Paso 1: Crear una carpeta para tus proyectos

La organización es clave. Antes de escribir código, crea una carpeta en tu ordenador (por ejemplo, en "Documentos") llamada CursoPython o algo similar. Ahí guardaremos todos los programas que hagamos.

Paso 2: Crear tu primer archivo de código

1. Abre Visual Studio Code.
2. Ve a **Archivo > Abrir carpeta...** y selecciona la carpeta que acabas de crear.
3. En la barra lateral izquierda (el Explorador), verás el nombre de tu carpeta. Pasa el ratón por encima y haz clic en el icono de "Nuevo archivo" (el que parece una hoja de papel con un +).
4. Nombra el archivo como `hola.py`. ¡Es crucial que termine en `.py`! Esta extensión le dice a VS Code y a tu ordenador que es un archivo de código Python.

Paso 3: Escribir el código

En el área principal del editor, que ahora estará vacía, escribe la siguiente línea de código. Asegúrate de escribirla exactamente igual, incluyendo las comillas y los paréntesis:

```
print("¡Hola, Mundo! Mi primer programa funciona.")
```

¿Qué hace esta línea?

- `print`: Es una **función** incorporada de Python. Le dice al intérprete: "Muestra algo en la pantalla".
- `(...)`: Los paréntesis son como los brazos de la función. Dentro de ellos colocamos lo que queremos que la función utilice o muestre.
- `"..."`: Las comillas le dicen a Python que lo que hay dentro es texto literal (una cadena de caracteres o *string*). Sin las comillas, Python intentaría interpretar esas palabras como comandos.

Paso 4: Guardar y Ejecutar el programa

1. Guarda el archivo (Ctrl+S o Cmd+S).
2. Para ejecutarlo, la forma más fácil en VS Code es hacer clic en el pequeño icono de un triángulo de reproducción () que suele aparecer en la esquina superior derecha del editor.
3. Si no ves el icono, puedes hacer clic derecho en cualquier parte del código y seleccionar **"Run Python File in Terminal"** (Ejecutar archivo de Python en Terminal).

Paso 5: Ver el resultado

Si todo ha ido bien, verás que se abre una nueva área en la parte inferior de VS Code llamada **"Terminal"**. Después de unos comandos automáticos, debería aparecer el mensaje que escribiste:

```
¡Hola, Mundo! Mi primer programa funciona.
```

```
¡Felicidades! Acabas de completar el ciclo fundamental de la  
programación: Escribir -> Guardar -> Ejecutar -> Ver resultado.
```

A partir de ahora, este será el proceso que repetiremos constantemente. Si algo no funciona, el resultado en la terminal te dará pistas sobre qué ha ido mal (un "error"). Aprender a leer esos errores es una habilidad que desarrollaremos.

Con esto terminamos el **Capítulo 2: Tu Taller de Trabajo**. Ya tienes tu entorno listo y has ejecutado tu primer programa.

Capítulo 3: Almacenando Información - Variables y Tipos de Datos Básicos

Un programa que solo dice "¡Hola, Mundo!" está bien para empezar, pero es muy limitado. Los programas útiles necesitan recordar información: el nombre de un usuario, el precio de un producto, la puntuación en un juego, el resultado de un cálculo...

Para "recordar" cosas en programación, usamos **variables**.

3.1 ¿Qué es una variable? Las "cajas con etiqueta" para tus datos

Imagina que estás organizando una mudanza. Tienes un montón de objetos que necesitas guardar para transportarlos. ¿Qué haces? Usas cajas.

Pero si solo metes cosas en cajas sin más, cuando llegues a tu nueva casa será un caos. "¿Dónde están los cubiertos? ¿En qué caja metí los libros?". Para evitar esto, **pones una etiqueta** a cada caja describiendo lo que hay dentro: "Cocina - Cubiertos", "Salón - Libros".

En programación, una **variable** es exactamente eso: **un espacio en la memoria del ordenador (la caja) donde guardamos un dato, al que le asignamos un nombre (la etiqueta) para poder encontrarlo y usarlo después.**

Creando variables en Python (Asignación)

En Python, crear una variable es tan fácil como darle un nombre y usar el signo igual = para decirle qué guardar dentro. Esto se llama **asignación**.

```
# Creamos una variable llamada 'mensaje' y guardamos texto en ella
mensaje = "Hola, esto es una variable"
```

```
# Creamos una variable llamada 'puntuacion' y guardamos un número
puntuacion = 100
```

```
# Ahora podemos usar esas "cajas" por su nombre
print(mensaje)
print(puntuacion)
```

Salida:

```
Hola, esto es una variable
100
```

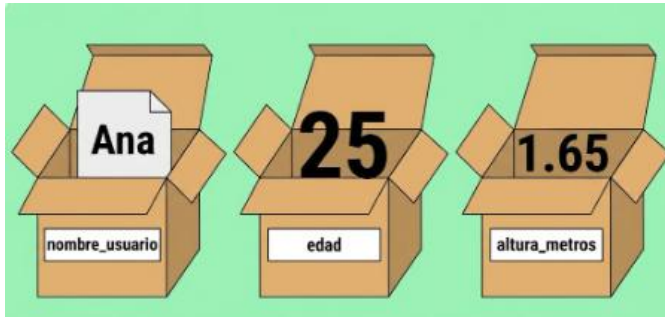
Cuando Python lee `print(mensaje)`, no imprime la palabra "mensaje". Busca la caja con esa etiqueta, mira lo que hay dentro y eso es lo que muestra.

Reglas para nombrar variables

Tú eliges los nombres de tus variables, pero hay algunas reglas y buenas prácticas:

1. **Deben ser descriptivos:** `x = "Juan"` funciona, pero `nombre_usuario = "Juan"` es mucho mejor. El código se lee más fácil.
2. **Solo pueden contener letras, números y guiones bajos (_).**
3. **No pueden empezar por un número.** (`1nombre` es incorrecto, `nombre1` es correcto).

4. **No pueden contener espacios.**
5. **Python distingue mayúsculas de minúsculas.** edad, Edad y EDAD son tres variables diferentes.
6. **Por convención en Python, usamos snake_case:** palabras en minúsculas separadas por guiones bajos (ej. total_a_pagar, fecha_nacimiento).



3.2 Tipos de Datos Esenciales

Python, como cualquier lenguaje de programación, necesita saber qué tipo de información estás guardando en una variable. Esto se debe a que no es lo mismo sumar dos números ($5 + 5 = 10$) que "sumar" dos textos ("Hola" + "Mundo" = "HolaMundo").

Los tipos de datos más básicos que usarás el 90% del tiempo son:

1. Números Enteros (int)

Son números sin parte decimal, positivos o negativos. Son perfectos para contar cosas (personas, días, vueltas en un bucle).

Ejemplos: 10, 0, -5, 1000000.

```
edad = 30
cantidad_visitas = 1500
temperatura_bajo_cero = -5
```

2. Números de Coma Flotante (float)

Son números con parte decimal. Se usan para medidas más precisas, cálculos científicos, dinero, etc. En Python, se usa un punto . para separar la parte entera de la decimal, no una coma.

Ejemplos: 3.14159, 0.5, -10.2, 100.0 (un número terminado en .0 también es un float).

```
pi = 3.14159
precio_producto = 19.99
altura_metros = 1.75
```

3. Cadenas de Texto (str)

Son secuencias de caracteres (letras, números, símbolos). Es la forma en que Python maneja el texto. **Siempre** deben ir entre comillas, simples ('...') o dobles ("...").

Ejemplos: "Hola Mundo", 'Python es genial', "12345" (esto es texto, no el número 12345), "¡@#\$\$%".

```
nombre = "Ana García"
mensaje_bienvenida = '¡Bienvenido a tu curso de Python!'
codigo_postal = "28001" # Los códigos postales se suelen tratar como texto
```

4. Booleanos (bool)

Este es el tipo de dato más simple, pero uno de los más importantes. Solo puede tener dos valores: Verdadero (True) o Falso (False). Se usan para tomar decisiones lógicas en el programa ("¿Es mayor de edad? Sí/No").

¡Importante! True y False en Python siempre empiezan con mayúscula.

```
es_mayor_de_edad = True
tiene_carnet_conducir = False
```

La función type()

A veces, no estás seguro de qué tipo de dato contiene una variable. Python tiene una función incorporada llamada type() que te lo dice.

```
edad = 30
precio = 19.99
nombre = "Ana"
es_estudiante = True
```

```
print(type(edad))
print(type(precio))
print(type(nombre))
```

```
print(type(es_estudiante))
```

Salida:

<class 'int'> (Es un entero)

<class 'float'> (Es un número de coma flotante)

<class 'str'> (Es una cadena de texto)

<class 'bool'> (Es un booleano)

Esta función es muy útil para depurar (encontrar errores) cuando tu código no se comporta como esperas.



3.3 Operaciones básicas: Matemáticas y uniendo texto

Las variables no sirven de mucho si solo se quedan ahí sentadas. La programación trata de transformar datos. Para ello, usamos **operadores**.

1. Operaciones Matemáticas (con int y float)

Python es una calculadora muy potente. Puedes realizar todas las operaciones aritméticas básicas utilizando los símbolos que ya conoces (o casi):

- **Suma:** +
- **Resta:** -
- **Multiplicación:** * (asterisco, no la 'x')
- **División:** / (barra inclinada)

Podemos operar directamente con números

```
print(5 + 10) # Salida: 15
```

```
print(100 - 50) # Salida: 50
```

```
print(4 * 5) # Salida: 20
```

¡Pero la gracia es usar variables!

```
precio_camiseta = 20
```

```
precio_pantalón = 30
```

```
descuento = 5
```

```
total_sin_descuento = precio_camiseta + precio_pantalón
```

```
total_final = total_sin_descuento - descuento
```

```
print(total_final) # Salida: 45
```

◆ Un detalle importante sobre la división

En Python 3, la división normal (/) **siempre** devuelve un número decimal (float), incluso si el resultado es exacto.

```
print(10 / 2) # Salida: 5.0 (fíjate en el .0)
```

Si quieres una **división entera** (que descarte los decimales), usa la doble barra //:

```
print(10 // 3) # Salida: 3 (descarta el .3333...)
```

Otro operador muy útil es el **Módulo** (%), que te da el **resto** de una división. Es genial para saber si un número es par o impar (si el resto de dividir entre 2 es 0, es par).

```
print(10 % 3) # Salida: 1 (porque 10 dividido entre 3 es 3, y sobra 1)
```

2. Operaciones con Texto: Concatenación

¿Qué pasa si intentas "sumar" dos textos? Python es listo y entiende que no quieres hacer matemáticas, sino unir las palabras. Esto se llama **concatenación**.

El operador para concatenar es el mismo símbolo de suma: +.

```
saludo = "Hola"  
nombre = "Ana"
```

```
# Unimos las dos cadenas  
mensaje = saludo + nombre  
print(mensaje) # Salida: HolaAna
```

¡Ups! Fíjate que los ha pegado literalmente. Python no añade espacios mágicamente. Si quieres un espacio, tienes que añadirlo tú como si fuera otro carácter más:

```
# Añadimos un espacio explícitamente en medio
```

```
mensaje_bonito = saludo + " " + nombre  
print(mensaje_bonito) # Salida: Hola Ana
```

⚠ ¡Cuidado con mezclar tipos! El operador + camaleónico

Aquí es donde el tipo de dato se vuelve crucial. El símbolo + se comporta de forma diferente dependiendo de quiénes sean sus vecinos.

- Si ve números a ambos lados, **suma**.
- Si ve texto a ambos lados, **concatena**.

Observa la diferencia:

```
numero_a = 5
numero_b = 5
print(numero_a + numero_b) # Salida: 10 (Matemáticas)
```

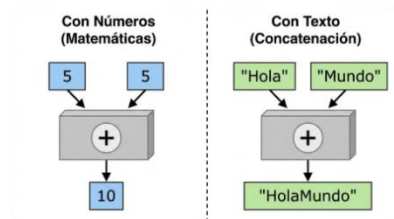
```
texto_a = "5"
texto_b = "5"
print(texto_a + texto_b) # Salida: 55 (Concatenación de texto)
```

¿Y si intentamos mezclar?

```
edad = 25
# ¡ERROR! No puedes sumar texto y números directamente
# print("Tengo " + edad + " años")
```

Si ejecutas esa línea comentada, Python te dará un error (TypeError), diciéndote básicamente: "No sé cómo sumar letras con números".

Para solucionar esto, necesitamos el siguiente concepto: la conversión de tipos o *casting*.



3.4 Conversión de Tipos (Casting): La metamorfosis de los datos

El *casting* es el superpoder de decirle a Python: "Sé que este dato es de un tipo, pero quiero que lo trates como si fuera de otro". Es como ponerle un disfraz a tu dato.

Para hacer esto, Python nos ofrece unas funciones con nombres muy intuitivos, que coinciden con los tipos de datos que ya conocemos: `str()`, `int()` y `float()`.

1. De cualquier cosa a Texto: La función `str()`

Esta es, quizás, la función de conversión más usada. Sirve para convertir casi cualquier valor en su representación de texto. Es la solución al problema que vimos en la sección anterior al intentar unir un número a una frase.

Problema anterior:

```
edad = 25
# print("Tengo " + edad + " años") <-- ¡ERROR!
```

Solución con casting: Convertimos el número 25 (que es un `int`) en el texto "25" (un `str`) antes de concatenarlo.

```
edad = 25
edad_como_texto = str(edad) # Ahora 'edad_como_texto' vale "25"
print("Tengo " + edad_como_texto + " años")
# Salida: Tengo 25 años
# ¡Podemos hacerlo todo en la misma línea!
print("El precio es de " + str(19.99) + " euros.")
# Salida: El precio es de 19.99 euros.
```

2. De Texto a Número Entero: La función int()

Esto es muy común cuando recibimos datos del usuario (que siempre llegan como texto) y queremos usarlos para hacer cálculos matemáticos.

```
texto_numero = "50"
# print(texto_numero + 10) <-- ¡ERROR! Estarías intentando sumar "50" + 10
# Convertimos el texto "50" al número entero 50
numero_real = int(texto_numero)
print(numero_real + 10)
# Salida: 60
```

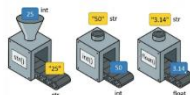
¡Mucho cuidado! La función int() es un poco exigente. El texto que le pases debe parecerse a un número entero.

- int("123") -> Funciona, te da el número 123.
- int("123.45") -> **¡ERROR!** Porque un entero no puede tener decimales.
- int("hola") -> **¡ERROR!** Porque "hola" no se parece en nada a un número.

3. De Texto (o Entero) a Número Flotante: La función float()

Funciona igual que int(), pero sirve para obtener números con decimales. Es más flexible porque puede convertir textos que parezcan enteros o decimales.

```
precio_texto = "19.99"
precio_numero = float(precio_texto)
print(precio_numero * 2) # Podemos hacer cálculos con él
# Salida: 39.98
# También funciona con textos que parecen enteros
print(float("10"))
# Salida: 10.0 (Fíjate que le añade el .0)
```



3.5 Ejercicio práctico: Calculadora de Índice de Masa Corporal (IMC)

El Índice de Masa Corporal (IMC) es una medida utilizada para determinar si una persona tiene un peso saludable para su estatura. La fórmula es muy sencilla:

$$IMC = \frac{\text{Peso (kg)}}{\text{Altura (m)}^2}$$

(El peso en kilogramos dividido por la altura en metros elevada al cuadrado).

Tu misión: Escribir un programa en Python que calcule el IMC de una persona dados su peso y su altura, y muestre el resultado por pantalla de forma amigable.

Pasos recomendados (Descomposición del problema):

1. **Define las variables de entrada:** Crea dos variables, una para el peso (ej. 70 kg) y otra para la altura (ej. 1.75 m). Piensa qué tipo de dato (int o float) es el más adecuado para cada una.
2. **Aplica la fórmula:** Crea una nueva variable para guardar el resultado del cálculo.
 - *Pista:* Para elevar un número al cuadrado en Python, puedes multiplicarlo por sí mismo (altura * altura) o usar el operador de potencia ** (ej. altura ** 2).
3. **Muestra el resultado:** Usa print() para mostrar un mensaje como: "Tu IMC es: [resultado]".
 - *Recuerda:* El resultado será un número, pero print necesita texto para unirlo a la frase. ¡Tendrás que usar *casting*!

¡Inténtalo tú primero!

Abre tu VS Code, crea un archivo llamado `calculadora_imc.py` e intenta escribir el código antes de mirar la solución..

Solución propuesta

Aquí tienes cómo podría quedar el código. Compara tu solución con esta. No te preocupes si no es idéntica; en programación hay muchas formas de llegar al mismo resultado.

```
# --- Calculadora de IMC ---
# 1. Definimos las variables con los datos de entrada
# Usamos 'float' para el peso por si alguien pesa 70.5 kg, por ejemplo.
peso_kg = 70.0
# Usamos 'float' porque la altura en metros tiene decimales.
altura_m = 1.75
# 2. Calculamos el IMC usando la fórmula
# altura_m ** 2 es lo mismo que decir altura_m elevado al cuadrado
imc = peso_kg / (altura_m ** 2)
# 3. Mostramos el resultado
# IMPORTANTE: Convertimos el número 'imc' a texto (str) para poder unirlo a la
# frase.
print("Para un peso de " + str(peso_kg) + " kg y una altura de " + str(altura_m) +
      " m:")
print("Tu IMC es: " + str(imc))
# Salida esperada:
# Para un peso de 70.0 kg y una altura de 1.75 m:
# Tu IMC es: 22.857142857142858
```

Reto extra (Opcional):

El resultado tiene demasiados decimales, ¿verdad? Python tiene una función llamada `round(numero, decimales)` para redondear. ¿Te atreves a buscar cómo funciona e implementarla para que el IMC solo muestre 2 decimales?

(Solución al reto extra: `imc_redondeado = round(imc, 2)`)

¡Felicidades! Has completado el **Capítulo 3**. Has pasado de no saber dónde guardar datos a crear una pequeña herramienta de cálculo funcional.

En el próximo capítulo, haremos que nuestros programas sean mucho más interactivos, permitiendo que el usuario introduzca sus propios datos en lugar de tener que escribirlos fijos en el código.

Capítulo 4: Interactuando con el Mundo - Entrada y Salida

La "Entrada y Salida" (I/O o Input/Output en inglés) es la forma en que un programa se comunica con el exterior.

- **Entrada (Input):** El programa recibe datos (del teclado, del ratón, de un archivo...).
- **Salida (Output):** El programa muestra datos (en la pantalla, en un archivo, por los altavoces...).

4.1 Mostrando resultados: La función print() a fondo

Ya conoces a print(), nuestra fiel amiga para mostrar cosas en pantalla. Pero hasta ahora solo hemos usado el 10% de su poder.

1. Imprimiendo múltiples cosas a la vez (la coma mágica)

Hasta ahora, para unir texto y variables, usábamos la concatenación con + y teníamos que convertir los números a texto manualmente con str(). ¡Qué pereza!

```
nombre = "Luis"
edad = 30
# La forma antigua y trabajosa:
print("Hola, me llamo " + nombre + " y tengo " + str(edad) + " años.")
```

print() tiene un superpoder: si separas las cosas con **comas**, él se encarga de unir las y, lo mejor de todo, **añade un espacio automáticamente entre ellas!** Además, no necesitas convertir los números a texto; print() es lo suficientemente listo para hacerlo solo.

```
nombre = "Luis"
edad = 30
# La forma moderna y fácil:
print("Hola, me llamo", nombre, "y tengo", edad, "años.")
# Salida exacta: Hola, me llamo Luis y tengo 30 años.
```

¡Fíjate qué limpio queda el código!

2. Personalizando el separador (sep)

Por defecto, cuando usas comas, print() pone un espacio. ¿Pero y si quieres separar los datos con otra cosa, como un guion, o que no haya separación?

Puedes usar un parámetro especial llamado sep (de "separador") al final de la función print.

```
dia = 25
mes = 12
anio = 2023
# Queremos mostrar la fecha con guiones: 25-12-2023
print(dia, mes, anio, sep="-")
# Queremos mostrarla con barras: 25/12/2023
print(dia, mes, anio, sep="/")
# Queremos pegarlo todo sin espacios: 25122023
print(dia, mes, anio, sep="")
```

3. Controlando el final de la línea (end)

¿Te has fijado que cada vez que usas print(), lo siguiente que se imprime aparece en la línea de abajo?

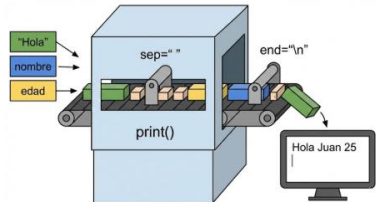
```
print("Hola")
print("Mundo")
# Salida:
# Hola
# Mundo
```

Esto pasa porque print() añade automáticamente un "salto de línea" invisible al final de cada mensaje.

Podemos cambiar este comportamiento usando el parámetro end.

```
# Queremos que "Mundo" aparezca en la misma línea, separado por tres
puntos
print("Hola", end="...")
print("Mundo")
# Salida:
# Hola...Mundo
# Un truco para barras de carga o contadores en la misma línea:
print("Cargando", end=" ")
```

```
print("10%", end=" ")
print("50%", end=" ")
print("¡Listo!")
# Salida: Cargando 10% 50% ¡Listo!
```



4.2 Preguntando al usuario: La función input()

Hasta ahora, nuestros programas tenían toda la información "programada" dentro. Si queríamos cambiar el nombre en el saludo, teníamos que cambiar el código.

Para hacer programas interactivos, necesitamos que el usuario pueda introducir información mediante el teclado mientras el programa se está ejecutando. La herramienta para esto es la función input().

¿Cómo funciona?

Cuando Python se encuentra con la función input():

1. **Pausa** la ejecución del programa por completo.
2. Muestra un cursor parpadeante en la terminal, esperando a que el usuario escriba algo.
3. Cuando el usuario pulsa la tecla **Enter**, el programa reanuda su marcha.
4. La función input() "recoge" todo lo que el usuario escribió antes de pulsar Enter y lo devuelve como un dato.

Para que esa información no se pierda, ¡debemos guardarla inmediatamente en una variable!

```
print("Programa de saludo iniciado...")
# El programa se detendrá aquí y esperará.
# Lo que escribas se guardará en la variable 'nombre_usuario'.
nombre_usuario = input()
print("Hola,", nombre_usuario, "¡encantado de conocerte!")
```

El mensaje de "Prompt" (la pregunta)

El ejemplo anterior funciona, pero es un poco seco. El usuario ve un cursor parpadeando y no sabe qué se espera que escriba.

Afortunadamente, `input()` nos permite poner un mensaje entre sus paréntesis para guiar al usuario. Esto se llama "prompt".

Un consejo de diseño: Añade siempre un espacio al final de tu pregunta para que lo que escriba el usuario no quede pegado al mensaje.

```
# Forma correcta y amable de preguntar:
nombre = input("Por favor, escribe tu nombre: ")
ciudad = input("¿En qué ciudad vives? ")
print("Genial,", nombre, ". He oído que", ciudad, "es muy bonita.")
```

Salida en la terminal:

```
Por favor, escribe tu nombre: Carlos
¿En qué ciudad vives? Madrid
Genial, Carlos . He oído que Madrid es muy bonita.
```

⚠ LA GRAN TRAMPA: Todo es texto

Aquí es donde el 99% de los principiantes tropiezan al principio. Presta mucha atención.

La función `input()` SIEMPRE, SIEMPRE devuelve un texto (cadena o `str`), sin importar lo que el usuario haya escrito.

Aunque el usuario escriba únicamente números, Python lo recibirá como si fueran letras entre comillas.

Mira este desastre:

```
edad = input("¿Cuántos años tienes? ")
# Imagina que el usuario escribe: 20
# Intentamos calcular la edad el año que viene
# print("El año que viene tendrás", edad + 1, "años.")

# ¡ERROR! Python te dirá: TypeError: can only concatenate str (not "int") to str
# Estás intentando sumar el texto "20" con el número 1.
```

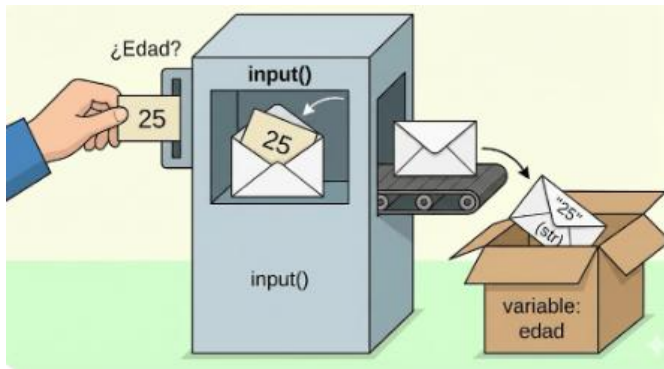
La solución: Combinar `input()` con Casting

Como ya aprendimos en el Capítulo 3, si queremos hacer matemáticas con la entrada del usuario, tenemos que convertir ese texto a número usando `int()` o `float()` inmediatamente después de recibirlo.

La forma más profesional y común de hacerlo es "envolviendo" el `input()` con la función de conversión:

```
# Pedimos un número entero (edad, cantidad de hijos, etc.)
edad_texto = input("¿Cuántos años tienes? ")
edad_numero = int(edad_texto) # Convertimos a entero
print("El año que viene tendrás", edad_numero + 1, "años.")
# Ahora sí funciona: suma 20 + 1 = 21.
# --- La forma "pro" en una sola línea ---
# Pedimos un número con decimales (precio, altura, temperatura)
```

```
altura = float(input("¿Cuánto mides en metros? (ej. 1.75): "))  
# Podemos operar directamente  
print("Tu altura en centímetros es:", altura * 100)
```



4.3 Formato de texto dinámico: Las F-strings (la forma moderna)

Hasta ahora, para mezclar texto y variables, hemos usado dos métodos:

1. **Concatenación con +:** Es tedioso porque hay que convertir los números a texto con `str()` y gestionar los espacios manualmente.

```
print("Hola, soy " + nombre + " y tengo " + str(edad) + " años.") # ¡Qué lío!
```

2. **Comas en `print()`:** Es mejor, pero te fuerza a tener un espacio entre cada elemento, lo cual no siempre quieres.

```
print("Hola, soy", nombre, "y tengo", edad, "años.") # Mejor, pero limitado.
```

Desde la versión 3.6 de Python, existe una tercera forma que es, sin duda, la favorita de los programadores modernos: las **F-strings** (Formatted strings o cadenas con formato).

¿Cómo funcionan?

Una F-string es una cadena de texto normal, pero con dos características especiales:

1. Empieza con una letra **f** (o **F**) justo antes de la primera comilla.
2. Dentro del texto, puedes usar llaves **{}** como marcadores de posición.

Lo mágico: Dentro de esas llaves, puedes escribir directamente el nombre de una variable. Python se encargará de sustituir esas llaves por el valor de la variable, je incluso convertirá los números a texto automáticamente!

Mira la diferencia:

```
nombre = "Lucía"
edad = 28
altura = 1.65
```

```
# La forma antigua y liosa:
# print("La usuaria " + nombre + " tiene " + str(edad) + " años y mide " +
#       str(altura) + "m.")
# --- LA FORMA MODERNA CON F-STRINGS ---
mensaje = f"La usuaria {nombre} tiene {edad} años y mide {altura}m."
print(mensaje)
# Salida: La usuaria Lucía tiene 28 años y mide 1.65m.
```

¿Ves lo limpio que queda? Es como escribir una plantilla y decirle a Python dónde rellenar los huecos.

¡Puedes hacer operaciones dentro de las llaves!

Las F-strings son muy potentes. No solo puedes poner variables, sino también expresiones completas.

```
producto = "Portátil"
precio_sin_iva = 800
```

iva = 0.21

Calculamos el total directamente dentro del mensaje

```
print(f"El precio final del {producto} es: {precio_sin_iva * (1 + iva)} euros.")
```

Salida: El precio final del Portátil es: 968.0 euros.



¡Fantástico! Hemos llegado al momento cumbre del Capítulo 4.

Ahora que dominas la entrada (input), la salida (print), la conversión de tipos (int, float) y el formato elegante con F-strings, tienes todas las herramientas necesarias para construir tu primer programa verdaderamente interactivo y útil.

Vamos a crear una herramienta que podrías usar en tu vida diaria.

4.4 Ejercicio práctico: Calculadora de Propinas Interactiva

La situación: Imagina que estás en un restaurante con amigos. Llega la cuenta y decidís dejar una propina. Quieres saber exactamente cuánto dinero es la propina y cuál es el total final a pagar, pero no quieres hacer los cálculos mentales.

Tu misión: Escribir un programa que le pregunte al usuario:

1. El total de la cuenta (puede tener decimales).
2. El porcentaje de propina que quiere dejar (por ejemplo, 10, 15, 20).

El programa debe calcular los importes y mostrarlos en pantalla con un formato claro y profesional usando F-strings.

Guía paso a paso (Descomposición del problema):

1. **Entrada 1 (La cuenta):** Usa `input()` para pedir el total de la cuenta. Recuerda: ¡la entrada es texto! ¿Qué función necesitas para convertirla en un número con decimales?
2. **Entrada 2 (El porcentaje):** Usa `input()` para pedir el porcentaje. ¿Qué tipo de dato numérico es el más adecuado aquí? (Suele ser un número entero, como 15).
3. **Procesamiento (Los cálculos):**
 - Calcula la cantidad exacta de propina: $(\text{porcentaje} / 100) * \text{total_cuenta}$.
 - Calcula el total final: $\text{total_cuenta} + \text{cantidad_propina}$.
4. **Salida (El resultado):** Usa F-strings para mostrar un resumen final. Por ejemplo: "La propina es X€ y el total a pagar es Y€".

¡Inténtalo tú primero!

Abre VS Code, crea un archivo llamado `calculadora_propinas.py` y trata de resolverlo. ¡No tengas miedo de cometer errores, es la mejor forma de aprender!.

Solución propuesta

Aquí tienes una posible solución. Fíjate en cómo usamos los comentarios para explicar cada paso y cómo las F-strings hacen que el código final sea muy limpio.

```
# --- Calculadora de Propinas Interactiva ---
print("=== Bienvenido a la Calculadora de Propinas ===")

# 1. ENTRADAS
# Pedimos el total de la cuenta y lo convertimos a float inmediatamente.
# Usamos un prompt claro que indica un ejemplo.
cuenta_texto = input("¿Cuál es el total de la cuenta? (ej. 45.50): ")
cuenta_float = float(cuenta_texto)
# Pedimos el porcentaje. Lo convertimos a int (asumimos que es un número
    entero como 10, 15).
porcentaje_texto = input("¿Qué porcentaje de propina quieres dejar? (ej. 15): ")
porcentaje_int = int(porcentaje_texto)

# 2. PROCESAMIENTO
# Calculamos cuánto dinero representa ese porcentaje.
# (Si el porcentaje es 15, multiplicamos la cuenta por 0.15)
cantidad_propina = cuenta_float * (porcentaje_int / 100)
# Calculamos el total final a pagar.
total_final = cuenta_float + cantidad_propina

# 3. SALIDA
# Usamos F-strings para mostrar los resultados.
# Dejamos una línea en blanco al principio (\n) para que se vea más ordenado.
print(f"\nResumen para una cuenta de {cuenta_float}€ con un
    {porcentaje_int}% de propina:")
```

```
print(f"- Cantidad de propina: {cantidad_propina}€")
print(f"- TOTAL FINAL A PAGAR: {total_final}€")
# Ejemplo de ejecución:
# ¿Cuál es el total de la cuenta? (ej. 45.50): 50
# ¿Qué porcentaje de propina quieres dejar? (ej. 15): 20
#
# Resumen para una cuenta de 50.0€ con un 20% de propina:
# - Cantidad de propina: 10.0€
# - TOTAL FINAL A PAGAR: 60.0€
```

💡 Un truco "Pro" para las F-strings: Formato de decimales

Si pruebas el programa con números raros (por ejemplo, cuenta de 33.33 y propina del 15%), es posible que el resultado te salga con muchos decimales feos, como 4.9995€. Cuando hablamos de dinero, solo queremos dos decimales.

Las F-strings tienen un truco maravilloso para esto. Dentro de las llaves, después de la variable, puedes añadir dos puntos : y un código de formato.

Para redondear a **2** decimales flotantes, el código es `:.2f`.

Mira cómo mejora la salida:

```
# ... (código anterior) ...
# 3. SALIDA MEJORADA CON FORMATO
print(f"\nResumen para una cuenta de {cuenta_float:.2f}€:")
print(f"- Cantidad de propina: {cantidad_propina:.2f}€")
# ¡Mira qué bonito queda el total!
print(f"- TOTAL FINAL A PAGAR: {total_final:.2f}€")
```

Si usas este truco, incluso si el cálculo da 4.9995, la F-string mostrará automáticamente 5.00. ¡Magia!

Capítulo 5: Tomando Decisiones (Lógica Condicional)

La vida está llena de decisiones basadas en condiciones:

- **SI** llueve, cojo el paraguas.
- **SI** tengo hambre, como algo.
- **SI** la nota es mayor o igual a 5, apruebo el examen.

En programación, esto se llama **lógica condicional**, y es la base de la "inteligencia" de cualquier software.

5.1 El guardián de la puerta: La sentencia if

La estructura más básica para tomar una decisión en Python es la sentencia `if` (que significa "si..." en inglés, como condicional).

Funciona como un portero de discoteca: mira una condición y, solo si se cumple, te deja pasar a ejecutar un bloque de código específico.

La sintaxis del `if`

Fíjate bien en la estructura, porque introduce un concepto nuevo y VITAL en Python: la **indentación** (o sangría).

```
condicion = True
if condicion:
    # ESTE BLOQUE TIENE SANGIRÍA (4 espacios)
    print("La condición es verdadera.")
    print("Este código se ejecuta.")
# ESTE BLOQUE YA NO TIENE SANGRÍA
```

```
print("Este código está fuera del if y se ejecuta siempre.")
```

Puntos clave:

1. **La palabra clave if:** Indica que empieza una decisión.
2. **La condición:** Es una expresión que Python evaluará. **Siempre** tiene que dar como resultado un **Booleano** (True o False).
3. **Los dos puntos (:):** ¡Son obligatorios! Indican que termina la condición y empieza el bloque de acción.
4. **La INDENTACIÓN (Sangría):** Esto es fundamental. Todas las líneas de código que quieras que se ejecuten *solo si* la condición es cierta, **deben** estar movidas hacia la derecha (normalmente con la tecla Tabulador o 4 espacios).

Si la condición es False, Python salta por encima de todo el bloque indentado y continúa con el código que está pegado al margen izquierdo.

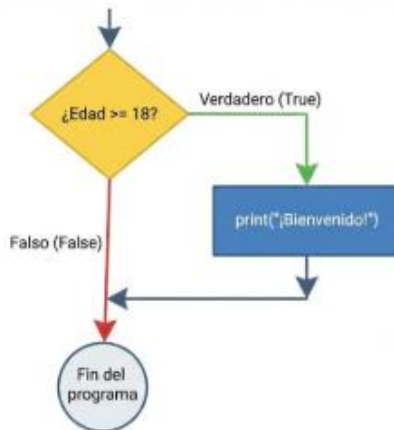
Ejemplo práctico: El portero de discoteca

Vamos a escribir un programa que pregunte la edad al usuario y decida si puede entrar o no a un local para mayores de 18 años.

```
# Pedimos la edad y la convertimos a entero
edad = int(input("¿Cuántos años tienes? "))

# Aquí está la decisión.
# La condición es "edad >= 18" (¿es la edad mayor o igual a 18?)
if edad >= 18:
    # Este bloque solo se ejecuta si la respuesta es Sí (True)
    print("¡Bienvenido! Puedes pasar.")
    print("Disfruta de la fiesta.")
print("Fin del programa de control de acceso.")
```

Si ejecutas esto e introduces 20, verás los mensajes de bienvenida. Si introduces 15, el programa saltará directamente a "Fin del programa...".



5.2 La alternativa: else (Si no...)

Volvamos al ejemplo del portero de discoteca. El código anterior era un poco maleducado con los menores de edad: si tenías 15 años, el programa simplemente se quedaba callado y terminaba.

Lo ideal es que el portero diga: "**SI** eres mayor de 18, pasas. **SI NO**, te digo que no puedes entrar".

Para esto sirve la sentencia else ("si no" o "en otro caso"). Es el bloque de código que se ejecuta **exclusivamente** cuando la condición del if resulta ser **Falsa (False)**.

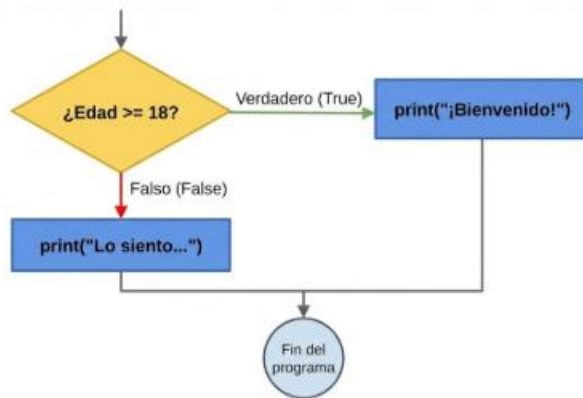
Sintaxis de if-else

Observa cómo el else se alinea verticalmente con el if (sin sangría), lleva dos puntos al final, y su propio bloque de código va indentado.

```
# Pedimos la edad
edad = int(input("¿Cuántos años tienes? "))
# Primera bifurcación: El camino del SÍ
if edad >= 18:
    # Bloque IF (se ejecuta si es True)
    print("¡Bienvenido! Puedes pasar.")
    print("Toma una pulsera.")
# Segunda bifurcación: El camino del NO
else:
    # Bloque ELSE (se ejecuta si es False)
    # Fíjate que 'else' NO lleva condición propia.
    print("Lo siento, eres menor de edad.")
    print("No puedes entrar esta noche.")
# Este código ya está fuera de los dos bloques y se ejecuta siempre
print("--- Fin del control ---")
```

Puntos clave del else:

1. **Nunca va solo:** Siempre tiene que haber un if justo encima.
2. **No tiene condición:** No puedes escribir `else edad < 18:`. El else es simplemente "todo lo que no haya atrapado el if anterior".
3. **Son mutuamente excluyentes:** O se ejecuta el bloque del if, o se ejecuta el bloque del else. **Jamás se ejecutarán los dos a la vez.** Es como llegar a una bifurcación en el camino: tienes que elegir izquierda o derecha, no puedes ir por los dos lados.



¡Perfecto! Ya dominamos el "blanco o negro". Pero el mundo real suele tener más matices.

5.3 Más de dos opciones: La sentencia elif (Si no, si...)

Imagina que estás programando una máquina expendedora. No te vale con saber si el usuario ha puesto dinero o no (if/else). Necesitas saber *cuánto* ha puesto para darle un producto u otro.

Si tiene 1€, le das agua. **SI NO**, si tiene 1.50€, le das un refresco. **SI NO**, si tiene 2€, le das un bocadillo. **SI NO** (cualquier otra cantidad menor), le dices que no le llega.

Para esta cadena de decisiones usamos elif, que es una abreviatura de "else if" (en español: "si no, si...").

¿Cómo funciona la cadena if-elif-else?

Es como una serie de puertas de seguridad en fila:

1. Python llega al primer if y evalúa la condición.
 - Si es True, entra, ejecuta su código y **salta al final de toda la cadena.**
 - Si es False, pasa a la siguiente puerta (elif).
2. Evalúa la condición del primer elif.
 - Si es True, entra, ejecuta y salta al final.
 - Si es False, pasa al siguiente...
3. Si llega al final y ninguna condición anterior se ha cumplido, ejecuta el bloque else (si lo hay).

¡Importante! En una cadena if-elif-else, **solo se ejecutará UNO de los bloques**, el primero que cumpla la condición.

Ejemplo: Calificando un examen

Vamos a asignar una nota (A, B, C, F) basándonos en una puntuación numérica de 0 a 100.

```
# Pedimos la puntuación
puntuacion = int(input("¿Qué nota has sacado (0-100)? "))
# Empieza la cadena de decisiones
if puntuacion >= 90:
    # Solo entra si la nota es 90, 91, ..., 100
    print("¡Excelente! Tienes una A.")
elif puntuacion >= 80:
    # Solo entra si la anterior falló (es menor que 90) Y esta es cierta (es >= 80)
```

```
# Por ejemplo, con un 85 entraría aquí.  
print("Muy bien. Tienes una B.")
```

```
elif puntuacion >= 70:
```

```
# Solo entra si es menor que 80 Y mayor o igual a 70.
```

```
print("Bien. Tienes una C.")
```

```
else:
```

```
# Si ha llegado hasta aquí, es porque todas las anteriores fueron falsas.
```

```
# Es decir, la nota es obligatoriamente menor que 70.
```

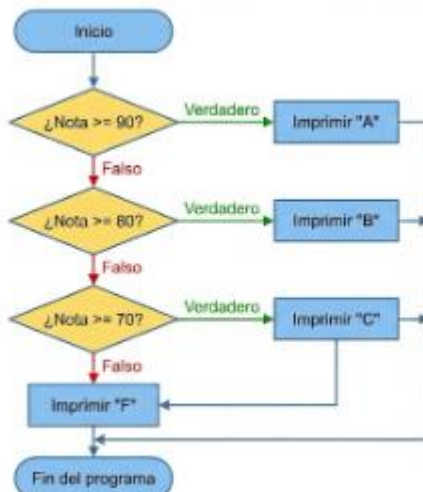
```
print("Necesitas estudiar más. Tienes una F.")
```

```
print("--- Fin de la calificación ---")
```

Prueba mental: Si introduces un 85:

1. if puntuacion >= 90: ¿85 >= 90? Falso. Pasa al siguiente.
2. elif puntuacion >= 80: ¿85 >= 80? **Verdadero**. Entra al bloque, imprime "Tienes una B" y **salta** hasta "Fin de la calificación".

He preparado un diagrama de flujo para que veas visualmente cómo el programa va "cayendo" por las diferentes condiciones hasta encontrar la correcta.



5.4 Los Jueces del Código: Operadores de Comparación

Los operadores de comparación son símbolos matemáticos que utilizamos para comparar dos valores. El resultado de usar cualquiera de estos operadores **siempre** es un valor Booleano: True (Verdadero) o False (Falso).

Son los ladrillos con los que construimos las condiciones de nuestros if.

Aquí tienes la lista de los más importantes. Presta especial atención al primero, es el error número uno de los principiantes.

Operador	Significado	Ejemplo	Resultado
==	Igual a (¿Son idénticos?)	5 == 5	True
!=	Diferente de (¿Son distintos?)	5 != 10	True
>	Mayor que	10 > 5	True
<	Menor que	5 < 10	True
>=	Mayor o igual que	10 >= 10	True
<=	Menor o igual que	5 <= 8	True

⚠ LA TRAMPA MORTAL: = vs ==

Este es el concepto más importante de esta sección. Grábatelo a fuego en la memoria:

- **Un solo igual (=): Es ASIGNACIÓN.** Sirve para **guardar** un valor en una variable. Es una orden: "¡Variable X, ahora vales 5!".
- **Doble igual (==): Es COMPARACIÓN.** Sirve para **preguntar** si dos cosas son iguales. Es una pregunta: "¿Es el valor de X idéntico al valor de Y?".

Ejemplo del desastre:

```
nota = 5
# ERROR COMÚN: Usar un solo '=' en un if
# if nota = 10: <-- Esto dará un error de sintaxis (SyntaxError)
# Python se queja: "No puedes asignar valores dentro de una pregunta if"
# FORMA CORRECTA: Usar el doble '=='
if nota == 10:
    print("¡Matrícula de honor!")
else:
    print("No es una matrícula.")
```

Ejemplos en acción

Vamos a usar estos operadores en un contexto real.

```
usuario_registrado = "admin"
contrasena_correcta = "1234"

usuario_input = input("Usuario: ")
contrasena_input = input("Contraseña: ")
# Usamos '==' para verificar si el texto introducido es idéntico al guardado
if usuario_input == usuario_registrado:
    print("Usuario correcto.")
# Podemos anidar (meter dentro) otro if
```

```

if contraseña_input == contraseña_correcta:
    print("¡Acceso concedido!")
else:
    # Usamos '!=' para indicar diferencia
    print("Contraseña incorrecta.")
else:
    print("El usuario no existe.")

```



¡Estupendo! Ya sabemos hacer preguntas simples ("¿Es A igual a B?"). Pero la vida real es más compleja. A veces, para tomar una decisión, necesitamos que se cumplan varias condiciones a la vez.

Por ejemplo, para entrar en una atracción de feria, quizás necesites: tener más de 12 años Y medir más de 1.40m.

Para combinar condiciones, Python nos ofrece los Operadores Lógicos. Son palabras en inglés muy intuitivas: and, or y not.

5.5 Combinando condiciones: Operadores Lógicos (and, or, not)

Estos operadores funcionan como el pegamento que une varias preguntas de "Verdadero/Falso" para formar una súper-pregunta final.

1. El operador and (Y)

Es el más exigente de todos. Para que la condición final sea True, TODAS las condiciones individuales unidas por and deben ser True. Si falla una sola, todo se va al traste y el resultado final es False.

Analogía: Imagina un pasillo con varias puertas de seguridad seguidas. Solo puedes llegar al final si todas las puertas están abiertas.

Ejemplo: Requisitos para una beca

edad = 22

nota_media = 8.5

ingresos_familiares = 20000

Queremos saber si cumple TRES condiciones a la vez:

1. Menor de 25 años

2. Nota media superior a 8

3. Ingresos inferiores a 30000

if edad < 25 and nota_media > 8 and ingresos_familiares < 30000:

Solo entra aquí si las TRES partes son True

print("¡Felicidades! Cumples los requisitos para la beca.")

else:

print("Lo sentimos, no cumples todos los requisitos.")

En este caso: True and True and True -> Resultado final: True

Si cambiáramos nota_media = 7, la segunda condición sería False, y aunque las otras dos fueran ciertas, el resultado total del if sería False.

2. El operador or (O)

Es el más flexible. Para que la condición final sea True, basta con que AL MENOS UNA de las condiciones individuales sea True. Solo si todas son falsas, el resultado final será False.

Analogía: Imagina un recinto con varias puertas de entrada en paralelo. Puedes entrar si cualquiera de ellas está abierta.

```
# Ejemplo: ¿Es fin de semana?
dia = input("¿Qué día de la semana es hoy? (lunes, martes...): ").lower()
# Queremos saber si es sábado O domingo
if dia == "sábado" or dia == "domingo":
    # Entra si se cumple ALGUNA de las dos
    print("¡Bien! Es fin de semana, a descansar.")
else:
    # Solo llega aquí si no es ni sábado ni domingo
    print("Toca trabajar, es día de diario.")
```

3. El operador not (NO)

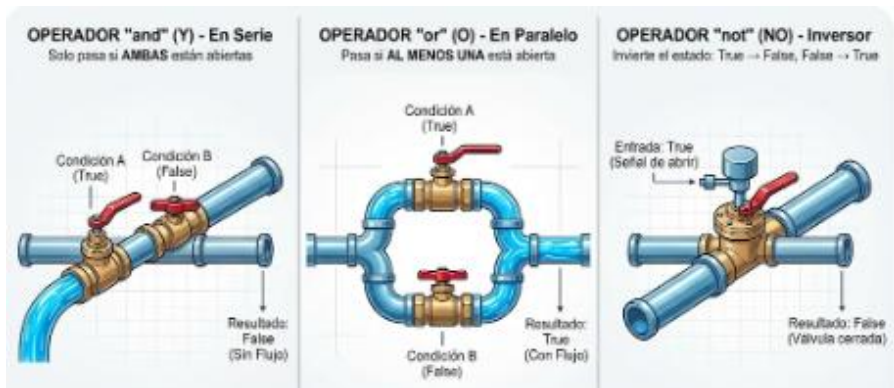
Es el "llevacontrarias". Simplemente invierte el valor de una condición. Si algo es True, el not lo convierte en False, y viceversa. Se coloca justo delante de la condición que queremos negar.

Analogía: Un portero que hace exactamente lo contrario de lo que le dice la norma.

```
esta_lloviendo = True
# Preguntamos: "SI NO está lloviendo..."
if not esta_lloviendo:
    # (not True) se convierte en False. No entramos aquí.
    print("Vamos a la playa.")
else:
    # Entramos aquí
```

```
print("Mejor nos quedamos en casa viendo una peli.")
```

```
# Ejemplo práctico: Usuariobaneado
usuario_baneado = False
# "Si el usuario NO está baneado, permítele escribir"
if not usuario_baneado:
    print("Puedes enviar mensajes.")
else:
    print("Tu cuenta está bloqueada.")
```



¡Llegamos al gran final del Capítulo 5!

Has aprendido a hacer preguntas simples (if), a tener planes alternativos (else), a manejar múltiples opciones (elif), a comparar valores (==, >, <) y a combinar condiciones como un profesional de la lógica (and, or, not).

Es hora de juntar todas estas piezas para crear un programa que realmente "piense".

5.6 Ejercicio Práctico: El Guardián del Bosque Mágico

Vamos a crear un pequeño juego de rol textual.

La situación: Eres un aventurero que intenta entrar en el Bosque Mágico de Eldoria. En la entrada hay un Guardián Elfo muy estricto que solo deja pasar a quienes cumplan DOS requisitos indispensables:

Ser un adulto (tener 18 años o más).

Poseer el "Amuleto de Roble" (una reliquia mágica).

Tu misión: Escribir un programa que actúe como este Guardián. Debe hacer las preguntas necesarias al usuario y, basándose en sus respuestas, decidir si le deja pasar o le deniega la entrada, explicando el motivo.

Guía paso a paso (Descomposición del problema):

Entrada 1 (La Edad): Pide la edad al usuario. ¡Recuerda convertirla a número entero (int)!

Entrada 2 (El Amuleto): Pregunta si tiene el amuleto. Dile que responda "si" o "no".

Pista Pro: Los usuarios son impredecibles. Pueden escribir "Si", "si", "SI". Para que tu programa no falle por esto, puedes añadir `.lower()` al final del `input()`. Esto convierte lo que escriban a minúsculas antes de guardarlo.

Ejemplo: `respuesta = input("¿Sí o no? ").lower()`

Lógica Principal (El éxito): El camino ideal es que cumpla ambas condiciones. ¿Qué operador lógico necesitas para unir "`edad >= 18`" y "`respuesta amuleto es igual a 'si'`"?

Lógicas Alternativas (Los fallos):

¿Qué pasa si es menor de 18 años? (El Guardián le dice que vuelva cuando crezca).

¿Qué pasa si es mayor de edad pero NO tiene el amuleto? (El Guardián le dice que le falta la llave).

¡Inténtalo tú primero!

Abre VS Code, crea el archivo guardian_bosque.py y ponte en la piel del elfo.
¡A programar!

.....

Solución propuesta

Aquí tienes una forma de resolverlo. Fíjate en cómo el uso de and, elif y los comentarios hacen que la lógica sea fácil de seguir.

```
print("--- La Puerta del Bosque Mágico ---")
print("Un alto elfo con armadura plateada te bloquea el paso.")
# 1. ENTRADAS
# Pedimos la edad y convertimos a entero.
edad_viajero = int(input("\nGuardián: 'Alto ahí. ¿Qué edad tienes, viajero? '"))
# Pedimos sobre el amuleto.
# Usamos .lower() para que si escribe "Si" o "SI", se guarde como "si".
tiene_amuleto = input("Guardián: '¿Y posees el Amuleto de Roble? (responde si/no) '").lower()
print("\n*El Guardián te mira fijamente mientras evalúa tus respuestas...*\n")
```

2. LÓGICA CONDICIONAL

--- CASO DE ÉXITO TOTAL ---

Necesitamos que AMBAS cosas sean verdad: edad >= 18 Y respuesta == "si"

if edad_viajero >= 18 and tiene_amuleto == "si":

print("Guardián: 'Veo sabiduría en tus ojos y portas la llave correcta.'")

print("Guardián: '¡Adelante, héroe! Las puertas de Eldoria se abren para ti.'")

print("*Las inmensas puertas de madera mágica se abren lentamente*")

--- CASO DE FALLO 1: Es demasiado joven ---

Si la primera condición falló, comprobamos si el problema es la edad.

elif edad_viajero < 18:

Usamos una f-string para personalizar el mensaje

print(f"Guardián: 'Aún eres joven, solo tienes {edad_viajero} años.'")

print("Guardián: 'El bosque es peligroso. Vuelve cuando seas un adulto.'")

--- CASO DE FALLO 2: Es adulto, pero le falta el objeto ---

Si llegamos al 'else', significa que NO entró en el primer 'if' (éxito)

y TAMPOCO entró en el 'elif' (no es menor de edad).

Por descarte, sabemos que es mayor de edad pero NO tiene el amuleto.

else:

print("Guardián: 'Tienes la edad necesaria para entrar, pero te falta la llave.'")

print("Guardián: 'Sin el Amuleto de Roble, no puedo dejarte pasar.

Búscalos.'")

print("\n--- Fin del encuentro ---")

Prueba el programa varias veces:

Edad 20, Amuleto "si": ¡Debería entrar!

Edad 15, Amuleto "si": Debería rechazarle por joven (el amuleto no importa si eres menor).

Edad 25, Amuleto "no": Debería rechazarle por falta del amuleto.

Capítulo 6: El poder de la repetición (Bucles)

Un bucle (o loop en inglés) es una estructura de código que permite repetir un bloque de instrucciones varias veces. Es como decirle a tu programa: "Haz esto otra vez, y otra vez, y otra vez... hasta que yo te diga que pares".

Python tiene dos tipos principales de bucles: while (mientras) y for (para). Empezaremos por el que más se parece a lo que ya conoces.

6.1 El bucle while: Repetir mientras sea verdad

Piensa en el bucle while como un hermano hiperactivo de la sentencia if.

Un if dice: "Si esta condición es verdad, ejecuto este bloque una vez".

Un while dice: "Mientras esta condición sea verdad, sigo ejecutando este bloque una y otra vez".

La estructura de un while

La sintaxis es casi idéntica a la de un if. Necesita una condición (que sea True o False), dos puntos y un bloque de código indentado.

```
# Imagina que estás comiendo cereales
tengo_hambre = True
while tengo_hambre:
```

```
# Este bloque se repite una y otra vez
print("Como una cucharada de cereales.")
print("¡Qué ricos!")
# ... ¿Qué falta aquí? ...
print("Ya estoy lleno, dejo de comer.")
```

⚠ PELIGRO: El Bucle Infinito

Si ejecutas el código de arriba tal cual... ¡tu ordenador se volverá loco imprimiendo "Como una cucharada..." para siempre! Tendrás que forzar el cierre del programa (en VS Code, suele ser con Ctrl + C en la terminal).

¿Por qué? Porque la condición `tengo_hambre` es `True` al principio, y nada dentro del bucle la cambia. El programa entra en un ciclo sin fin.

La regla de oro del `while`: Para que un bucle `while` termine alguna vez, algo debe cambiar dentro del bucle que haga que la condición acabe siendo `False`.

El ejemplo clásico: La cuenta atrás

Vamos a arreglar el ejemplo anterior usando una variable contadora para controlar las repeticiones.

```
# Inicializamos una variable de control
contador = 5
print("Iniciando cuenta atrás...")
# La condición: Mientras el contador sea mayor que 0
while contador > 0:
    # 1. Hacemos la acción
    print(f"Quedan {contador} segundos...")
    # 2. ¡PASO CRUCIAL! Actualizamos la variable de control.
    # Restamos 1 al contador en cada vuelta.
    # contador = contador - 1 <-- Forma larga
    contador -= 1      # <-- Forma corta y profesional
print("¡DESPEGUE! 🚀")
```

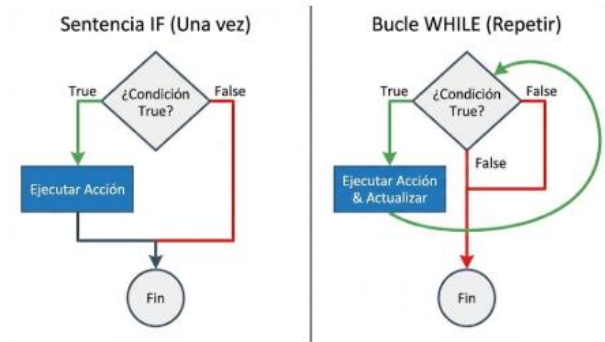
¿Qué sucede paso a paso?

contador es 5. ¿5 > 0? Sí (True). Entra al bucle. Imprime "Quedan 5...".
contador pasa a ser 4.

Vuelve arriba. ¿4 > 0? Sí. Imprime "Quedan 4...". contador pasa a ser 3.

... se repite para 3, 2, 1 ...

Vuelve arriba. contador es 0. ¿0 > 0? No (False). El bucle termina y salta a la línea "¡DESPEGUE!".



6.2 El bucle `for`: Repetir para cada elemento

El bucle `for` (para) en Python es ligeramente distinto al de otros lenguajes de programación, y es mucho más potente y fácil de leer.

En lugar de decirle "repite 10 veces", en Python le decimos: **"Para cada elemento que haya en esta colección, haz esto"**.

Es como si tuvieras una lista de la compra y fueras recorriendo los productos uno a uno: "Para cada artículo en mi lista, lo meto en el carro". Sabes exactamente cuándo vas a terminar: cuando se acabe la lista.

1. Recorriendo texto (Strings)

Una cadena de texto es, en el fondo, una colección ordenada de letras. El bucle `for` es perfecto para deletrear.

La sintaxis: `for <variable_temporal> in <coleccion>:`

```
nombre = "Python"
# Traducción: "Para cada 'letra' que encuentres en la palabra 'Python'..."
for letra in nombre:
    # En cada vuelta, la variable 'letra' toma un valor distinto:
    # Vuelta 1: letra = "P"
    # Vuelta 2: letra = "y"
    # ... etc.
    print(f"Dame una '{letra}'!")
print("¡PYTHON!")
```

¡Fíjate qué limpieza! No hay contadores que sumar ni condiciones complejas. Python se encarga de todo el trabajo sucio de pasar al siguiente elemento.

2. Repetir un número de veces: La función `range()`

"Vale", me dirás, "pero ¿y si yo solo quiero repetir algo 5 veces, como con el `while`, pero más fácil?".

Para eso, Python tiene una función mágica llamada `range()` (rango). Esta función genera una secuencia de números sobre la marcha.

- `range(5)` genera los números: 0, 1, 2, 3, 4 (Empieza en 0 y genera 5 números en total).

¡Ojo! En programación, casi siempre se empieza a contar desde 0, y el número final que pones en el `range` **no se incluye**.

```
print("Empieza el entrenamiento...")

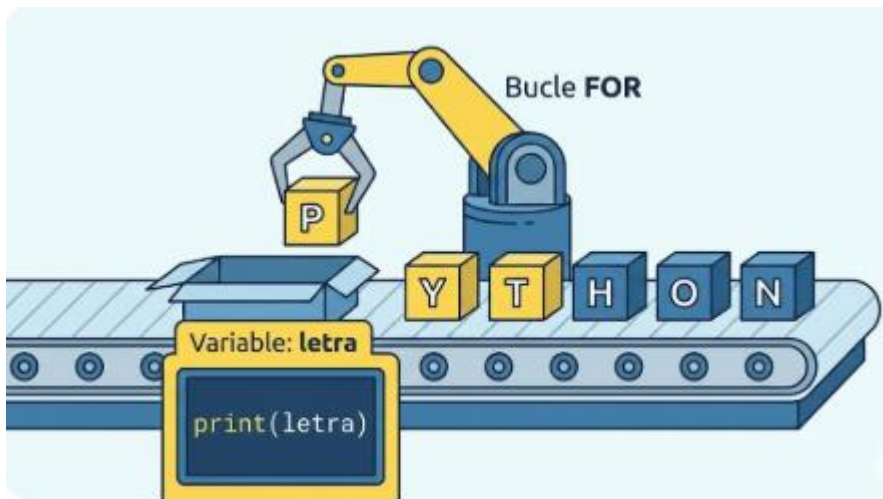
# Queremos hacer 5 flexiones.
# range(5) crea la secuencia [0, 1, 2, 3, 4]
for repeticion in range(5):
    # La variable 'repeticion' irá tomando los valores 0, 1, 2, 3, 4
    # Como queremos contar de forma humana (1 a 5), sumamos 1 al mostrarlo.
    print(f"¡Flexión número {repeticion + 1} hecha!")

print("Entrenamiento terminado.")
```

¿Cuándo usar `while` y cuándo usar `for`?

Esta es una duda muy común. Aquí tienes una regla general:

- **Usa `for` (el 90% de las veces):** Cuando sepas de antemano cuántas veces quieres repetir algo, o cuando quieras recorrer una colección de datos (una lista de nombres, las letras de una palabra, un rango de números...). Es más seguro porque es difícil crear un bucle infinito.
- **Usa `while`:** Cuando **NO** sepas cuándo va a terminar la repetición, porque depende de una condición externa (por ejemplo, "repite hasta que el usuario escriba 'salir'", o "repite mientras el sensor de temperatura marque menos de 20 grados").



Has aprendido a tomar decisiones (`if/else`) y a repetir tareas (`while/for`). Por separado son útiles, pero **juntas son imparables**. Casi cualquier programa complejo que imagines (un videojuego, una app del banco, un robot) se basa en combinar estas dos estructuras.

Vamos a crear tu primer juego interactivo real.

6.3 Ejercicio Práctico: El Juego de las Adivinanzas

La situación: Tu ordenador se aburre y quiere jugar contigo. Ha pensado un número secreto entre el 1 y el 20, y tú tienes que adivinarlo.

La mecánica: El programa te pedirá que introduzcas un número.

- Si aciertas, te felicitará y el juego terminará.
- Si fallas, el programa **te dará una pista** (te dirá si el número secreto es mayor o menor que el que has dicho) y **volverá a preguntarte**.
- El juego no termina hasta que aciertes.

Tu misión: Escribir este programa utilizando un bucle para repetir la pregunta y condicionales para comprobar la respuesta y dar pistas.

Guía paso a paso (Descomposición del problema):

1. **Preparación:** Define una variable con el número secreto (por ejemplo, `numero_secreto = 13`).
2. **La variable de control:** Necesitas una variable para guardar el intento del usuario. Inicialízala con un valor que seguro que no es el secreto (por ejemplo, `intento = 0`) para que la primera vez la condición del bucle sea cierta.
3. **El Bucle:** ¿Sabes cuántas veces va a necesitar el usuario para adivinarlo? No. Entonces, ¿qué bucle es mejor, `for` o `while`? (Pista: Necesitas repetir *mientras* no acierte).
 - La condición del bucle debería ser: "`Mientras el intento sea diferente de (!=) numero_secreto`".
4. **Dentro del bucle:**

- Pide un número al usuario y conviértelo a entero (`int`). ¡Guárdalo en la variable `intento` para actualizarla!
 - Usa una estructura `if/elif` para comparar `intento` con `numero_secreto`.
 - Si `intento < numero_secreto`: Imprime una pista ("Es mayor").
 - Si `intento > numero_secreto`: Imprime una pista ("Es menor").
 - *Nota*: No necesitas un `else` para cuando acierte dentro del bucle, porque si acierta, la condición del `while` será falsa en la siguiente vuelta y el bucle terminará solo.
5. **Final del juego**: Fuera del bucle (cuando ya haya terminado), escribe un mensaje de felicitación.

¡Inténtalo tú primero!

Abre VS Code, crea el archivo `adivinanza.py` y ponte a ello. ¡Es muy satisfactorio cuando funciona!.

Solución propuesta

Aquí tienes una implementación usando el bucle `while`. Fíjate cómo se combinan todas las piezas que hemos aprendido.

```
print("=== JUEGO DE LAS ADIVINANZAS ===")
print("He pensado un número del 1 al 20. ¿Puedes adivinarlo?")
```

```
# 1. Definimos el número secreto (lo ponemos "fijo" por ahora)
numero_secreto = 13
```

```
# 2. Inicializamos la variable del intento con un valor que fuerce a entrar al
bucle
```

```
intento = 0
```

```
# Opcional: un contador de intentos para hacerlo más interesante
```

```
contador_intentos = 0
```

```
# 3. Bucle WHILE: Se repite mientras el usuario NO acierte
```

```
# Leemos: "Mientras el intento sea diferente del número secreto..."
```

```
while intento != numero_secreto:
```

```
    # Pedimos el número y actualizamos la variable 'intento'
```

```
    # Es crucial hacer esto DENTRO del bucle.
```

```
    intento_texto = input("\nIntroduce tu apuesta: ")
```

```
    intento = int(intento_texto)
```

```
    # Sumamos 1 al contador de intentos
```

```
    contador_intentos += 1 # Esto es igual a: contador_intentos =
```

```
    contador_intentos + 1
```

```
# 4. Lógica condicional para dar pistas
```

```
if intento < numero_secreto:
```

```
    print("❌ ¡Fallaste! El número secreto es MAYOR.")
```

```
elif intento > numero_secreto:
```

```
    print("❌ ¡Fallaste! El número secreto es MENOR.")
```

```
# No hace falta un 'else' para el acierto aquí, el while se encargará.
```

```
# 5. Fin del juego
```

```
# Si el programa llega aquí, es que el bucle while ha terminado.
```

```
# Y la única forma de que termine es que intento == numero_secreto.
```

```
print(f"\n🎉 ¡FELICIDADES! ¡Has acertado!")
```

```
print(f"El número era el {numero_secreto}.")
```

```
# Usamos una f-string para mostrar cuántos intentos le ha costado
```

```
print(f"Lo has conseguido en {contador_intentos} intentos.")
```

```
print("=== Fin del juego ===")
```

Reto Extra (Para nota):

Ahora mismo el número secreto es siempre 13. ¡Qué aburrido si juegas dos veces! Python tiene una "caja de herramientas" llamada `random` para generar números aleatorios. Intenta buscar en Google "python random randint" y mira si puedes cambiar la línea `numero_secreto = 13` por algo que genere un número distinto cada vez que ejecutas el programa.

¡Enhorabuena!, ya tienes los conocimientos fundamentales de la programación estructurada:

1. **Variables y Tipos:** Guardar información.
2. **Entrada/Salida:** Comunicarte con el usuario.
3. **Lógica Condicional (`if`):** Tomar decisiones.
4. **Bucles (`while/for`):** Repetir tareas.

Con estas cuatro herramientas, puedes construir programas increíblemente complejos. Todo lo que sigue es aprender a organizar mejor el código y usar herramientas más potentes.

En la próxima parte, veremos cómo los programadores organizan sus datos cuando tienen muchos, usando **Listas**.

Capítulo 7: Listas (Las súper-variables)

Imagina una lista de la compra, una lista de tareas pendientes o la alineación de un equipo de fútbol. Son colecciones ordenadas de elementos. Eso es exactamente una **lista** en Python.

Es como una súper-variable que puede contener muchas variables normales dentro, una detrás de otra, en un orden específico.

7.1 ¿Cómo se crea una lista?

La sintaxis es muy sencilla: usamos **corchetes []** para definir dónde empieza y termina la lista, y separamos los elementos internos con **comas ,**.

```
# Una lista vacía (útil para llenarla luego)
mi_mochila = []

# Una lista de números (puntuaciones de un juego, por ejemplo)
puntuaciones = [10, 50, 25, 100, 5]

# Una lista de cadenas de texto (la lista de la compra)
compra = ["Pan", "Leche", "Huevos", "Café"]

# ¡Python permite mezclar tipos! (Aunque no siempre es recomendable)
bolsa_mezclada = [42, "Hola", True, 3.14]

print(compra)

# Salida: ['Pan', 'Leche', 'Huevos', 'Café']
```

7.2 Accediendo a los elementos (El truco del índice 0)

Aquí viene una de las reglas sagradas de la programación que más cuesta asimilar al principio:

⚠ EN PROGRAMACIÓN, CASI SIEMPRE SE EMPIEZA A CONTAR DESDE CERO.

El primer elemento de la lista no está en la posición 1, ¡está en la posición **0**! A este número de posición se le llama **índice**.

Para acceder a un elemento concreto, escribimos el nombre de la lista seguido del índice entre corchetes.

```
frutas = ["Manzana", "Banana", "Cereza", "Dátil"]
# Índices: 0      1      2      3
print(frutas[0]) # Salida: Manzana (El primero)
print(frutas[2]) # Salida: Cereza (El tercero, ¡no el segundo!)
# ¡ERROR COMÚN! Intentar acceder a un índice que no existe
# print(frutas[4])
# Esto dará un error: IndexError: list index out of range
```

El truco de los índices negativos

Python tiene una característica genial que no tienen todos los lenguajes. Si quieres acceder al **último** elemento de una lista larga sin tener que contar cuántos tiene, puedes usar índices negativos.

- -1 es el último.
- -2 es el penúltimo.
- etc.

```
frutas = ["Manzana", "Banana", "Cereza", "Dátil"]
print(frutas[-1]) # Salida: Dátil
```

7.3 Modificando listas (Son mutables)

A diferencia de otros tipos de datos (como las cadenas de texto, que una vez creadas no se pueden cambiar "por dentro"), las listas son **mutables**. Puedes cambiar, añadir o borrar elementos después de haber creado la lista.

Para cambiar un elemento, simplemente accedemos a él por su índice y usamos el operador de asignación `=`.

```
lista_tareas = ["Limpiar cocina", "Estudiar Python", "Poner lavadora"]
print("Antes:", lista_tareas)
# ¡Uy! Ya he limpiado la cocina, voy a cambiar esa tarea.
# Accedo al índice 0 y le asigno un nuevo valor.
lista_tareas[0] = "Descansar en el sofá"
print("Después:", lista_tareas)
# Salida: Después: ['Descansar en el sofá', 'Estudiar Python', 'Poner lavadora']
```



¡Exacto! Ya sabemos crear listas y acceder a elementos sueltos. Pero el verdadero poder de las listas surge cuando queremos procesar **todos** los datos que contienen.

Imagina que tienes una lista con los precios de 100 productos y quieres aplicarles el IVA a todos. ¿Vas a escribir 100 líneas de código, una para cada índice? ¡Claro que no!

Aquí es donde el bucle for que vimos en el capítulo anterior se convierte en tu mejor amigo.

7.4 Recorriendo listas con bucles for

Como vimos, el bucle for en Python está diseñado específicamente para recorrer colecciones. Una lista es la colección por excelencia.

La estructura es:

```
for variable_temporal in nombre_de_la_lista:  
    # Hacer algo con la variable_temporal
```

En cada repetición del bucle, la `variable_temporal` tomará automáticamente el valor del siguiente elemento de la lista, desde el primero hasta el último.

Ejemplo 1: La lista de invitados

Vamos a imprimir una invitación personalizada para cada persona en una lista.

```
invitados = ["Ana", "Berto", "Carla", "Daniel"]

print("--- Enviando invitaciones ---")
# En la primera vuelta, 'persona' será "Ana"
# En la segunda, 'persona' será "Berto", etc.
for persona in invitados:
    print(f"¡Hola {persona}! Estás invitado a mi fiesta.")
print("--- Todas las invitaciones enviadas ---")
```

¿Ves qué limpio? No hemos tenido que preocuparnos por índices (invitados[0], invitados[1], etc.). Python lo hace por nosotros.

Ejemplo 2: Procesando datos numéricos

Vamos a calcular el cuadrado de una lista de números.

```
numeros = [2, 5, 8, 10]
print("Calculando cuadrados...")
for num in numeros:
    cuadrado = num * num # O num ** 2
    print(f"El cuadrado de {num} es {cuadrado}")
```



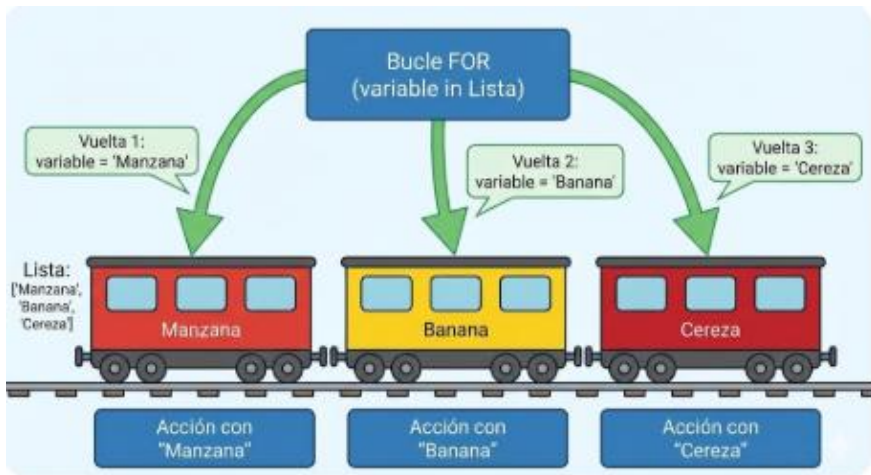
Un truco "Pro": ¿Qué pasa si necesito el índice? (enumerate)

A veces, mientras recorres la lista, no solo necesitas el valor del elemento, sino también saber en qué posición (índice) está. Por ejemplo, para hacer una lista numerada.

La forma "novata" (pero válida) sería usar un contador manual o `range(len(lista))`.

La forma **Pythonica y elegante** es usar la función mágica `enumerate()`. Esta función te devuelve **dos cosas** en cada vuelta: el índice y el valor.

```
ranking_peliculas = ["El Padrino", "Pulp Fiction", "El Caballero Oscuro"]
print("--- Mi Ranking de Películas ---")
# Fíjate cómo ahora el 'for' tiene DOS variables temporales
# 'i' tomará el índice (0, 1, 2...)
# 'peli' tomará el valor ("El Padrino", etc.)
for i, peli in enumerate(ranking_peliculas):
    # Como el índice empieza en 0, le sumamos 1 para que la lista empiece en 1
    posicion = i + 1
    print(f"{posicion}. {peli}")
# Salida:
# 1. El Padrino
# 2. Pulp Fiction
# 3. El Caballero Oscuro
```



¡Exacto! Ya sabemos recorrerlas. Pero, ¿y si la lista necesita crecer o encogerse?

Hasta ahora, nuestras listas eran fijas. Pero en el mundo real, las cosas cambian: a la lista de la compra le añades "tomates" y le tachas "leche" cuando la metes en el carro.

Python nos da una "caja de herramientas" pegada a cada lista para hacer esto. A estas herramientas las llamamos **Métodos**.

7.5 Modificando listas: Métodos clave (.append, .remove)

Un **método** es como una función, pero que pertenece a un objeto específico (en este caso, a una lista). Se usan poniendo un punto . después del nombre de la lista, seguido del nombre del método y paréntesis ().

Es como decirle a la lista: lista.haz_esto(con_este_dato).

1. Añadir al final: El método .append()

Este es, con diferencia, el método más usado. append significa "adjuntar" o "añadir al final". Coge el elemento que le des y lo coloca en el último vagón del tren, haciendo la lista una unidad más larga.

```
# Empezamos con una lista de tareas pequeña
tareas = ["Limpiar", "Cocinar"]
print("Inicio:", tareas)
```

```
# ¡Oh! Me he acordado de que tengo que estudiar.
# Añadimos la tarea AL FINAL.
```

```

tareas.append("Estudiar Python")
print("Después de append:", tareas)
# Salida: ['Limpiar', 'Cocinar', 'Estudiar Python']
# Podemos usarlo en un bucle para construir listas dinámicamente
numeros_pares = []
for i in range(1, 11):
    if i % 2 == 0:
        # Si el número es par, lo añadimos a nuestra lista
        numeros_pares.append(i)
print("Números pares hasta el 10:", numeros_pares)
# Salida: [2, 4, 6, 8, 10]

```

2. Quitar por valor: El método .remove()

Si sabes *qué* quieres quitar, pero no sabes *dónde* está (en qué índice), .remove() es tu amigo. Busca la **primera aparición** de ese valor en la lista y la elimina.

```

frutas = ["Manzana", "Banana", "Pera", "Banana", "Kiwi"]
print("Antes:", frutas)
# No nos gusta la Banana. Vamos a quitarla.
# ¡OJO! Solo quitará la PRIMERA "Banana" que encuentre.
frutas.remove("Banana")
print("Después de remove:", frutas)
# Salida: ['Manzana', 'Pera', 'Banana', 'Kiwi']

```

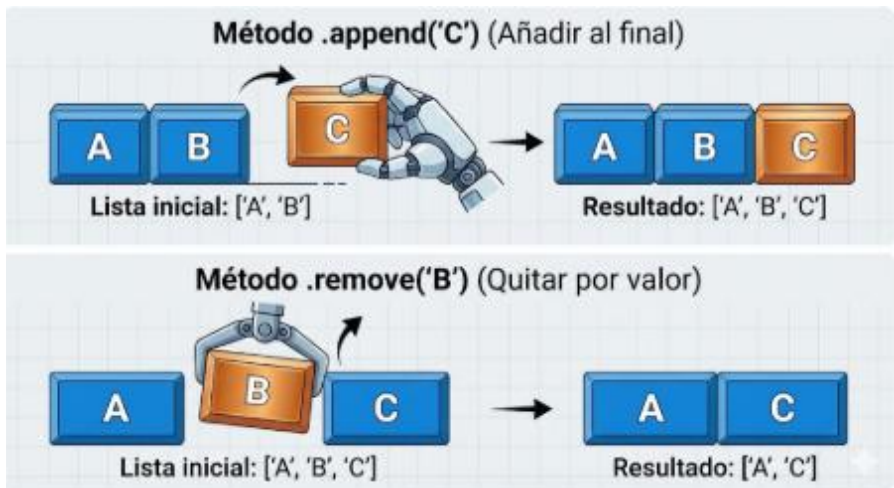
⚠️ **¡Advertencia!** Si intentas hacer .remove() de algo que NO está en la lista, Python te dará un error (ValueError).

💡 Un truco "Pro": Quitar por posición (.pop())

A veces no quieres quitar "la Banana", sino "lo que sea que esté en la segunda posición". Para quitar un elemento usando su **índice**, usamos el método .pop().

Si no le pasas ningún índice, `.pop()` quita el **último** elemento. Es el opuesto exacto de `.append()`.

```
compra = ["Pan", "Leche", "Huevos"]  
# Quitamos el elemento en el índice 1 ("Leche")  
compra.pop(1)  
print(compra) # Salida: ['Pan', 'Huevos']  
# Quitamos el último elemento  
compra.pop()  
print(compra) # Salida: ['Pan']
```



Capítulo 8: El libro de los secretos (Diccionarios)

Un **Diccionario** en Python (llamado *hash map* u *objeto* en otros lenguajes) funciona exactamente igual que un diccionario de papel de la vida real.

En un diccionario real, tienes una **palabra** (la clave) que usas para encontrar su **definición** (el valor).

En Python, un diccionario es una colección de **pares Clave-Valor** (*Key-Value pairs*).

- **La Clave (Key):** Es el identificador único. Es lo que usas para buscar. (Ej: "nombre", "edad", "telefono").
- **El Valor (Value):** Es el dato asociado a esa clave. (Ej: "Ana", 25, 666555444).

A diferencia de las listas, los diccionarios **no tienen orden** (conceptualmente). No hay "primero" ni "último". Solo hay "claves" que abren "valores".

8.1 ¿Cómo se crea un diccionario?

Si las listas usaban corchetes [], los diccionarios usan **llaves {}**. Dentro, escribimos los pares separados por dos puntos :, y cada par separado por comas.

```
# Una lista (ordenada por índice)
# jugador_lista = ["Aragorn", "Montaraz", 10]
# Para saber el nombre, tengo que recordar que es el índice 0. ¡Qué lío!

# Un diccionario (asociado por clave)
```

```
# ¡Mucho más claro! Sabemos qué es cada dato.
jugador = {
    "nombre": "Aragorn",
    "clase": "Montaraz",
    "nivel": 10
}
# Otro ejemplo: Una mini agenda telefónica
agenda = {
    "Ana": 666777888,
    "Berto": 612345678
}
print(jugador)
# Salida: {'nombre': 'Aragorn', 'clase': 'Montaraz', 'nivel': 10}
```

8.2 Accediendo a los datos (La llave maestra)

Aquí está la gran diferencia con las listas. **No usamos números de índice**. Usamos la clave entre corchetes para "abrir" el valor.

```
jugador = {
    "nombre": "Aragorn",
    "clase": "Montaraz",
    "nivel": 10
}
# Quiero saber el nombre del jugador:
print(jugador["nombre"]) # Salida: Aragorn

# Quiero saber su nivel:
print(jugador["nivel"]) # Salida: 10
# ¡ERROR COMÚN! Intentar usar un índice numérico
# print(jugador[0])
# Esto dará un error: KeyError: 0 (porque no existe ninguna clave llamada 0)
```

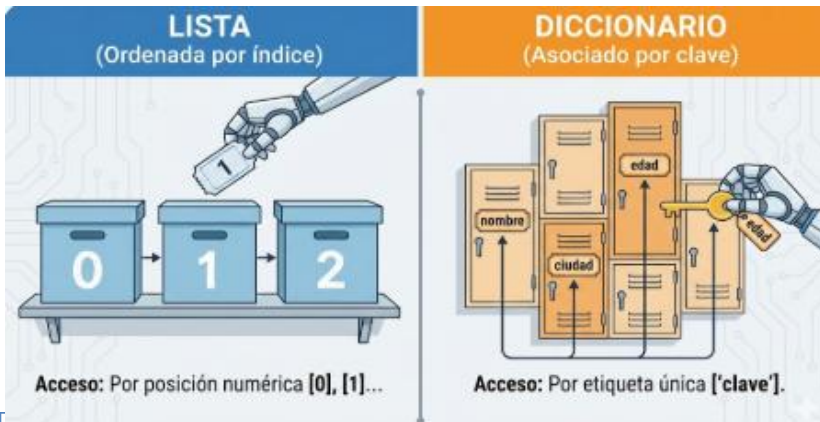
⚠ El peligro del KeyError y la solución segura: .get()

Si intentas acceder a una clave que no existe en el diccionario (por ejemplo, `jugador["puntos_de_vida"]`), Python entrará en pánico y te dará un error llamado `KeyError`, deteniendo tu programa.

Para evitar esto, los profesionales usan el método `.get()`.

- Si la clave existe, te da el valor.
- Si la clave NO existe, te devuelve `None` (un valor especial que significa "nada") sin dar error, o puedes decirle qué devolver por defecto.

```
jugador = {"nombre": "Aragorn", "nivel": 10}
# Forma arriesgada:
# print(jugador["arma"]) -> ¡ERROR! Crash.
# Forma segura con .get():
arma = jugador.get("arma")
print(arma) # Salida: None (no da error)
# Forma segura con valor por defecto:
# "Intenta buscar 'arma', si no existe, devuelve 'Puños'"
arma_actual = jugador.get("arma", "Puños")
print(f"El jugador usa: {arma_actual}")
# Salida: El jugador usa: Puños
```



8.3 Modificando el diccionario: Añadir, cambiar y borrar

La sintaxis para añadir y modificar es tan sencilla e intuitiva que a veces sorprende. Es exactamente la misma.

1. Añadir y Modificar datos

Para guardar un dato, simplemente decimos:

`diccionario["clave"] = valor.`

- **SI la clave NO existe:** Python crea una nueva "taquilla", le pega esa etiqueta y guarda el valor dentro. ¡Ha añadido una entrada nueva!
- **SI la clave YA existe:** Python busca esa "taquilla", tira lo que había dentro y mete el nuevo valor. ¡Ha modificado la entrada existente!

```
# Empezamos con nuestro jugador
```

```
jugador = {  
    "nombre": "Aragorn",  
    "nivel": 10  
}  
print("Inicio:", jugador)
```

```
# --- AÑADIR ---
```

```
# No tiene arma. Vamos a darle una.
```

```
# La clave "arma" no existe, así que la crea.
```

```
jugador["arma"] = "Espada de ratero"
```

```
print("Después de añadir arma:", jugador)
```

```
# Salida: {'nombre': 'Aragorn', 'nivel': 10, 'arma': 'Espada de ratero'}
```

```
# --- MODIFICAR ---
```

```
# ¡Ha subido de nivel y encontrado una espada mejor!
# La clave "nivel" ya existe, así que actualiza su valor.
jugador["nivel"] = 11
# La clave "arma" ya existe, así que actualiza su valor.
jugador["arma"] = "Andúril, Llama del Oeste"

print("Después de las mejoras:", jugador)
# Salida: {'nombre': 'Aragorn', 'nivel': 11, 'arma': 'Andúril, Llama del Oeste'}
```

¡Fíjate qué fácil! No necesitas métodos especiales como `.append()`. Solo asignas el valor a la clave, y Python se encarga del resto.

2. Borrar datos (La palabra clave `del`)

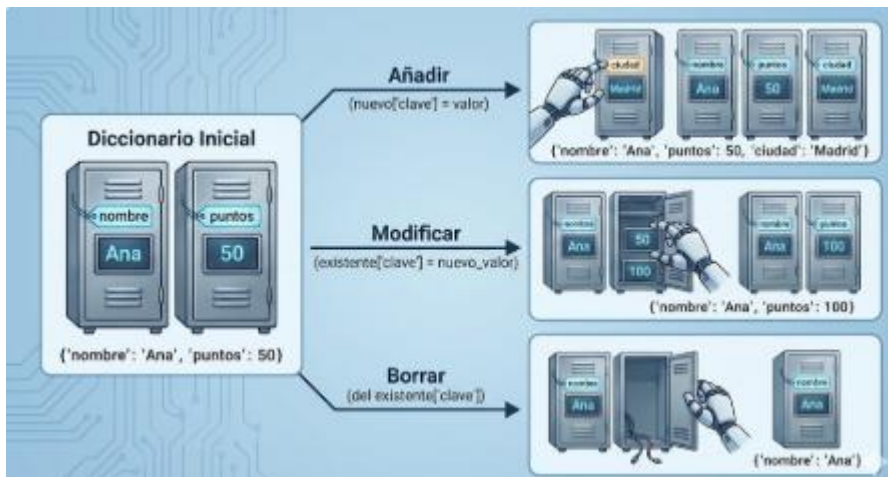
Para eliminar una pareja clave-valor por completo (quitar la etiqueta y el contenido), usamos la instrucción `del` (de *delete*, borrar) seguida del acceso a la clave.

```
jugador = {'nombre': 'Aragorn', 'nivel': 11, 'arma': 'Andúril', 'hechizo_temporal':
    'Invisibilidad'}
print("Antes de borrar:", jugador)

# El efecto del hechizo se ha pasado. Vamos a borrarlo.
del jugador["hechizo_temporal"]

print("Después de borrar:", jugador)
# Salida: {'nombre': 'Aragorn', 'nivel': 11, 'arma': 'Andúril'}
```

 **¡Advertencia!** Al igual que al acceder, si intentas hacer `del` de una clave que NO existe, Python te dará un `KeyError`. Si no estás seguro de si existe, puedes comprobarlo antes con `if "clave" in diccionario:`



8.4 Recorriendo diccionarios con bucles for

Imagina que tenemos el inventario de una pequeña tienda de pociones:

```
inventario_pociones = {  
    "Curación": 50,  
    "Maná": 30,  
    "Resistencia": 15  
}
```

Forma 1: Recorrer solo las CLAVES (El comportamiento estándar)

Si usas un `for` directamente sobre el diccionario, Python, por defecto, te dará las **claves** (las etiquetas) una por una.

Es útil si solo quieres saber *qué* tipos de cosas tienes, o si quieres usar la clave para buscar el valor dentro del bucle.

```
print("--- Tipos de pociones disponibles ---")
# La variable 'tipo' irá tomando los valores "Curación", "Maná", "Resistencia"
for tipo in inventario_pociones:
    print(f"- {tipo}")
    # Si quisiéramos el valor, podríamos hacer:
    # cantidad = inventario_pociones[tipo]
    # print(f" (Tenemos {cantidad})")
```

Forma 2: Recorrer solo los VALORES (El método .values())

A veces no te importan las etiquetas, solo te interesa lo que hay dentro. Por ejemplo, para sumar el total de pociones que tienes en la tienda, sin importar de qué tipo sean.

Usamos el método .values().

```
total_botellas = 0
print("\n--- Contando stock total ---")
# La variable 'cantidad' tomará los valores 50, 30, 15
for cantidad in inventario_pociones.values():
    total_botellas += cantidad # Sumamos al acumulador
print(f"Tenemos un total de {total_botellas} botellas en la tienda.")
```

Forma 3: La reina de la fiesta: Claves y Valores a la vez (El método .items())

Esta es la forma más usada y "Pythonica". Si necesitas tanto la etiqueta como el contenido al mismo tiempo, el método .items() es perfecto.

Este método te devuelve una parejita (clave, valor) en cada vuelta. Por eso, en el for usamos **dos variables temporales** separadas por coma para atraparlas.

```
print("\n--- Inventario detallado ---")
```

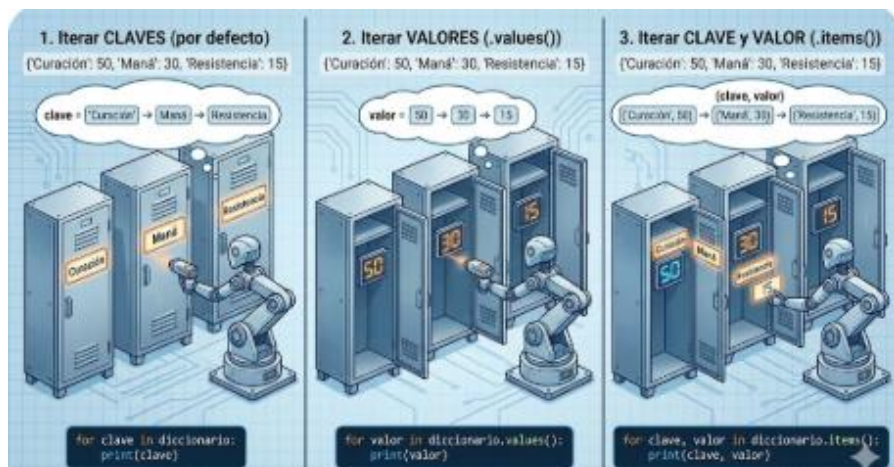
```
# 'tipo' atraparé la CLAVE
```

```
# 'cantidad' atraparé el VALOR
```

```
for tipo, cantidad in inventario_pociones.items():
```

```
    print(f"La poción de {tipo} tiene {cantidad} unidades en stock.")
```

¡Fíjate qué limpio y legible queda!



Capítulo 9: Funciones (Tus propias herramientas)

Una **función** es un bloque de código con un nombre, diseñado para realizar una tarea específica.

Piensa en las funciones como si fueran **pequeños electrodomésticos especializados** en tu cocina de programación.

- `print()` es una tostadora: le metes datos y te devuelve texto en pantalla.
- `input()` es un exprimidor: le pides algo y extrae información del usuario.
- `len()` es una báscula: pesa una lista y te dice cuántos elementos tiene.

Hasta ahora has usado las funciones que Python trae de fábrica (built-in functions). Ahora, ¡vas a aprender a fabricar tus propios electrodomésticos!

9.1 Definiendo nuestra primera función (def)

Para crear (definir) una función, usamos la palabra clave **def**, seguida del nombre que queramos darle, paréntesis `()` y dos puntos `:`. Todo el código que pertenezca a la función debe ir **indentado** debajo.

```
# --- DEFINICIÓN DE LA FUNCIÓN ---  
# Esto es como escribir la receta o fabricar la máquina.  
# Por sí solo, ESTE CÓDIGO NO HACE NADA visible todavía.
```

```
def saludar_con_entusiasmo():
```

```

print("¡Hola! ¡Bienvenido al programa!")
print("¡Espero que tengas un día fantástico!")
print("-----")
# --- PROGRAMA PRINCIPAL ---
print("Inicio del programa.")
# Si ejecuto esto así, solo imprimirá "Inicio del programa" y "Fin".
# La función está creada, pero guardada en el armario.
print("Fin del programa.")

```

Llamando (ejecutando) a la función

Para usar la máquina que hemos fabricado, tenemos que "llamarla" por su nombre seguido de paréntesis.

```

# --- DEFINICIÓN ---
def saludar_con_entusiasmo():
    print("\n¡Hola! ¡Bienvenido al programa!")
    print("¡Espero que tengas un día fantástico!")
    print("-----")
# --- PROGRAMA PRINCIPAL ---
print("Inicio del programa.")
# ¡Aquí sacamos la máquina del armario y pulsamos el botón!
saludar_con_entusiasmo()
print("Haciendo algunas cosas intermedias...")
# ¡Podemos usarla tantas veces como queramos!
saludar_con_entusiasmo()
print("Fin del programa.")

```

¡Fíjate! Hemos ejecutado 6 líneas de código (prints) escribiendo solo 2 líneas de llamada. Si mañana queremos cambiar el saludo, solo lo cambiamos en un sitio (la definición) y se actualizará en todas las llamadas. ¡Magia!

9.2 Funciones con entradas (Parámetros y Argumentos)

Nuestra función `saludar_con_entusiasmo` es un poco sosa. Siempre saluda igual. Es como una tostadora que solo sabe hacer tostadas con el nivel de tostado al 3.

Para que las funciones sean útiles de verdad, necesitan ser flexibles. Necesitan aceptar **datos de entrada** para trabajar con ellos.

Estos datos se meten por el "embudo" de los paréntesis.

- **Parámetros:** Son las variables que ponemos entre paréntesis en la *definición* de la función. Son los huecos que esperamos rellenar.
- **Argumentos:** Son los valores reales que le pasamos a la función cuando la *llamamos*.

Vamos a mejorar nuestra tostadora para que salude a quien nosotros queramos.

```
# --- DEFINICIÓN ---
# En los paréntesis definimos un parámetro llamado 'nombre_usuario'.
# Es una variable que SOLO EXISTE dentro de esta función.
def saludar_personalizado(nombre_usuario):
    print(f"\n¡Hola, {nombre_usuario}!")
    print("¡Bienvenido de nuevo!")
# --- PROGRAMA PRINCIPAL ---
# Llamada 1: Le pasamos el argumento "Ana"
# La función recibe "Ana" y lo guarda en la variable 'nombre_usuario'
saludar_personalizado("Ana")
# Llamada 2: Le pasamos otro argumento
saludar_personalizado("Berto")
jugador_actual = "Aragorn"
```

```
# Llamada 3: Le pasamos una variable como argumento
saludar_personalizado(jugador_actual)
# ¡ERROR! Si la función espera un parámetro, ¡hay que dárselo!
# saludar_personalizado() # Esto dará un TypeError
```

Múltiples parámetros

Podemos definir tantos parámetros como necesitemos, separándolos por comas.

```
def presentar_jugador(nombre, clase, nivel):
    print(f"\n--- Ficha de Personaje ---")
    print(f"Nombre: {nombre}")
    print(f"Clase: {clase} (Nivel {nivel})")
# Al llamarla, hay que pasar los datos EN EL MISMO ORDEN
presentar_jugador("Gandalf", "Mago", 20)
presentar_jugador("Legolas", "Arquero", 15)
```

Esto empieza a ponerse interesante. Las funciones nos permiten empaquetar lógica y reutilizarla con diferentes datos.

Pero a nuestros "electrodomésticos" les falta algo clave. Les metemos datos (el pan en la tostadora), hacen algo (tostarlo), pero... ¿cómo sacamos el resultado? Ahora mismo nuestras funciones solo saben hacer print en la pantalla.

En la siguiente sección veremos la pieza más importante de las funciones: cómo hacer que nos **devuelvan** un valor que podamos guardar en una variable. Prepárate para conocer la palabra clave **return**.

9.3 El valor de retorno (return): La entrega del pedido

Esta es, posiblemente, la lección que marca la diferencia entre un principiante y un programador capaz de crear sistemas complejos.

El error número 1 cuando empezamos es confundir print() con return. Vamos a usar la analogía de la **Tostadora** para entenderlo de una vez por todas.

La Analogía: La Tostadora Egoísta vs. La Tostadora Útil

Imagina que tienes dos tostadoras en tu cocina:

1. La Tostadora print() (Egoísta):

- Le metes pan.
- La tostadora calienta el pan.
- En lugar de dártelo, enciende una pantalla LED y muestra el mensaje: **"HE HECHO UNA TOSTADA"**.
- La tostada se queda dentro y se quema. Tú te quedas con hambre.
- **Resultado:** Ves el mensaje, pero **no puedes comerte la tostada** ni untarle mantequilla.

2. La Tostadora return (Útil):

- Le metes pan.
- La tostadora calienta el pan.
- Cuando termina, **expulsa la tostada hacia fuera**.
- Tú la coges (la guardas en una variable) y haces lo que quieras con ella (comerla, untarla, dársela al perro...).

En programación:

- `print()`: Muestra el dato en la pantalla para que **tú (humano)** lo leas, pero el dato se pierde para el programa.
- `return`: Devuelve el dato al **código** para que pueda ser guardado en una variable y usado después.

La sintaxis del return

Para que una función devuelva un valor, usamos la palabra clave `return` seguida de lo que queremos devolver.

Mira la diferencia en el código:

```
# --- LA FORMA INCORRECTA (Solo Print) ---
def sumar_con_print(a, b):
    resultado = a + b
    print(f"La suma es: {resultado}")
    # Muestra el texto, pero la función devuelve 'None' (nada)
# --- LA FORMA CORRECTA (Con Return) ---
def sumar_con_return(a, b):
    resultado = a + b
    return resultado
    # ¡Lanza el número hacia fuera!
```

Ahora, veamos qué pasa cuando intentamos guardar el resultado en una variable:

```
# Probamos la función 'mala'
print("--- Probando print ---")
x = sumar_con_print(5, 5)
# Veremos el mensaje en pantalla...
# Pero, ¿cuánto vale x?
```

```
print(f"El valor de x es: {x}")
# Salida: El valor de x es: None <-- ¡HORROR! La variable está vacía.
# Probamos la función 'buena'
print("\n--- Probando return ---")
y = sumar_con_return(10, 10)
# Aquí no sale nada en pantalla inmediatamente, PERO...
# El 20 está guardado sano y salvo dentro de 'y'.
print(f"El valor de y es: {y}")
# ¡Ahora podemos usar 'y' para seguir operando!
total = y + 50
print(f"El total final es: {total}")
```

Truco Pro: return es el final del camino

Ten mucho cuidado: cuando Python encuentra una instrucción **return**, **la función termina inmediatamente**. Cualquier código que escribas debajo del **return** será ignorado (es como si la tostadora se apagara al saltar el pan).

```
def funcion_ejemplo():
    print("Esto se imprime.")
    return "Fin"
    print("¡Esto NUNCA se imprimirá!") # Código inalcanzable (muerto)
resultado = funcion_ejemplo()
```

9.4 Ejercicio Práctico: Conversor de Moneda

Vamos a aplicar esto. Imagina que estás programando una tienda online internacional. Necesitas una función que convierta precios de Dólares a Euros, para poder sumar el total después.

Tu misión:

1. Crea una función llamada `convertir_a_euros`.
2. Debe recibir un parámetro: `cantidad_dolares`.
3. Debe calcular el cambio (imagina que 1 Dólar = 0.85 Euros).
4. Debe **retornar** (return) el resultado.
5. En el programa principal, convierte 100 dólares, guarda el resultado en una variable llamada `precio_en_euros` y luego súmale 5 euros de gastos de envío.

¡Inténtalo tú primero antes de mirar la solución!

...

Solución Propuesta:

1. Definimos la función calculadora

```
def convertir_a_euros(cantidad_dolares):  
    tasa_cambio = 0.85  
    resultado = cantidad_dolares * tasa_cambio  
    # ¡Devolvemos el dato para que sea útil fuera!  
    return resultado
```

2. Programa principal

```
print("--- Calculadora de envíos ---")  
precio_producto_dolares = 100
```

Llamamos a la función y CAPTURAMOS lo que nos devuelve en una variable

```
precio_en_euros = convertir_a_euros(precio_producto_dolares)
# Ahora podemos hacer matemáticas con ese valor
gastos_envio = 5
total_a_pagar = precio_en_euros + gastos_envio
print(f"Precio en dólares: ${precio_producto_dolares}")
print(f"Precio en euros: {precio_en_euros}€")
print(f"Total con envío: {total_a_pagar}€")
```

⚠ Advertencia Común: El misterio del None

Si alguna vez ejecutas tu código y ves que una variable vale None cuando esperabas un número o un texto, ¡revisa tu función! Casi seguro que **se te olvidó poner el return** y solo pusiste un print. Python, por defecto, hace que las funciones sin return devuelvan None.

¡Excelente! Ya sabemos meter datos (parámetros) y sacar datos (return). Ahora tenemos un pequeño problema de organización... ¿Qué pasa si creo una variable DENTRO de una función? ¿Puedo usarla FUERA?

9.5 El Ámbito (Scope): ¿Dónde vive mi variable?

Hasta ahora, hemos creado variables alegremente. Pero cuando empezamos a usar funciones, creamos "mundos separados".

Imagina que tu programa principal es la plaza del pueblo, y cada función que defines es una casa privada con las ventanas cerradas.

La Analogía: La Caja Fuerte vs. El Tablón de Anuncios

Para entender el "ámbito" (dónde una variable existe y puede ser usada), usemos esta analogía visual:

- **Variables Globales (El Tablón de Anuncios Público):** Son variables creadas en el programa principal, fuera de cualquier función. Todo el mundo (todas las funciones) puede salir a la plaza y leer lo que pone en el tablón.
- **Variables Locales (La Caja Fuerte Privada):** Son variables creadas *dentro* de una función (incluyendo sus parámetros). Solo existen dentro de esa "casa". Si intentas acceder a ellas desde fuera, no existen, es como si estuvieran bajo llave y la casa se hubiera demolido al terminar la función.



Demostración en Código

Veamos cómo se comporta esto en Python real, siguiendo la lógica de la imagen.

1. El error clásico: Intentar ver la caja fuerte desde fuera

Cuando creas una variable dentro de una función, es **local**. Nace cuando llamas a la función y muere cuando la función termina (o hace un return).

```
def crear_secreto():
    # Esta variable es LOCAL. Solo existe dentro de esta función.
    secreto = 123
    print(f"Dentro de la función, el secreto es: {secreto}")
# Programa Principal
crear_secreto() # Llamamos a la función, se crea el secreto, se imprime, y la
                # función termina.
# ¡PROBLEMA! Intentamos acceder a 'secreto' desde fuera.
# Python no sabe qué es 'secreto' aquí fuera.
# print(secreto)
# ⚠ Si descomentas la línea de arriba, obtendrás un:
# NameError: name 'secreto' is not defined
```

2. Leyendo el tablón de anuncios público

Las funciones pueden "mirar hacia fuera" y leer variables globales, pero no pueden modificarlas directamente (eso lo veremos más adelante, es un poco más complejo).

```
# Esta variable es GLOBAL. Vive en el programa principal.
mensaje_publico = "Hola Mundo"
```

```
def leer_tablon():
```

```
# La función mira fuera y lee la variable global
print(f"Desde dentro de la función leo: {mensaje_publico}")
# Programa Principal
print(f"Fuera en el programa principal: {mensaje_publico}")
leer_tablon() # La función puede leer el mensaje público
```

💡 Truco Pro: Sombras (Shadowing)

¿Qué pasa si tengo una variable global y creo una local con EL MISMO nombre?

Python siempre da prioridad a la variable **más local**. Es como si tuvieras una nota en tu mano y otra en el tablón a 50 metros. Primero miras la que tienes en la mano.

```
x = 10 # Variable GLOBAL
def mi_funcion():
    x = 5 # Variable LOCAL que "hace sombra" a la global solo aquí dentro
    print(f"Valor de x DENTRO de la función: {x}")
mi_funcion() # Imprimirá 5
print(f"Valor de x FUERA de la función: {x}")
# Imprimirá 10. La variable global NO ha cambiado.
# La función solo usó su versión local temporal.
```

Con esto, hemos cubierto los cimientos más importantes de las funciones: cómo definirlas, cómo pasarles información (parámetros), cómo sacar información de ellas (return), y cómo entender dónde viven las variables que creamos (ámbito).

9.6 Funciones anónimas: Las pequeñas lambda

Hasta ahora, hemos creado funciones como si estuviéramos construyendo máquinas industriales: les damos un nombre importante con `def`, definimos sus entradas, escribimos varias líneas de instrucciones y finalmente un `return`. Son robustas y reutilizables.

Pero a veces, en programación, solo necesitas hacer una tarea pequeña, una sola operación rápida, y no quieres montar toda la parafernalia de una función completa con nombre.

Aquí entran las **funciones lambda**, también llamadas "funciones anónimas" porque... ¡no necesitan nombre!

La Analogía: La máquina industrial vs. La navaja suiza desechable

- **Función con `def` (Máquina industrial):** Una máquina de café profesional instalada en una cafetería. Tiene un nombre ("La Cafetera 3000"), ocupa espacio, tiene un proceso claro y se usa miles de veces.
- **Función `lambda` (Navaja desechable):** Un pequeño abridor de plástico que viene en un paquete, lo usas una vez para abrir algo rápido y ya está. Es pequeño, hace solo una cosa y no necesitas ponerle un nombre en una placa.

¿Cómo se escriben?

Una función lambda es una función que se escribe **en una sola línea**.

La sintaxis es: lambda parámetros : expresión_a_devolver

Fíjate que **no tiene def**, **no tiene nombre** y **no tiene return explícito** (lo que pones después de los dos puntos se devuelve automáticamente).

Veamos la diferencia lado a lado. Imagina una función que simplemente eleva un número al cuadrado:

```
# --- Estilo Clásico (Máquina Industrial) ---
def elevar_cuadrado_clasico(numero):
    return numero * numero
print(elevar_cuadrado_clasico(5)) # Salida: 25

# --- Estilo Lambda (Navaja desechable) ---
# Se lee: "Una función lambda que toma 'x' y devuelve 'x * x'"
elevar_cuadrado_lambda = lambda x : x * x
# Aunque es anónima, podemos guardarla en una variable para usarla
print(elevar_cuadrado_lambda(5)) # Salida: 25
```

Ambas hacen exactamente lo mismo. Pero la versión lambda es mucho más compacta para operaciones sencillas.

¿Dónde brillan realmente las lambda?

Quizás pienses: "Bueno, para eso uso def y ya está". ¡Y tienes razón casi siempre!

Las lambda son útiles principalmente cuando necesitamos pasar una función pequeña como **argumento a otra función**. Esto es muy común al ordenar listas complejas.

Imagina que tienes una lista de tuplas con productos y precios, y quieres ordenarla por precio:

```
productos = [  
    ("Camiseta", 20),  
    ("Zapatos", 50),  
    ("Gorra", 15)  
]  
# Si intentamos ordenar esto directamente con .sort(), Python no sabe si  
# ordenar por nombre o por precio.  
# Usamos una lambda para decirle CÓMO ordenar.  
# La función sort recibe un parámetro 'key' (clave), que espera una función.  
# Le pasamos una lambda que dice: "De cada elemento (item), fíjate solo en la  
# posición 1 (el precio)".  
productos.sort(key = lambda item : item[1])  
print("Productos ordenados por precio:")  
for prod in productos:  
    print(prod)  
# Salida:  
# ('Gorra', 15)  
# ('Camiseta', 20)  
# ('Zapatos', 50)
```

¡Magia! Sin la lambda, hubiéramos tenido que definir una función completa con def solo para decirle al método sort cómo comportarse. La lambda nos ahorró trabajo y espacio.

⚠ Advertencia: No te pases de listo

Las funciones lambda están limitadas a **una sola expresión**. No puedes meter bucles for o condicionales if/else complejos dentro de una lambda.

Si tu función necesita más de una línea de lógica, **USA DEF**. El objetivo del código es que sea legible por humanos, no intentar hacerlo todo en una línea críptica.

Con esto cerramos la teoría del capítulo de funciones. ¡Hemos cubierto muchísimo terreno! Desde el def básico hasta el return, pasando por los parámetros, el ámbito de las variables y ahora las funciones anónimas.

Damos otro salto con la refactorización.

¿Qué es refactorizar?

Imagina que tienes tu habitación hecha un desastre. La ropa limpia está mezclada con la sucia, los libros están debajo de la cama y no encuentras nada.

Refactorizar es como ordenar esa habitación:

1. **No tiras nada:** Sigues teniendo la misma ropa y los mismos libros.
2. **Organizas:** Pones la ropa en el armario (una función), los libros en la estantería (otra función).

El resultado final es que la habitación "funciona" igual (sigues durmiendo en ella), pero ahora es mucho más fácil encontrar cosas, limpiarla y mantenerla ordenada.

En código, refactorizar significa: **Cambiar la estructura del código para hacerlo más limpio y organizado, SIN cambiar su comportamiento externo.**

9.7 Ejercicio práctico: De Monolito a Modular

Te presento un código "monolítico" (un solo bloque grande de piedra). Este pequeño programa pide una nota numérica al usuario y le dice qué letra le corresponde (sistema americano: A, B, C, D, F).

El código original (El desastre):

```
# --- CÓDIGO MONOLÍTICO (ANTES) ---
# Todo está mezclado en el bloque principal.
print("--- Evaluador de Notas v1.0 ---")
score_texto = input("Introduce tu puntuación numérica (0-100): ")
score = float(score_texto)
grade = ""
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
elif score >= 60:
    grade = "D"
else:
    grade = "F"
print(f"Para una puntuación de {score}, tu calificación es: {grade}")
print("--- Fin del programa ---")
```

Si lo ejecutas, funciona. Pero si quisiéramos usar esa lógica de calcular la nota en otra parte del programa, tendríamos que copiar y pegar todo ese bloque if/elif/else. ¡Mal asunto!

Tu Misión:

Vas a refactorizar este código extrayendo la lógica principal a una función. Sigue estos pasos:

1. **Crea una función** llamada `calcular_letra_nota`.
2. Esta función debe recibir **un parámetro**: la puntuación numérica (por ejemplo, `puntuacion`).
3. Mueve todo el bloque if/elif/else dentro de la función.
4. **¡IMPORTANTE!** La función NO debe hacer print. Debe usar `return` para devolver la letra resultante (A, B, C, D o F).
5. En el programa principal, pide el input al usuario, llama a tu nueva función guardando el resultado en una variable, y finalmente haz el print del resultado.

¡Dale caña! Intenta transformar ese código monolítico en un código modular usando lo que hemos aprendido. Cuando lo tengas, te enseño mi solución.



Solución al Ejercicio 9.7: Refactorizando el Evaluador de Notas

¡Espero que hayas disfrutado el proceso! Es muy satisfactorio ver cómo el código se vuelve más limpio.

Aquí tienes la solución detallada. Fíjate cómo hemos movido la lógica "pesada" de los if/else fuera del flujo principal y la hemos encapsulado en una función reutilizable que usa return.

```
# --- LA SOLUCIÓN REFACTORIZADA (V2.0) ---
# 1. Definimos la función especializada
# Esta es nuestra nueva "máquina" encargada solo de pensar en la letra.
# Recibe un parámetro: la puntuación numérica cruda.
def calcular_letra_not(puntuacion_numerica):
    """
    Esta función recibe una puntuación (float o int)
    y devuelve la letra correspondiente (str) según el sistema A-F.
    """
    letra_resultante = ""
    # Usamos el PARÁMETRO que hemos recibido para las comparaciones
    if puntuacion_numerica >= 90:
        letra_resultante = "A"
    elif puntuacion_numerica >= 80:
        letra_resultante = "B"
    elif puntuacion_numerica >= 70:
        letra_resultante = "C"
    elif puntuacion_numerica >= 60:
        letra_resultante = "D"
    else:
        letra_resultante = "F"
```

```

# ¡PUNTO CLAVE! Devolvemos el dato usando RETURN.
# Esta función NO imprime nada en pantalla.
return letra_resultante
# --- PROGRAMA PRINCIPAL (El Coordinador) ---
# Ahora el programa principal es mucho más fácil de leer.
# Su trabajo es solo coordinar: pedir datos, llamar al experto y mostrar
    resultados.
print("--- Evaluador de Notas v2.0 (Modular) ---")
# Paso 1: Obtener datos del usuario (Input)
texto_usuario = input("Introduce tu puntuación numérica (0-100): ")
score_float = float(texto_usuario)
# Paso 2: Procesar datos (Llamada a la función)
# Le pasamos el número a nuestra nueva función y CAPTURAMOS la respuesta.
nota_final_letra = calcular_letra_nota(score_float)
# Paso 3: Mostrar resultados (Output)
# Usamos la variable donde guardamos lo que nos devolvió la función.
print(f"Para una puntuación de {score_float}, tu calificación es:
    {nota_final_letra}")
print("--- Fin del programa ---")

```

¿Por qué esta versión es mejor?

1. **Reutilización:** Si mañana necesitas calcular la letra de una nota en otra parte de tu programa (por ejemplo, leyendo de un archivo), solo tienes que llamar a `calcular_letra_nota(85)`. No tienes que volver a copiar y pegar todos los `if/else`.
2. **Legibilidad:** El programa principal se lee como un guion paso a paso: "Pide input -> Calcula letra -> Imprime resultado". Es mucho más fácil de entender que un montón de condicionales anidados.

Capítulo 10: Organizando Proyectos Grandes - Módulos y Paquetes

Hasta ahora, en el "Capítulo 9", nos hemos vuelto expertos en crear funciones. Hemos aprendido a organizar el código *dentro* de un solo archivo.

Pero imagina que estás construyendo una casa. Las funciones son como saber fabricar ladrillos, ventanas y puertas. Ahora bien, no puedes tener todos los materiales tirados en una montaña gigante en el centro de la obra. Necesitas organizarlos: un almacén para ladrillos, otro para maderas, otro para herramientas eléctricas.

En programación, cuando un proyecto crece, no podemos tener miles de líneas de código en un solo archivo `mi_programa_gigante.py`. Sería inmanejable.

En este capítulo, aprenderemos a dividir nuestro código en múltiples archivos para mantenerlo ordenado, profesional y fácil de mantener.

10.1 ¿Qué es un módulo? Divide y vencerás

Empecemos por lo básico. La definición técnica de "módulo" en Python es tan simple que parece mentira:

Un módulo es cualquier archivo con extensión `.py`.

¡Ya está! Si creas un archivo de texto, escribes un par de funciones de Python dentro, y lo guardas como `mis_calculos.py`, ¡felicidades!, acabas de crear un módulo.

La Analogía: El Taller Mecánico Organizado

Para entender por qué esto es vital, visualicemos un taller:

1. **El Programa Monolítico (Un solo archivo):** Imagina un taller donde **TODAS** las herramientas (miles de llaves, destornilladores, martillos, sierras, repuestos) están tiradas en una montaña enorme y desordenada en el suelo. Encontrar la llave inglesa del número 10 es una pesadilla que te lleva horas.
2. **El Programa Modular (Varios archivos):** Imagina ahora un taller profesional con un gran mueble de herramientas con cajones etiquetados:
 - Un cajón etiquetado "Motor" (el archivo motor.py).
 - Un cajón etiquetado "Electricidad" (el archivo electricidad.py).
 - Un cajón etiquetado "Frenos" (el archivo frenos.py).

Cada cajón (módulo) contiene solo las funciones relacionadas con su tema. Cuando tienes que arreglar los frenos, vas directo al cajón de frenos.

10.2 Importando código: La sentencia `import`

Muy bien, hemos decidido ser ordenados y hemos dividido nuestro código en varios archivos `.py`. Pero ahora tenemos un problema: ¿Cómo hacemos que trabajen juntos?

Si estoy escribiendo mi programa principal (`main.py`), ¿cómo puedo usar esa función tan buena que guardé en el archivo `mis_calculos.py`?

Aquí entra en juego la palabra mágica: **import**.

import es como decirle a Python: "Oye, necesito usar las herramientas que están guardadas en *aquel* otro cajón. Ábrelo y déjamelas a mano".

Vamos a verlo con un ejemplo práctico. Vamos a simular que creamos **DOS archivos en la misma carpeta** de tu ordenador.

Archivo 1: El cajón de herramientas (mis_utilidades.py)

Este archivo es nuestro módulo. Solo contiene definiciones de variables y funciones útiles. No hace nada si lo ejecutas solo.

```
# --- CONTENIDO DEL ARCHIVO: mis_utilidades.py ---
# Este archivo es un MÓDULO donde guardamos herramientas.
# Podemos guardar variables
IMPUESTO_IVA = 0.21
# Y podemos guardar funciones
def calcular_precio_con_iva(precio_base):
    """Recibe un precio sin IVA y devuelve el precio final."""
    impuesto = precio_base * IMPUESTO_IVA
    precio_final = precio_base + impuesto
    return precio_final
def saludar_formalmente(nombre):
    """Devuelve un saludo muy educado."""
    return f"Estimado/a {nombre}, es un placer asistirle hoy."
```

Archivo 2: El programa principal (programa.py)

Este es el archivo que nosotros ejecutaremos para hacer el trabajo. Necesita las herramientas del otro archivo.

```
# --- CONTENIDO DEL ARCHIVO: programa.py ---
# Este es el programa principal.
```

```
print("--- Iniciando el programa ---")
```

```
# 1. IMPORTAMOS el módulo
```

```
# Usamos el nombre del archivo SIN la extensión .py
```

```
import mis_utilidades
```

```
# ¡Ya está! Ahora Python sabe que existe 'mis_utilidades'.
```

```
# 2. Usamos las herramientas del módulo.
```

```
# LA REGLA DE ORO: Usamos la "notación de punto":
```

```
# nombre_del_modulo.nombre_de_la_herramienta
```

```
# Usamos una función del módulo:
```

```
precio_producto = 100
```

```
# Fíjate en el punto: mis_utilidades.calcular...
```

```
precio_total = mis_utilidades.calcular_precio_con_iva(precio_producto)
```

```
print(f"El precio de {precio_producto}€ con IVA es: {precio_total}€")
```

```
# Usamos otra función del módulo:
```

```
saludo = mis_utilidades.saludar_formalmente("Ana")
```

```
print(saludo)
```

```
# Incluso podemos acceder a sus variables:
```

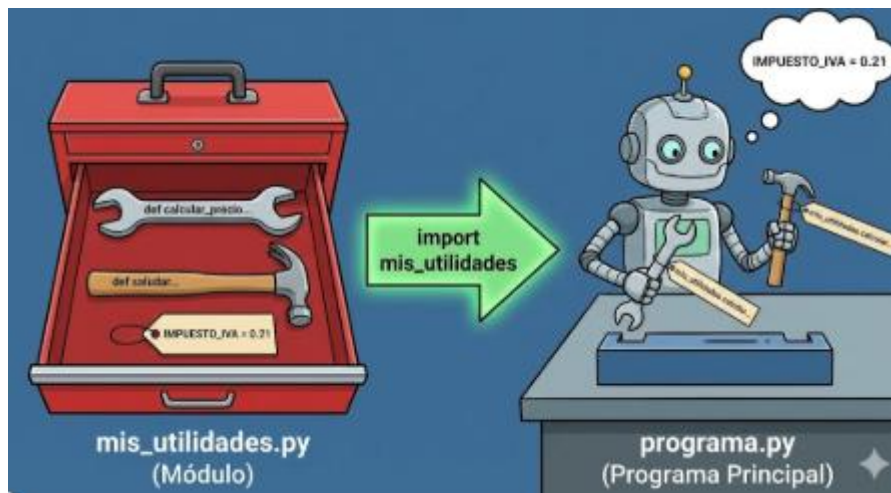
```
print(f"El IVA aplicado ha sido del: {mis_utilidades.IMPUESTO_IVA * 100}%")
```

```
print("--- Fin del programa ---")
```

¿Por qué es tan importante el punto (.)?

El punto (`mis_utilidades.saludar...`) es crucial para evitar confusiones. Imagina que en tu programa.py tú también hubieras creado una función llamada `saludar_formalmente` pero que hiciera algo distinto. Gracias al punto, Python sabe diferenciar perfectamente entre "la función `saludar` que vive en el módulo `utilidades`" y "la función `saludar` que vive aquí mismo".

Es como decir "el martillo *del cajón rojo*" frente a "el martillo *que tengo en la mano*".



A veces, no quieres traerte toda la caja de herramientas porque pesa mucho, o porque solo vas a usar el destornillador una vez.

Variante A: Importar solo lo que necesitas (from ... import ...)

Esta sintaxis le dice a Python: "Del cajón X, tráeme SOLO la herramienta Y, y déjamela encima de la mesa para usarla directamente, sin tener que decir de dónde viene".

La Analogía: En lugar de traer la caja de herramientas roja entera y decir "pásame el martillo *de la caja roja*", simplemente sacas el martillo de la caja, te lo guardas en el bolsillo y dices "pásame el martillo".

Mira la diferencia en el código:

```
# --- ARCHIVO: programa.py (Versión con 'from...import') ---
print("--- Iniciando el programa ---")
# 1. IMPORTAMOS SOLO lo que nos interesa
# Leemos: "DEL módulo 'mis_utilidades', IMPORTA la función
    'saludar_formalmente'"
from mis_utilidades import saludar_formalmente, IMPUESTO_IVA
# 2. Usamos las herramientas DIRECTAMENTE
# ¡Fíjate que ya NO usamos el punto!
# Es como si estas funciones/variables hubieran sido creadas aquí mismo.
saludo = saludar_formalmente("Carlos")
print(saludo)
print(f"El IVA es: {IMPUESTO_IVA}")
# ⚠ PRECAUCIÓN: Esto puede ser peligroso.
# Si tú tuvieras aquí otra variable que también se llamara IMPUESTO_IVA,
# la que importas sobrescribiría la tuya (o viceversa, dependiendo del orden).
# Por eso, el 'import module' básico con punto suele ser más seguro.
```

Variante B: Usar apodos o alias (as)

A veces, el nombre de un módulo es muy largo o complicado, y no quieres estar escribiéndolo todo el rato. Python te permite ponerle un "mote" o alias usando la palabra clave `as`.

La Analogía: Tu amigo se llama "Francisco Javier Rodríguez de la Fuente", pero tú para ir rápido le llamas "Fran".

Ejemplo 1: Alias para el módulo entero

```
# --- ARCHIVO: programa.py (Versión con alias de módulo) ---
# Leemos: "Importa 'mis_utilidades' pero llámalo 'util' para abreviar"
import mis_utilidades as util
# Ahora usamos 'util.' en lugar del nombre largo
precio = util.calcular_precio_con_iva(50)
print(f"Precio final: {precio}")
```

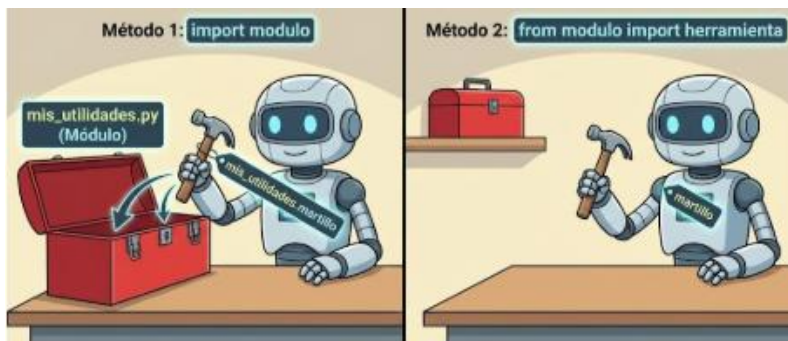
Ejemplo 2: Alias para una función específica

Esto es súper útil si importas dos funciones que se llaman igual de distintos módulos.

```
# Imagina que tenemos otro módulo 'otro_modulo.py' que también tiene
# 'calcular_precio_con_iva'
from mis_utilidades import calcular_precio_con_iva as calc_iva_normal
# from otro_modulo import calcular_precio_con_iva as calc_iva_especial #
# (Ejemplo hipotético)
# Usamos nuestro alias
precio = calc_iva_normal(100)
print(f"Precio: {precio}")
```

Resumen de las 3 formas de importar

Sintaxis	¿Cómo se usa?	Ventaja	Inconveniente
<code>import modulo</code>	<code>modulo.funcion()</code>	Muy claro de dónde viene todo. Evita conflictos de nombres.	Hay que escribir más (modulo.).
<code>from modulo import funcion</code>	<code>funcion()</code>	Más cómodo de escribir.	Peligro de colisión de nombres si no tienes cuidado.
<code>import modulo as alias</code>	<code>alias.funcion()</code>	Cómodo para nombres de módulo largos.	Tienes que acordarte del alias.



10.3 La Biblioteca Estándar de Python: El tesoro incluido

Python tiene una filosofía famosa llamada "**baterías incluidas**" (batteries included).

Esto significa que, al instalar Python, no solo obtienes el lenguaje básico, sino también una colección inmensa de módulos preescritos por expertos para hacer casi cualquier cosa que te imagines. ¡No tienes que reinventar la rueda!

La Analogía: La caja de herramientas vacía vs. El taller profesional

- **Otros lenguajes:** A veces, te dan una caja de herramientas vacía. Si quieres un martillo, tienes que fabricarlo tú mismo o buscar dónde comprarlo.
- **Python:** Te da las llaves de un Home Depot o Leroy Merlin gigantesco. ¿Necesitas hacer cálculos matemáticos complejos? Hay un pasillo para eso. ¿Necesitas manejar fechas y horas? Hay otro pasillo. ¿Necesitas conectarte a internet? También lo hay.

Todo esto ya está en tu ordenador, esperando a que uses la palabra mágica `import`.

Vamos a ver tres de los módulos estándar más populares e imprescindibles:

1. El módulo math: Para los genios de las matemáticas

Si necesitas hacer algo más complejo que sumar y restar, no intentes programar tú mismo cómo calcular una raíz cuadrada o el valor de Pi. Usa el experto.

Importamos el módulo de matemáticas

```
import math
print("--- Ejemplo con el módulo 'math' ---")
# Usamos una constante del módulo: PI
print(f"El valor de Pi es: {math.pi}")
# Usamos una función: raíz cuadrada (square root)
numero = 49
raiz = math.sqrt(numero)
print(f"La raíz cuadrada de {numero} es: {raiz}")
# Usamos otra función: redondear hacia arriba (techo/ceiling)
nota_examen = 9.1
nota_final = math.ceil(nota_examen)
print(f"La nota redondeada hacia arriba es: {nota_final}")
```

2. El módulo random: El azar y la suerte

Este es fundamental para juegos, simulaciones o cualquier cosa que necesite un comportamiento impredecible.

```
# Importamos el módulo de aleatoriedad
import random
print("\n--- Ejemplo con el módulo 'random' ---")

# Generar un número entero aleatorio entre dos valores (ambos incluidos)
dado = random.randint(1, 6)
print(f"Has tirado un dado y ha salido un: {dado}")
# Generar un número decimal (float) aleatorio entre 0.0 y 1.0
probabilidad = random.random()
```

```
print(f"El color elegido por la suerte es: {color_elegido}")
```

3. El módulo datetime: Controlando el tiempo

Manejar fechas y horas manualmente es una pesadilla (años bisiestos, zonas horarias...). Deja que este módulo sufra por ti.

```
print(f"Fecha formateada: {fecha_bonita}")
```



10.4 Creando tus propios módulos "profesionales"

Ya sabemos la regla de oro: **Cualquier archivo .py es un módulo.**

Pero hay un pequeño problema muy común cuando empezamos. Imagina que estás escribiendo un módulo con funciones matemáticas increíbles. Para probar si funcionan, añades unos `print()` al final del archivo.

```
# --- ARCHIVO: mis_mates_v1.py (Versión problemática) ---
def sumar(a, b):
    return a + b
# Pruebas que añadido al final para ver si funciona
print("--- Probando mi módulo ---")
print(f"La suma de 2+3 es {sumar(2, 3)}")
```

Funciona genial cuando lo ejecutas. Pero ahora, imagina que en tu programa principal (`main.py`) solo quieres usar la función `sumar`, sin que salga el texto de prueba.

```
# --- ARCHIVO: main.py ---
import mis_mates_v1 # ¡ALERTA! Al importar, Python ejecuta TODO el archivo.
resultado = mis_mates_v1.sumar(10, 10)
print(f"Resultado principal: {resultado}")
# SALIDA EN CONSOLA:
# --- Probando mi módulo --- <-- ¡ESTO NO DEBERÍA SALIR AQUÍ!
# La suma de 2+3 es 5 <-- ESTO TAMPOCO
# Resultado principal: 20
```

¡Qué desastre! Al importar el módulo para usar una herramienta, se nos han ejecutado las pruebas que tenía dentro.

El secreto profesional: `if __name__ == "__main__":`

Para evitar esto, los programadores de Python usan un "bloque mágico" al final de sus módulos. Es una de esas cosas que al principio parecen jeroglíficos, pero es muy fácil de entender.

La Analogía: El Actor de Cine

- Un archivo .py es como un actor famoso.
- **Cuando está "en el set de rodaje" (cuando lo ejecutas directamente):** El actor actúa, dice sus frases y hace las escenas de acción (ejecuta el código de prueba).
- **Cuando está "dando una entrevista" (cuando es importado por otro):** El actor se comporta con normalidad, no se pone a gritar las frases de su película en mitad de la entrevista. Solo está ahí disponible para responder preguntas (ofrecer sus funciones).

¿Cómo sabe el archivo si está en el set o en una entrevista? Mirando su "carnet de identidad" secreto, una variable interna llamada `__name__`.

- Si ejecutas el archivo directamente, su `__name__` vale `"__main__"` (soy el principal).
- Si lo importas, su `__name__` vale el nombre del archivo (ej. `"mis_mates_v1"`).

La estructura correcta de un módulo

Vamos a reescribir nuestro módulo matemático usando esta técnica. Esta es la plantilla que deberías usar siempre.

```
# --- ARCHIVO: mis_mates_pro.py (Versión Profesional) ---
"""
Este es el docstring del módulo.
Aquí explicas brevemente qué hace este archivo.
Por ejemplo: Módulo con operaciones matemáticas avanzadas.
"""

# 1. Zona de IMPORTS (si los necesitaras)
# import math

# 2. Definición de CONSTANTES (variables globales útiles)
NUMERO_AUREO = 1.618

# 3. Definición de FUNCIONES y CLASES
def sumar_pro(a, b):
    """Devuelve la suma de dos números."""
    return a + b

def es_par(numero):
    """Devuelve True si el número es par."""
    return numero % 2 == 0

# =====
# EL BLOQUE MÁGICO: Zona de Pruebas/Ejecución
# =====
# Esta línea significa: "Si este archivo se está ejecutando DIRECTAMENTE
# (no ha sido importado), entonces entra aquí".
if __name__ == "__main__":
    print("--- EJECUTANDO PRUEBAS DEL MÓDULO 'mis_mates_pro' ---")
```

```
print("Esto solo se ve si ejecutas este archivo directamente.")
```

```
test_suma = sumar_pro(5, 5)
```

```
print(f"Prueba de suma (5+5): {test_suma}")
```

```
test_par = es_par(4)
```

```
print(f"Prueba de par (4): {test_par}")
```

else:

```
# Esto es opcional, pero para que veas la diferencia.
```

```
# Esto ocurre si el módulo es importado.
```

```
print("--> Módulo 'mis_mates_pro' importado con éxito.")
```

Prueba de fuego:

1. Si ejecutas `mis_mates_pro.py` directamente, verás las pruebas.
2. Si creas otro archivo `main.py` y pones `import mis_mates_pro`, solo verás el mensaje de "importado con éxito", pero **NO** se ejecutarán las pruebas de suma. ¡Limpio y profesional!

¿Se entiende la utilidad de este bloque `if __name__ == "__main__":`?
Es la diferencia entre un script de aficionado y un módulo reutilizable.



¡Estupendo! Ya dominas los módulos individuales. Has subido de nivel.

Pero ahora imagínate que estás creando un videojuego gigante o un sistema complejo como un banco online. Vas a acabar teniendo cientos de archivos .py (módulos). Si los dejas todos sueltos en una sola carpeta, ¡eso va a parecer el escritorio de un adolescente en época de exámenes!

Necesitamos una forma de organizar esos módulos en grupos lógicos. Aquí es donde entran los **Paquetes**.

10.5 ¿Qué es un paquete? Carpetas con superpoderes

Si un **módulo** es como un **cajón** de herramientas, un **paquete** es como el **armario entero** donde guardas varios cajones relacionados.

En términos técnicos de Python:

Un paquete es simplemente una CARPETA (directorio) que contiene módulos y un archivo especial.

La señal secreta: `__init__.py`

Para que Python entienda que una carpeta normal y corriente de tu ordenador es un "Paquete de Python" importable, esa carpeta suele contener un archivo muy especial llamado `__init__.py`.

- Tiene guiones bajos dobles al principio y al final.
- A menudo **está vacío**. ¡Y eso está bien!
- Su única función es decirle a Python: "Oye, esta carpeta no es una carpeta cualquiera, es un paquete que contiene módulos".

(Nota: En versiones muy modernas de Python (3.3+) ya no es estrictamente obligatorio, pero sigue siendo una buenísima práctica y se usa en casi todos los proyectos para evitar ambigüedades).

Estructura de ejemplo

Imagina que estamos haciendo un programa para manejar imágenes. Podríamos organizarlo así en nuestro disco duro:

```
mi_proyecto_fotos/    <-- Carpeta raíz del proyecto
|
|-- programa_principal.py <-- Desde aquí ejecutamos todo
|
└─ utilidades_imagen/  <-- ¡ESTO ES EL PAQUETE! (Es una carpeta)
    |
    ├── __init__.py    <-- El archivo mágico que marca la carpeta como paquete
    ├── cargar.py      <-- Módulo 1 (para leer archivos jpg, png...)
    └─ filtros.py      <-- Módulo 2 (para poner blanco y negro, sepia...)
```

Cómo importar desde un paquete

Ahora, la sintaxis de import utiliza puntos para navegar por las carpetas. Es como una dirección: Carpeta.Archivo.

Supongamos que en filtros.py hay una función llamada aplicar_sepia().

En tu programa_principal.py harías esto:

```
# --- ARCHIVO: programa_principal.py ---
# OPCIÓN A: Importar usando la ruta completa
import utilidades_imagen.filtros
print("Aplicando filtro...")
# La ruta es larga para usar la función:
imagen_vieja = utilidades_imagen.filtros.aplicar_sepia(mi_foto)
# OPCIÓN B (Más común): Usar 'from ... import ...' para navegar
# Leemos: "DESDE el paquete 'utilidades_imagen', IMPORTA el módulo 'filtros'"
from utilidades_imagen import filtros
print("Aplicando filtro...")
# Ahora es más corto:
imagen_vieja = filtros.aplicar_sepia(mi_foto)
# OPCIÓN C (Ir al grano): Importar la función directamente
# Leemos: "DESDE el módulo 'filtros' que está en el paquete
# 'utilidades_imagen'..."
from utilidades_imagen.filtros import aplicar_sepia
print("Aplicando filtro...")
# Usamos la función directamente:
imagen_vieja = aplicar_sepia(mi_foto)
```

¡Es así de fácil! Los paquetes son solo una forma de usar carpetas para que tu código no sea un caos de archivos sueltos.



10.6 Introducción a pip y PyPI: El supermercado global

Imagina que la Biblioteca Estándar es como el kit de herramientas básicas que te viene con la casa. Pero ahora quieres construir una nave espacial. Necesitas piezas especializadas que no tienes. ¿Qué haces? Vas a la tienda gigante a comprarlas.

1. PyPI (Python Package Index): La gran tienda en la nube

PyPI (se pronuncia "pai-pi-ai") es el **Índice de Paquetes de Python**. Es un repositorio gigantesco en internet (pypi.org) donde miles de programadores de todo el mundo suben sus propios paquetes de forma gratuita para que otros los usen.

Hay cientos de miles de paquetes para todo lo que te imagines:

- **Juegos:** pygame
- **Ciencia de datos:** pandas, numpy
- **Desarrollo web:** Django, Flask
- **Inteligencia artificial:** tensorflow, scikit-learn

¡Es el supermercado de código más grande del mundo!

2. pip: Tu servicio de reparto a domicilio

Tú no puedes ir físicamente a la nube a coger los paquetes. Necesitas alguien que vaya, los busque, los traiga y los instale en tu ordenador.

Ese alguien es **pip** (Package Installer for Python). Es un pequeño programa que se instaló en tu ordenador junto con Python.

La Analogía: La compra online

1. **Tú:** Quieres una librería para poner texto de colores en la terminal.
2. **PyPI:** Es el almacén gigante de Amazon donde está esa librería.
3. **pip:** Es el camión de reparto que va al almacén, recoge el paquete y lo deja en la puerta de tu casa (tu instalación de Python) listo para usar.

Cómo usar pip (¡En la terminal!)

A diferencia del código Python que escribimos en archivos .py, **pip es un comando que se ejecuta en la terminal (o consola) de tu sistema operativo** (Command Prompt en Windows, Terminal en Mac/Linux).

Vamos a probar a instalar un paquete muy divertido y sencillo llamado **colorama**, que sirve para poner texto de colores en la consola.

Paso 1: Abre tu terminal. (En VS Code, puedes ir al menú "Terminal" -> "New Terminal").

Paso 2: Escribe el comando mágico y pulsa Enter.

```
pip install colorama
```

Verás que aparecen unas barras de progreso y textos diciendo "Downloading..." e "Installing...". Cuando termine, te dirá algo como "Successfully installed colorama".

Paso 3: ¡Úsalo en tu código Python! Ahora que el "camión de reparto" ha dejado el paquete, puedes importarlo como si fuera un módulo estándar.

Crea un archivo de prueba (ej. prueba_colores.py):

```
# Importamos cosas del paquete que acabamos de instalar
from colorama import Fore, Back, Style, init
# Inicializamos colorama (necesario en Windows para que funcionen los
    colores)
init(autoreset=True)
print("--- Probando el paquete externo 'colorama' ---")
# Usamos las herramientas del paquete para cambiar el color del texto
print(Fore.RED + "¡Este texto es rojo!")
print(Fore.GREEN + "¡Este texto es verde!")
print(Back.YELLOW + Fore.BLACK + "¡Fondo amarillo y texto negro!")
print(Style.BRIGHT + Fore.CYAN + "¡Texto cian brillante!")
print("Este texto vuelve a ser normal gracias al autoreset=True.")
```

¡Si ejecutas ese código, verás una explosión de color en tu terminal! Has logrado extender las capacidades de Python usando código que escribió otra persona.



Capítulo 11: Cuando las Cosas Salen Mal - Manejo de Errores y Excepciones

Empezamos este capítulo aceptando una verdad universal de la programación: **Los errores van a ocurrir. Siempre.**

Tu trabajo no es evitar que ocurran (eso es imposible), sino anticiparte a ellos y gestionarlos con elegancia para que tu programa no se cierre de golpe en la cara del usuario.

11.1 Los errores son inevitables: Conoce a tu enemigo

Antes de aprender a luchar contra ellos, debemos entender qué tipos de errores existen. En Python, podemos clasificarlos en tres grandes familias.

La Analogía: Escribiendo una receta de cocina

Imagina que estás escribiendo una receta para hacer un pastel.

1. Errores de Sintaxis (Syntax Errors): "No entiendo tu letra"

Son errores gramaticales en tu código. Python ni siquiera intenta ejecutar el programa porque no entiende lo que has escrito. Es como si en la receta escribieras "Hvrno a 180 grad°s" en lugar de "Horno a 180 grados". El cocinero se detiene antes de empezar.

```
# --- Ejemplo de Error de Sintaxis ---  
# Falta el paréntesis de cierre al final  
print("Hola mundo"  
# Python te gritará inmediatamente:  
# SyntaxError: '(' was never closed
```

Son los más fáciles de corregir porque Python te dice exactamente dónde está el problema antes de empezar.

2. Errores en Tiempo de Ejecución (Excepciones): "¡La cocina está ardiendo!"

El código está bien escrito gramaticalmente. Python empieza a ejecutarlo paso a paso. Pero, de repente, ocurre algo inesperado que hace que sea imposible continuar. Es como si la receta dice "Añadir 3 huevos", pero al ir a la nevera, te das cuenta de que **no quedan huevos**. ¡Pánico! El proceso se detiene en seco.

En Python, estos eventos inesperados se llaman **Excepciones**.

```
# --- Ejemplo de Excepción (Runtime Error) ---  
# La sintaxis es correcta, pero matemáticamente esto es imposible.  
resultado = 10 / 0  
# Python ejecutará el código hasta llegar a esta línea y ¡BUM!:  
# ZeroDivisionError: division by zero
```

Otros ejemplos clásicos: intentar abrir un archivo que no existe (FileNotFoundError), o intentar convertir "hola" en un número (ValueError).

3. Errores Lógicos (Bugs): "El pastel sabe a rayos"

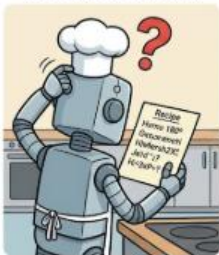
Estos son los peores. El programa se ejecuta perfectamente de principio a fin. No hay errores de sintaxis ni excepciones. Pero el resultado es incorrecto. Es como si sigues la receta al pie de la letra, pero en vez de azúcar pusiste sal por error. El pastel sale, pero está incomible.

```
# --- Ejemplo de Error Lógico ---
# Queremos calcular el promedio de dos notas: 8 y 10.
nota1 = 8
nota2 = 10
# ¡ERROR EN LA FÓRMULA! Falta el paréntesis.
promedio = nota1 + nota2 / 2
# El programa funciona, pero el resultado es desastroso:
print(f"El promedio es: {promedio}")
# Salida: El promedio es: 13.0 (Debería ser 9.0)
```

Python no puede ayudarte aquí. Eres tú contra tu propia lógica. Se resuelven probando y revisando el código con calma (debugging).

En este capítulo, nos vamos a centrar en dominar los del segundo tipo: las **Excepciones**. Vamos a aprender a capturarlas al vuelo para que, si faltan huevos, el programa no explote, sino que nos diga amablemente: "Disculpe, no se puede continuar porque faltan ingredientes".

1. ERROR DE SINTAXIS
("No entiendo tu letra")



¡El programa ni siquiera empieza!

2. EXCEPCIÓN
("¡Faltan ingredientes!")



¡Se detiene en seco a mitad del proceso!

3. ERROR LÓGICO
("Sal en vez de azúcar")



Funciona hasta el final...
¡pero el resultado es incorrecto!

11.2 El bloque try-except: Tu red de seguridad

El mecanismo para manejar errores en Python se llama try-except (intentar-excepto).

La Analogía: El Trapecista y la Red

Imagina que tu código es un trapecista haciendo una pirueta peligrosa en el circo.

1. **Sin try-except:** Si el trapecista falla, cae al suelo duro. El espectáculo se acaba abruptamente y el público se asusta. (El programa se rompe).
2. **Con try-except:** Instalas una red de seguridad debajo.
 - El trapecista **intenta (try)** hacer la pirueta.
 - Si todo va bien, sigue el espectáculo.
 - Si falla, **cae en la red (except)**. No se hace daño, se levanta, saluda al público y el espectáculo puede continuar.

¿Cómo se usa en código?

Envolvemos la parte "peligrosa" del código (la pirueta) dentro de un bloque try, y definimos qué hacer en caso de caída en el bloque except.

Vamos a verlo con el ejemplo clásico: **La división por cero**.

El programa suicida (Sin red)

```
# --- Programa SIN protección ---
print("Iniciando programa de división...")
numero = 10
divisor = 0 # ¡PELIGRO!
resultado = numero / divisor # <-- Aquí ocurrirá la catástrofe
print(f"El resultado es: {resultado}") # Esta línea NUNCA se ejecutará
print("Programa terminado correctamente.") # Esta tampoco
```

Si ejecutas esto, obtendrás un feo `ZeroDivisionError` y el programa morirá instantáneamente en la línea de la división.

El programa robusto (Con red try-except)

Vamos a proteger la línea peligrosa.

```
# --- Programa CON protección try-except ---
print("Iniciando programa de división segura...")
numero = 10
divisor = 0
try:
    # BLOQUE TRY: "Intenta hacer esto"
    # Aquí ponemos el código peligroso que podría fallar.
    print("Intentando dividir...")
    resultado = numero / divisor
    # Si la línea de arriba falla, saltamos INMEDIATAMENTE al 'except'.
    # Si no falla, seguimos aquí.
    print(f"¡Éxito! El resultado es: {resultado}")
except:
    # BLOQUE EXCEPT: "Si algo sale mal en el 'try', ven aquí"
    # Esta es nuestra red de seguridad.
    print("¡ERROR! ¡Ups! Ha ocurrido un problema.")
    print("No se puede dividir por cero (o ha pasado otra cosa rara).")
```

Aquí podríamos asignar un valor por defecto al resultado, por ejemplo.
resultado = 0

```
# El programa continúa su ejecución normal después del bloque try/except
print("\n---")
print(f"El valor final de la variable resultado es: {resultado}")
print("Programa terminado correctamente. ¡Seguimos vivos!")
```

Salida del programa protegido:

Iniciando programa de división segura...

Intentando dividir...

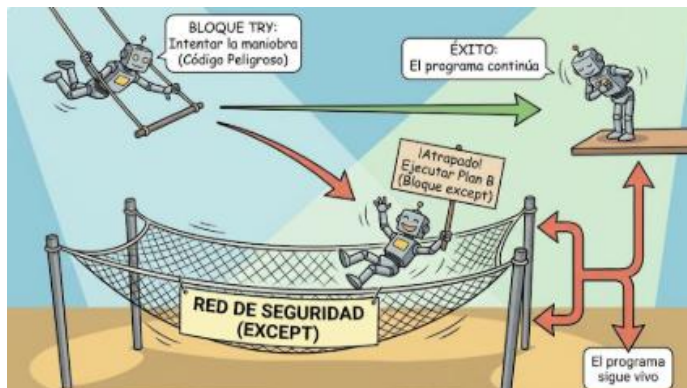
¡ERROR! ¡Ups! Ha ocurrido un problema.

No se puede dividir por cero (o ha pasado otra cosa rara).

El valor final de la variable resultado es: 0

Programa terminado correctamente. ¡Seguimos vivos!

¡Fíjate en la diferencia! El programa **no se ha roto**. Ha detectado el problema, ha ejecutado el código de emergencia (el bloque except) y luego ha seguido funcionando hasta el final.



¡Perfecto! Has captado la idea de la "red de seguridad" a la primera.

Ahora vamos a refinar esa técnica. Usar un except genérico (como hicimos antes) es como tener una red gigante que atrapa cualquier cosa: un trapecista, un elefante o un payaso.

Pero en programación, a veces necesitas reaccionar de forma diferente según qué haya sido lo que falló. No es lo mismo que el usuario intente dividir por cero, a que el archivo que buscas no exista.

11.3 Capturando excepciones específicas: Un plan para cada desastre

Python tiene "nombres y apellidos" para casi cualquier tipo de error imaginable. Cuando ocurre una excepción, Python "levanta" una bandera con el nombre específico del error.

Nosotros podemos preparar nuestros bloques except para que estén atentos a banderas concretas.

La Analogía: Los equipos de emergencia especializados

Volvamos al circo. Imagina que en lugar de una sola red, tienes varios equipos de emergencia esperando:

1. **Equipo Médico (ValueError):** Si el trapecista se marea (el usuario mete texto en vez de número).
2. **Equipo de Bomberos (ZeroDivisionError):** Si se prende fuego el aro (intentas dividir por cero).
3. **Equipo de Control de Animales (FileNotFoundError):** Si se escapa el león (no encuentras el archivo).

Cuando ocurre un accidente, solo sale el equipo correspondiente. ¡No quieres que los bomberos salgan a poner una tiritita!

Cómo se hace en código

Podemos encadenar varios bloques except, cada uno con el nombre del error que quiere atrapar.

Vamos a crear un programa que pida un número al usuario y divida 100 entre ese número. Este programa puede fallar de dos maneras principales:

1. Que el usuario escriba "cero".
2. Que el usuario escriba "hola" (texto).

```
print("--- Programa de División Avanzado ---")
```

```
try:
```

```
    # BLOQUE TRY: El código peligroso
```

```
    entrada_usuario = input("Introduce un número divisor para 100: ")
```

```
    # PELIGRO 1: Convertir texto a número. Puede fallar si escriben "hola".
```

```
    divisor = float(entrada_usuario)
```

```
    # PELIGRO 2: La división. Puede fallar si el divisor es 0.
```

```
    resultado = 100 / divisor
```

```
    print(f"¡Éxito! 100 dividido entre {divisor} es: {resultado:.2f}")
```

```
except ValueError:
```

```
    # ESTE BLOQUE SOLO SALTA SI FALLA LA CONVERSIÓN (float())
```

```
    # Es el "Equipo Médico"
```

```
    print("¡ERROR DE VALOR! No has introducido un número válido.")
```

```
    print("Por favor, usa dígitos (ej: 5, 10.5), no texto.")
```

```
except ZeroDivisionError:
```

```
    # ESTE BLOQUE SOLO SALTA SI EL DIVISOR ES CERO
```

```
    # Es el "Equipo de Bomberos"
```

```
    print("¡ERROR MATEMÁTICO! No se puede dividir por cero.")
```

```
    print("El universo implosionaría si lo intentáramos.")
```

```
except:
```

```
# BLOQUE GENÉRICO (Opcional pero útil como último recurso)
# Atrapa cualquier otra cosa rara que no hayamos previsto.
print("¡ERROR DESCONOCIDO! Ha pasado algo muy raro que no
esperábamos.")
print("\n--- El programa continúa y finaliza correctamente ---")
```

Prueba este código tú mismo tres veces:

1. Introduce 5. (Funcionará bien).
2. Introduce 0. (Saltará el ZeroDivisionError).
3. Introduce hola. (Saltará el ValueError).

💡 Truco Pro: No uses la "red gigante" si puedes evitarlo

Es una mala práctica usar un except: "pelado" (sin nombre de error) para todo. ¿Por qué? Porque puede ocultar errores graves que tú *no* querías atrapar (como un error tipográfico en el nombre de una variable).

Es mucho mejor ser específico: **Atrapa solo los errores que esperas y sabes cómo manejar.**



¡Perfecto! Ya tenemos a nuestros equipos de emergencia especializados listos para actuar.

Pero ahora imagina esto: el espectáculo del circo termina. Haya salido bien la pirueta (éxito en el try) o haya salido mal y el trapecista haya caído en la red (capturado en el except), hay algo que **siempre** tiene que pasar al final: hay que apagar las luces y cerrar la carpa.

En programación, esto es crucial. A veces abrimos recursos "caros" (como un archivo, una conexión a una base de datos o a internet) que debemos cerrar obligatoriamente, pase lo que pase, para no dejar el sistema sucio.

Aquí entra el bloque finally.

11.4 El bloque finally: Lo que sucede sí o sí

El bloque finally es la tercera pata opcional de la estructura try-except. Se coloca al final de todo.

La regla de oro: El código dentro de finally se ejecutará **SIEMPRE**, sin importar si hubo un error en el try o si todo salió perfecto.

La Analogía: Cerrar el circo

- **try:** El trapecista intenta el salto.
- **except:** Si cae, la red lo atrapa.
- **finally:** Pase lo que pase (éxito o caída), al final del número, el conserje siempre apaga las luces de la pista.

Ejemplo Práctico: Simulando una conexión

Como aún no hemos visto cómo abrir archivos de verdad (¡eso es el próximo capítulo!), vamos a simular una conexión a un servidor imaginario.

Observa el flujo de ejecución en los dos escenarios.

Escenario A: Todo sale BIEN

```
print("--- INICIO: Escenario SIN errores ---")
try:
    print("1. [TRY] Abriendo conexión al servidor...")
    # Simulamos que trabajamos con la conexión
    print("2. [TRY] Descargando datos importantes...")
    print("3. [TRY] ¡Datos descargados correctamente!")
except Exception as e:
    # Esto NO se ejecutará porque no hubo error
    print(f"4. [EXCEPT] ¡Ups! Algo falló: {e}")
finally:
    # Esto se ejecuta SIEMPRE
    print("5. [FINALLY] Cerrando la conexión al servidor (Limpieza).")
print("--- FIN: El programa sigue su curso ---")
```

Salida A:

Plaintext

```
--- INICIO: Escenario SIN errores ---
1. [TRY] Abriendo conexión al servidor...
2. [TRY] Descargando datos importantes...
3. [TRY] ¡Datos descargados correctamente!
5. [FINALLY] Cerrando la conexión al servidor (Limpieza). <-- ¡Se ejecutó!
--- FIN: El programa sigue su curso ---
```

Escenario B: Algo sale MAL

Ahora vamos a provocar un error a propósito dentro del try.

```
print("\n--- INICIO: Escenario CON error ---")
try:
    print("1. [TRY] Abriendo conexión al servidor...")
    # Simulamos un error fatal
    print("2. [TRY] Intentando descargar...")
    provocar_error = 1 / 0 # ¡Zas! División por cero.
    print("3. [TRY] ¡Esto nunca se imprimirá!")
except ZeroDivisionError:
    # Atrapamos el error
    print("4. [EXCEPT] ¡Error detectado! No se pudo descargar.")
finally:
    # ¡ESTO SE EJECUTA IGUALMENTE!
    print("5. [FINALLY] Cerrando la conexión al servidor (Limpieza OBLIGATORIA).")
print("--- FIN: El programa se recuperó y sigue ---")
```

Salida B:

```
--- INICIO: Escenario CON error ---
1. [TRY] Abriendo conexión al servidor...
2. [TRY] Intentando descargar...
4. [EXCEPT] ¡Error detectado! No se pudo descargar.
5. [FINALLY] Cerrando la conexión al servidor (Limpieza OBLIGATORIA). <--
    ¡También se ejecutó!
--- FIN: El programa se recuperó y sigue ---
```

⚠ Advertencia Pro: finally es muy testarudo

El bloque finally es tan potente que incluso si dentro del bloque try hay una instrucción return (que normalmente acaba la función inmediatamente), Python **primero** ejecutará el finally antes de salir de verdad de la función. ¡Es la garantía última de limpieza!



¡Vamos a por la última pieza del rompecabezas de los errores!

Hasta ahora, hemos jugado un papel defensivo. Hemos puesto redes (try-except) para atrapar los errores que Python nos lanza (como dividir por cero o no encontrar un archivo).

Pero, ¿qué pasa si queremos ser nosotros los que "provoquemos" el error a propósito?

11.5 Lanzando tus propias excepciones con raise

A veces, tu programa llega a una situación que técnicamente para Python es correcta, pero **lógicamente para tu aplicación es un desastre**.

La Analogía: El Árbitro de Fútbol

- **Errores de Python (Gravedad):** Si un jugador corre y tropieza, se cae. Es una ley física inevitable. Python detecta la caída (`ZeroDivisionError`).
- **Tus Errores (raise):** Un jugador toca el balón con la mano. Físicamente es posible, pero va contra las reglas del juego. Tú eres el árbitro. Tienes que soplar el silbato, parar el juego y **levantar (raise) una tarjeta roja**.

¿Cuándo usar raise?

Imagina que estás haciendo un programa para registrar usuarios y les pides su edad.

Para Python, el número -50 es un número entero perfectamente válido. Si haces cálculos con él, Python no se quejará.

Pero para tu programa, ¡una edad de -50 años es imposible! Aquí es donde tú, como programador árbitro, debes intervenir y lanzar una excepción.

Cómo se usa en código

Usamos la palabra clave `raise` (que significa "levantar" o "elevar"), seguida del tipo de excepción que queremos lanzar y un mensaje descriptivo.

```
def registrar_usuario(nombre, edad):
    print(f"Intentando registrar a {nombre} con {edad} años...")
    # VALIDACIÓN: Nosotros definimos las reglas del juego.
    if edad < 0:
        # ¡ESTO NO PUEDE SER! Soplamos el silbato.
        # Levantamos una excepción de tipo ValueError (error de valor)
        # y le ponemos un mensaje personalizado.
        raise ValueError("¡La edad no puede ser negativa!")
    if edad > 150:
        # Otra regla nuestra.
        raise ValueError("¡Nadie vive tanto tiempo, comprueba el dato!")
    # Si pasa las validaciones, todo está bien.
    print(f"✅ Usuario {nombre} registrado correctamente.")
# --- Programa Principal ---
print("--- Inicio del registro ---")
try:
    # Caso 1: Todo bien
    registrar_usuario("Ana", 30)
    # Caso 2: Esto va a provocar NUESTRO error personalizado
    registrar_usuario("Bender", -500)
except ValueError as e:
    # Atrapamos nuestro propio error
    # La variable 'e' contiene el mensaje que escribimos en el raise
    print(f"\n❌ ¡ALERTA! El árbitro ha pitado falta.")
    print(f"El motivo del error es: {e}")
print("\n--- Fin del programa ---")
```

Salida:

--- Inicio del registro ---

Intentando registrar a Ana con 30 años...

✅ Usuario Ana registrado correctamente.

Intentando registrar a Bender con -500 años...

❌ ¡ALERTA! El árbitro ha pitado falta.

El motivo del error es: ¡La edad no puede ser negativa!

--- Fin del programa ---

¡Fíjate en lo que ha pasado! Cuando intentamos registrar a Bender, la función `registrar_usuario` detectó el problema y ejecutó el `raise`. Eso detuvo inmediatamente la función (como un `return` abrupto) y lanzó la "tarjeta roja" hacia fuera. Nuestro bloque `try-except` principal atrapó esa tarjeta y mostró el mensaje.

Es una forma súper potente de **proteger tus funciones** para que nadie las use con datos incorrectos.



Capítulo 12: Manejando Archivos - Memoria a Largo Plazo

12.1 RAM vs. Disco: El cerebro y la libreta

Para entender por qué necesitamos archivos, pensemos en cómo funciona un ordenador (o tu propia cabeza).

- **La Memoria RAM (Tus variables hasta ahora):** Es como tu **cerebro en este momento**. Es rapidísima pensando y calculando. Pero si te vas a dormir (apagas el ordenador), se te olvida el número de teléfono que te acaban de decir si no lo apuntaste. Es **volátil**.
- **El Disco Duro (Los archivos):** Es como una **libreta física**. Es mucho más lento escribir y leer de una libreta que simplemente pensarlo. Pero si escribes algo ahí, cierras la libreta y te vas a dormir... ¡mañana seguirá estando ahí! Es **persistente**.

En programación, usamos la RAM para trabajar rápido, y el disco para guardar los resultados finales.

12.2 Abriendo y cerrando la "libreta"

Trabajar con archivos en Python es como trabajar con una libreta física. No puedes escribir en ella si no la abres primero, y debes cerrarla cuando termines para que no se arruguen las hojas.

Paso 1: Abrir el archivo con `open()`

La función `open()` es la llave. Necesita dos cosas principales:

1. **El nombre del archivo:** ¿Qué libreta quieres? (ej. "mis_notas.txt")
2. **El modo:** ¿Qué vas a hacer con ella? ¿Solo leer? ¿Escribir desde cero? ¿Añadir algo al final?

Los modos más importantes (¡tatúatelos en la mente!):

- **'r' (Read / Leer):** Modo por defecto. Abre para leer. **Da error si el archivo no existe.**
- **'w' (Write / Escribir):** Abre para escribir. **¡PELIGRO!** Si el archivo ya existía, **BORRA todo su contenido** y empieza de cero. Si no existe, lo crea.
- **'a' (Append / Añadir):** El más seguro para escribir. Abre para escribir, pero **añade al final** sin borrar lo que había. Si no existe, lo crea.

Paso 2: Cerrar el archivo con `.close()`

Esto es CRUCIAL. Si abres un archivo y te olvidas de cerrarlo, los datos podrían no guardarse bien o el archivo podría quedar bloqueado para otros programas.

La forma antigua (y peligrosa) de hacerlo:

```
# --- Forma ANTIGUA (No recomendada) ---
archivo = open("prueba_antigua.txt", "w") # Abrimos en modo escritura ('w')
archivo.write("Hola, esto es una prueba arriesgada.")
# Si aquí ocurre un error y el programa se rompe...
# ... la línea siguiente NUNCA se ejecutará y el archivo quedará abierto.
archivo.close()
```

La forma profesional: El bloque with open(...)

Python tiene una forma maravillosa de manejar esto, llamada "Context Manager" (Gestor de Contexto), usando la palabra clave `with`.

Es como tener un asistente automático que se asegura de cerrar la libreta por ti, incluso si hay un terremoto (una excepción) mientras escribes. **Esta es la forma que usarás el 99% de las veces.**

La Analogía: La puerta con muelle de cierre automático

No tienes que preocuparte de si la cerraste o no. Se cierra sola cuando sales.

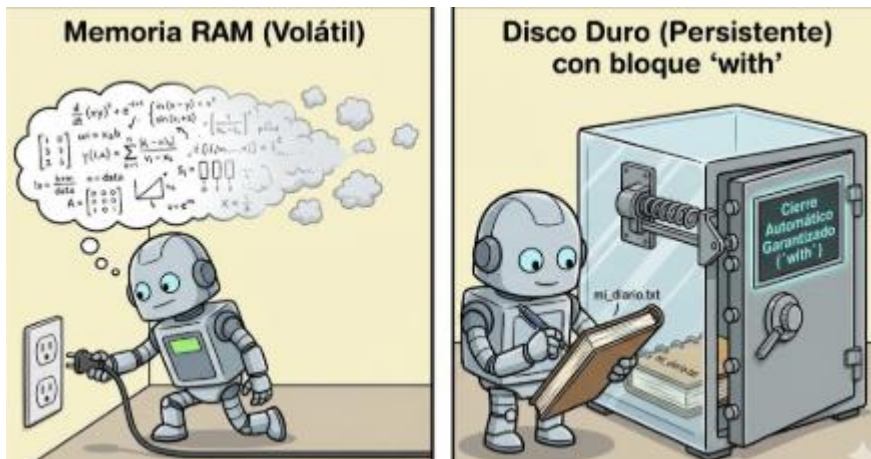
```
# --- Forma MODERNA y SEGURA (Recomendada) ---
print("Iniciando escritura segura...")
# Leemos: "CON el archivo 'mi_diario.txt' abierto en modo escritura ('w')
#     asígnale el nombre temporal 'archivo', y haz lo siguiente:"
with open("mi_diario.txt", "w") as archivo:
    # Dentro de este bloque, 'archivo' es nuestra conexión a la libreta.
    print("Escribiendo datos...")
    archivo.write("Querido diario:\n") # \n es un salto de línea
    archivo.write("Hoy he aprendido a guardar archivos en Python.\n")
```

```

archivo.write("¡Es genial!")
# Aquí podría ocurrir un error, y Python CERRARÍA el archivo
# automáticamente.
# Justo aquí, al salir del bloque 'with', Python cierra el archivo solo.
# No hace falta poner archivo.close()
print("¡Terminado! Verifica si se ha creado el archivo 'mi_diario.txt' en tu
carpeta.")

```

Si ejecutas este código, verás que aparece un nuevo archivo de texto en la misma carpeta donde tengas tu script de Python. ¡Ábrelo y verás tu mensaje!



¡Perfecto! Ya hemos escrito nuestra primera página en el diario. Ahora toca la segunda parte esencial: ser capaces de volver a abrir esa libreta y leer lo que hay dentro.

12.3 Leyendo archivos: Recuperando la memoria

Vamos a usar el mismo archivo que creamos en el paso anterior: `mi_diario.txt`. Si por alguna razón no lo creaste, el código que vamos a ver ahora fallará, ¡pero eso es bueno porque nos servirá para practicar el manejo de errores!

Para leer, usamos de nuevo `with open()`, pero esta vez cambiamos el modo a `'r'` (**Read / Leer**).

La importancia de ser precavido (`FileNotFoundError`)

¿Recuerdas el Capítulo 12 sobre excepciones? Aquí es vital. Si intentas abrir en modo lectura (`'r'`) un archivo que **no existe**, Python lanzará un error llamado `FileNotFoundError` y el programa se detendrá.

Un buen programador **siempre** envuelve la lectura de archivos en un bloque `try-except`.

Métodos para leer

Una vez abierto el archivo, hay varias formas de leerlo. Las dos más comunes son:

1. **Leerlo todo de golpe (`.read()`):** Ideal para archivos pequeños.
2. **Leerlo línea por línea (`Iterar`):** Ideal para archivos grandes, ya que no carga todo en la memoria RAM de golpe.

Vamos a ver ambos métodos en acción con nuestro diario.

```
# Nombre del archivo que queremos leer
nombre_archivo = "mi_diario.txt"
print(f"--- Intentando leer el archivo: {nombre_archivo} ---")
try:
    # 1. Intentamos abrir el archivo en modo LECTURA ('r')
    # Usamos 'with' para que se cierre solo.
    with open(nombre_archivo, 'r') as archivo_lectura:
        print("\n--- MÉTODO 1: Leyendo todo de golpe (.read()) ---")
        # .read() chupa todo el contenido y lo mete en una variable de texto
        gigante.
        contenido_completo = archivo_lectura.read()
        print(contenido_completo)
        print("-----")
    # NOTA IMPORTANTE:
    # Al salir del 'with', el archivo se ha cerrado.
    # Si queremos volver a leerlo desde el principio, hay que volver a abrirlo.
    with open(nombre_archivo, 'r') as archivo_lectura_v2:
        print("\n--- MÉTODO 2: Leyendo línea por línea (Bucle for) ---")
        # ¡Magia de Python! Podemos recorrer el archivo como si fuera una lista.
        # Esto es muy eficiente para archivos gigantes.
        numero_linea = 1
        for linea in archivo_lectura_v2:
            # .strip() es útil para quitar los espacios y saltos de línea
            # que sobran al principio y final de cada línea leída.
            linea_limpia = linea.strip()
            print(f"Línea {numero_linea}: {linea_limpia}")
            numero_linea += 1
        print("-----")
except FileNotFoundError:
    # ¡Aquí está nuestra red de seguridad del Capítulo 12!
    # Si el archivo no existe, caemos aquí en lugar de romper el programa.
    print(f"\n🚫 ¡ERROR! No se encontró el archivo '{nombre_archivo}'.")
```

```
print("Asegúrate de haber ejecutado el código del apartado 13.2 primero.")
except Exception as e:
    # Por si pasa cualquier otra cosa rara (ej. no tener permisos)
    print(f"\n❌ Ocurrió un error inesperado al leer: {e}")
print("\nFin del proceso de lectura.")
```

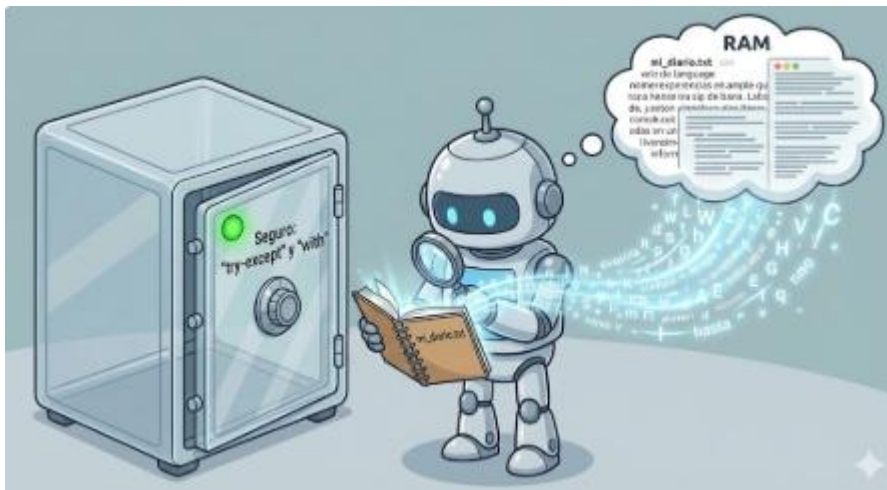
Si todo va bien, la salida será:

```
--- Intentando leer el archivo: mi_diario.txt ---
--- MÉTODO 1: Leyendo todo de golpe (.read()) ---
Querido diario:
Hoy he aprendido a guardar archivos en Python.
¡Es genial!
-----
--- MÉTODO 2: Leyendo línea por línea (Bucle for) ---
Línea 1: Querido diario:
Línea 2: Hoy he aprendido a guardar archivos en Python.
Línea 3: ¡Es genial!
-----

Fin del proceso de lectura.
```

Resumen de Modos Clave

Modo	Descripción	¿Si no existe?	¿Si existe?
'r'	Leer (Read)	Error (FileNotFoundError)	Lee desde el principio.
'w'	Escribir (Write)	Lo crea.	¡Lo borra todo! y empieza de cero.
'a'	Añadir (Append)	Lo crea.	Escribe al final, respetando lo anterior.



12.4 Escribiendo y añadiendo datos: ¡Cuidado con el borrador!

Volvamos a los modos de apertura que vimos de pasada. Los dos protagonistas para meter información en un archivo son `'w'` (write/escribir) y `'a'` (append/añadir). Confundirlos es el error más común y destructivo del principiante.

El Modo `'w'` (Write): El folio en blanco (peligroso)

Piensa en el modo `'w'` como si arrancaras la hoja de la libreta y pusieras una nueva completamente en blanco.

- **Si el archivo NO existe:** Lo crea nuevo. ¡Genial!
- **Si el archivo YA existe: ¡Lo borra todo automáticamente!** sin preguntar, y empieza a escribir desde cero.

Es útil para guardar, por ejemplo, la configuración actual de un programa, donde solo te importa el estado presente y no el histórico.

¡Vamos a probar el peligro!

```
nombre_archivo = "lista_compra.txt"

# 1. Creamos la lista inicial
print("--- Creando archivo con modo 'w' ---")
with open(nombre_archivo, "w") as archivo:
    archivo.write(" - Leche\n")
    archivo.write(" - Huevos\n")

print("Archivo creado. Contenido actual:")
with open(nombre_archivo, "r") as f: print(f.read())
```

```
# 2. ¡Ups! Se nos olvidó el pan. Volvemos a abrir con 'w'
print("\n--- Volviendo a abrir con 'w' para añadir el pan... ¡ERROR! ---")
with open(nombre_archivo, "w") as archivo:
    # ¡ALERTA! Justo al ejecutarse esta línea, el archivo se ha vaciado.
    archivo.write(" - Pan\n")

print("Contenido final tras el desastre:")
# ¡La leche y los huevos han desaparecido!
with open(nombre_archivo, "r") as f: print(f.read())
```

El Modo 'a' (Append): Escribir al final (seguro)

Este es el modo que queremos para nuestro diario o para listas que crecen. Piensa en él como abrir la libreta, buscar la última línea escrita y ponerte a escribir justo debajo.

- **Si el archivo NO existe:** Lo crea nuevo.
- **Si el archivo YA existe:** Respeta el contenido anterior y añade lo nuevo al final.

¡Vamos a arreglar la lista de la compra!

```
print("\n-----")
print("--- Usando el modo 'a' (Append) para arreglarlo ---")

# Volvemos a crear la lista base con 'w' (para empezar limpio el ejemplo)
with open(nombre_archivo, "w") as archivo:
    archivo.write(" - Leche\n")
    archivo.write(" - Huevos\n")
print("Lista inicial restaurada.")

# AHORA SÍ: Usamos 'a' para añadir cosas
print("\nAñadiendo elementos con modo 'a'...")
with open(nombre_archivo, "a") as archivo:
```

Es importante añadir el salto de línea (\n) tú mismo

```
archivo.write(" - Pan\n")  
archivo.write(" - Café\n")
```

```
print("Contenido final CORRECTO:")
```

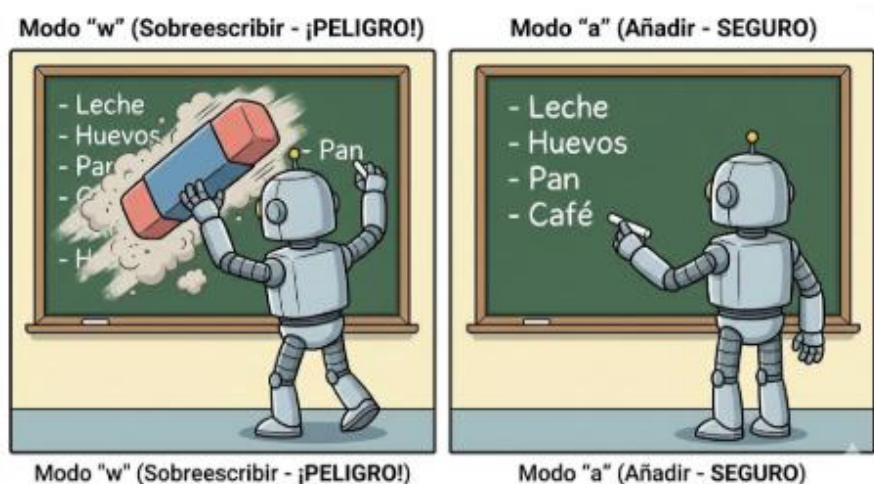
```
# ¡Ahora está todo!
```

```
with open(nombre_archivo, "r") as f: print(f.read())
```

💡 Un pequeño detalle importante: El salto de línea \n

Te habrás fijado en ese extraño \n al final de cada frase. La función `.write()` es muy literal: escribe exactamente lo que le dices. Si no pones \n (que es el código invisible para "pulsar Enter"), te escribirá todo seguido en una sola línea larguísima: - Leche - Huevos - Pan - Café.

¡Acuérdate siempre de ponerlo si quieres que cada cosa vaya en su línea!



¡Vamos a por la pieza final del rompecabezas de los archivos!

Hasta ahora, hemos asumido que el archivo de texto (`mi_diario.txt` o `lista_compra.txt`) estaba mágicamente en la misma carpeta donde tenías guardado tu script de Python (`.py`).

Esto se llama usar una **ruta relativa**. Es cómodo para empezar, pero en el mundo real, los proyectos se organizan en carpetas. Imagina un videojuego: tendrá una carpeta para música, otra para gráficos y otra para guardar las partidas.

Si tu programa no sabe dónde buscar, te encontrarás con el temido `FileNotFoundError`. Necesitamos un mapa.

12.5 Rutas de archivos: El mapa del tesoro (y el lío de Windows vs Mac)

Para decirle a Python dónde está un archivo, usamos una "ruta" (path). Hay dos tipos:

1. Ruta Relativa: "Desde donde estoy ahora..."

Son instrucciones basadas en tu posición actual.

- Ejemplo: `"datos/mi_archivo.txt"` significa "Busca una carpeta llamada 'datos' aquí al lado, y entra en ella".
- **Ventaja:** Es corta y fácil de mover el proyecto entero a otro ordenador.
- **Desventaja:** Si ejecutas el programa desde una terminal situada en otra carpeta diferente, las instrucciones ya no sirven y fallará.

2. Ruta Absoluta: "La dirección GPS exacta"

Es la dirección completa desde la raíz del disco duro.

- Ejemplo Windows:
"C:\Usuarios\Nombre\Proyectos\MiJuego\datos\partida.sav"
- Ejemplo Mac/Linux:
"/Users/Nombre/Proyectos/MiJuego/datos/partida.sav"
- **Ventaja:** Siempre funciona, no importa desde dónde ejecutes el script.
- **Desventaja:** Es larguísima y **no funciona si cambias de ordenador** (porque tu amigo no se llama igual que tú en su PC).

El Gran Problema: La guerra de las barras (\ vs /)

Fíjate en los ejemplos de arriba.

- **Windows** usa la barra invertida (\) para separar carpetas.
- **Mac y Linux** usan la barra normal (/).

Si escribes esto en tu código: ruta = "carpeta\archivo.txt"

Tu programa funcionará genial en tu Windows, pero si se lo pasas a un amigo con Mac, **¡se romperá!**

La Solución Profesional: El módulo os al rescate

¿Recuerdas la biblioteca estándar del Capítulo 10? Python tiene un módulo llamado `os` (Operating System) que es un experto en lidiar con estas diferencias entre sistemas.

En lugar de escribir las barras tú mismo, **deja que Python las construya por ti.**

La función estrella es `os.path.join()`. Le das las piezas del camino, y él pone el pegamento (la barra) correcto según dónde se esté ejecutando.

```
import os # Importamos al experto en sistemas operativos
print("--- Dónde estoy ahora ---")
# os.getcwd() nos dice el "Current Working Directory" (Directorio de trabajo
    actual)
donde_estoy = os.getcwd()
print(f"El script se está ejecutando desde: {donde_estoy}")
print("\n--- Construyendo rutas inteligentes ---")
# Queremos acceder al archivo: carpetita/datos.txt
carpeta = "carpetita"
archivo = "datos.txt"
# FORMA INCORRECTA (Esto fallará en Windows o en Mac/Linux):
# ruta_mala = carpeta + "/" + archivo
# FORMA CORRECTA Y PROFESIONAL:
# os.path.join une las partes usando el separador correcto del sistema.
ruta_inteligente = os.path.join(carpeta, archivo)
print(f"Python ha construido esta ruta para tu sistema: {ruta_inteligente}")
# Ahora podemos usar esa ruta segura para abrir archivos
# (Este bloque fallará si no creas primero la carpeta 'carpetita',
# pero sirve para ver cómo se usa la ruta).
try:
    print(f"Intentando abrir: {ruta_inteligente} ...")
    # Usamos la variable de ruta en el open()
    with open(ruta_inteligente, "w") as f:
        f.write("¡Prueba de ruta en subcarpeta!")
```

```
print("¡Archivo creado con éxito en la subcarpeta!")
except FileNotFoundError:
    print(f"❌ Error: Parece que la carpeta '{carpeta}' no existe.")
    print(" (Python puede crear archivos, ¡pero no carpetas automáticamente al abrir!)")
```

REGLA DE ORO: Siempre usa `os.path.join()` cuando trabajes con carpetas.
Tu 'yo del futuro' te lo agradecerá cuando cambies de ordenador.

Si ejecutas esto, fíjate en cómo `ruta_inteligente` usa la barra correcta (`\` o `/`) dependiendo del sistema operativo que estés usando ahora mismo. ¡Magia!



Capítulo 13: Clases y Objetos - El molde y la galleta

La POO se basa en modelar el mundo real dentro de nuestro código. Si tu programa trata sobre una tienda online, en tu código habrá "objetos" que representen "Productos", "Clientes" y "Carritos de compra".

Pero antes de poder crear esos objetos, necesitamos definir qué *son*.

13.1 La analogía fundamental: El Plano vs. La Casa

Este es el concepto más importante de toda la POO. Si entiendes esto, entiendes el 80%.

Imagina que eres un arquitecto y quieres construir una urbanización con 100 casas idénticas.

1. La Clase (El Plano):

Primero, dibujas un plano. El plano NO es una casa. No puedes vivir en el plano. Es solo un papel con instrucciones que define cómo serán las casas:

- **Atributos (Datos):** Tendrá 2 plantas, paredes blancas, 3 ventanas.
- **Métodos (Acciones):** La puerta se podrá abrir, las luces se podrán encender.
- *En Python, a este "plano" o plantilla abstracta lo llamamos **CLASE (Class)**.*

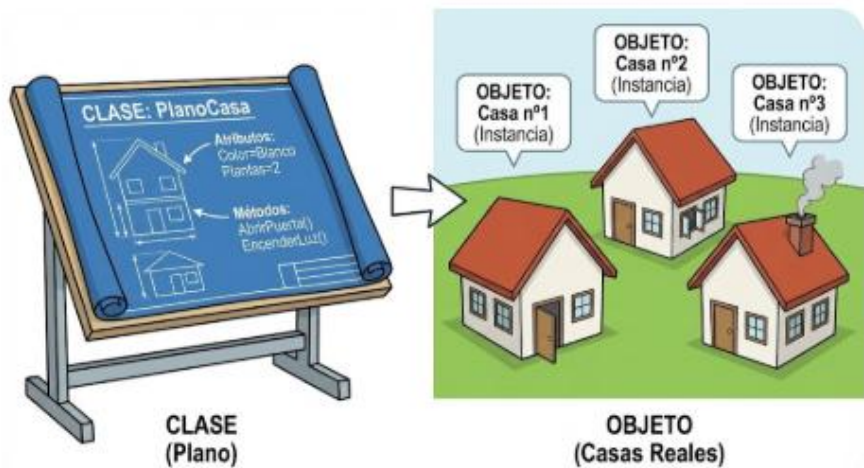
2. El Objeto (La Casa):

Una vez que tienes el plano, llamas a los albañiles y empiezan a construir. Usando ese único plano, construyen la casa N°1, luego la casa N°2, y así hasta la N°100.

- Cada casa es real, tangible, ocupa un espacio físico y puedes vivir en ella.
- Todas están hechas con el mismo molde, pero son **independientes**: si pintas la casa N°1 de azul, la casa N°2 sigue siendo blanca.
- *En Python, a cada una de estas casas concretas creadas a partir de la clase la llamamos **OBJETO** (o **Instancia**).*

En resumen:

Concepto	Analogía	Definición Técnica	¿Es tangible en memoria?
Clase	El plano arquitectónico (o molde de galletas).	Es la plantilla que define las propiedades (variables) y comportamientos (funciones) comunes de un tipo de cosa.	No. Es solo la definición.
Objeto	La casa construida (o la galleta horneada).	Es una instancia específica creada a partir de una clase. Es la "cosa" real con la que trabaja el programa.	Sí. Ocupa espacio en la RAM.



¡Genial! Ya tenemos la idea del plano en la cabeza. Ahora, vamos a coger el teclado y a dibujar ese plano usando código Python.

Vamos a crear la definición de una clase para un **Robot**.

13.2 Definiendo tu primera Clase: El plano del Robot

Para decirle a Python que vamos a crear un nuevo "tipo de cosa", usamos la palabra clave **class**.

Regla de estilo: Los nombres de las clases en Python se escriben en **PascalCase** (la primera letra de cada palabra en mayúscula, sin guiones bajos). Ejemplos: Robot, CuentaBancaria, EnemigoFinal.

Paso 1: La estructura básica y los Atributos (Datos)

Un plano define qué propiedades tiene el objeto. En Python, los **atributos** son simplemente variables definidas dentro de la clase.

Vamos a definir un robot que, por defecto, sea un modelo genérico y de color gris.

Python

```
class Robot:
```

```
    """
```

```
    Este es el plano (Clase) para crear robots.
```

```
    Define sus datos (atributos) y acciones (métodos).
```

```
    """
```

```
    # --- 1. ATRIBUTOS (Datos que definen cómo es) ---
```

```
    # Por ahora, pondremos valores fijos para todos los robots que  
    creamos.
```

```
    nombre_modelo = "Unidad Genérica X-1"
```

```
    color = "Gris estándar"
```

```
    nivel_bateria = 100
```

```
    encendido = False
```

Si ejecutas solo este código, no pasará nada. Has guardado el plano en un cajón, pero no has construido nada aún.

Paso 2: Los Métodos (Acciones) y el misterioso self

Un plano también define qué puede **hacer** el objeto. En Python, los **métodos** son funciones definidas dentro de la clase.

Aquí viene la parte que confunde a todos los principiantes: **El parámetro self**.

Cuando tengas 100 robots fabricados con este plano y le digas a uno que salude, ese robot necesita una forma de saber que es **él mismo** quien tiene que hablar, y que tiene que usar **su propio** nombre y color, no los de otro.

- **self es la referencia interna del objeto a sí mismo.**
- Es una regla obligatoria en Python: el primer parámetro de casi cualquier método que definas en una clase **DEBE** llamarse self.

No te preocupes, es un parámetro "invisible" para ti. Cuando uses el método, Python se encarga de rellenarlo automáticamente.

Añadamos métodos a nuestro robot usando self para acceder a sus propios atributos.

```
class Robot:
    # --- ATRIBUTOS ---
    nombre_modelo = "Unidad Genérica X-1"
    color = "Gris estándar"
    nivel_bateria = 100
    encendido = False

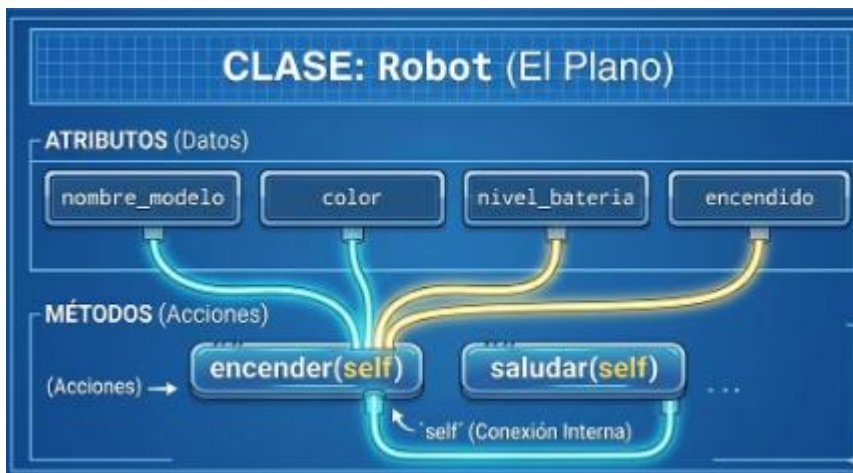
    # --- 2. MÉTODOS (Acciones que puede hacer) ---
    def encender(self):
```

```

# Usamos 'self.encendido' para modificar EL PROPIO estado de este robot.
print("🤖 Bip... Zumbido... Iniciando sistemas.")
self.encendido = True
def saludar(self):
    # Este método necesita saber si el robot está encendido.
    # ¡Gracias a 'self' puede consultarlo!
    if self.encendido:
        print("-----")
        print("🤖 ¡Bip, bop! Saludos, unidad biológica.")
        # Usamos 'self.color' para acceder a SU PROPIO color.
        print(f"Soy una {self.nombre_modelo} de color {self.color}.")
        print(f"Mi batería actual está al {self.nivel_bateria}%.")
        print("-----")
    else:
        print("(El robot no responde. Parece estar apagado.)")

```

¡Ya está! Hemos completado nuestro primer plano. Tenemos datos (color, batería...) y tenemos comportamientos que usan esos datos (saludar verifica si está encendido y dice su color).



13.3 Creando Objetos: ¡La fábrica empieza a producir!

El proceso de usar una clase para crear un objeto real se llama **instanciación** (crear una instancia).

La sintaxis es increíblemente sencilla. Es como si "llamaras" a la clase como si fuera una función:

```
# (Asume que el código de la clase Robot del apartado anterior está justo
    encima de esto)
# --- Programa Principal ---
print("--- Iniciando la fábrica ---")
# 1. INSTANCIACIÓN: Creamos nuestro primer robot real.
# Le decimos a Python: "Usa el plano 'Robot' y construye uno nuevo".
# Guardamos la referencia a ese nuevo robot en la variable 'robot1'.
robot1 = Robot()
print("¡Robot 1 creado con éxito!")
# Si intentamos imprimir la variable, veremos que es un objeto de tipo Robot
print(robot1) # Salida: <__main__.Robot object at 0x...> (La dirección de
    memoria)
```

En este punto, Python ha reservado un hueco en la memoria RAM, ha creado la estructura definida en el plano y la ha rellenado con los valores por defecto (nombre_modelo="Unidad Genérica X-1", encendido=False, etc.).

13.4 Usando el objeto: La notación del punto (.)

Ahora que tenemos a `robot1` (que es nuestro mando a distancia para controlar a ese robot específico), ¿cómo interactuamos con él?

Usamos el **operador punto (.)**. Es la llave universal en POO.

- **Acceder a un atributo (dato):** `objeto.nombre_atributo`
- **Llamar a un método (acción):** `objeto.nombre_metodo()`
(¡No olvides los paréntesis!)

¿Recuerdas el misterioso `self`? Aquí ocurre la magia. Cuando tú escribes `robot1.encender()`, Python hace un truco por detrás. Automáticamente agarra al propio objeto (`robot1`) y lo pasa como el primer argumento (el `self`) al método `encender`.

Tú escribes: `robot1.encender()` Python interpreta internamente:
`Robot.encender(robot1) <--` ¡Aquí se rellena el `self`!

Vamos a probarlo con el código completo:

```
# --- DEFINICIÓN DE LA CLASE (El Plano) ---
class Robot:
    # Atributos
    nombre_modelo = "Unidad Genérica X-1"
    color = "Gris estándar"
    nivel_bateria = 100
    encendido = False

    # Métodos
    def encender(self):
        print("🤖 Bip... Zumbido... Iniciando sistemas.")
```

```

        self.encendido = True # 'self' se refiere al robot específico que llamamos
def saludar(self):
    # Usamos 'self' para comprobar el estado de ESTE robot
    if self.encendido:
        print(f"🤖 ¡Hola! Soy un {self.nombre_modelo} de color {self.color}.")
    else:
        print("(El robot no responde. Parece estar apagado.)")
# --- PROGRAMA PRINCIPAL (Usando los objetos) ---
print("--- Fábrica de Robots ---")
# 1. Creamos el objeto
mi_robot = Robot()
print("Se ha fabricado un nuevo robot.")
# 2. Accedemos a sus atributos (Datos) usando el punto
print(f"El estado inicial es: Encendido={mi_robot.encendido},
      Batería={mi_robot.nivel_bateria}%")
# 3. Intentamos usar un método (Acción)
print("\n--- Intentando que salude (está apagado) ---")
mi_robot.saludar()
# 4. Usamos otro método para modificar su estado interno
print("\n--- Encendiendo el robot ---")
# Al llamar a esto, Python pasa 'mi_robot' al 'self' del método 'encender'
mi_robot.encender()
# Verificamos que el atributo ha cambiado
print(f"Nuevo estado: Encendido={mi_robot.encendido}")

# 5. Ahora que su estado interno ha cambiado, el método saludar funcionará
diferente
print("\n--- Intentando que salude (ahora está encendido) ---")
mi_robot.saludar()

```

Salida:

--- Fábrica de Robots ---

Se ha fabricado un nuevo robot.

El estado inicial es: Encendido=False, Batería=100%

--- Intentando que salude (está apagado) ---

(El robot no responde. Parece estar apagado.)

--- Encendiendo el robot ---

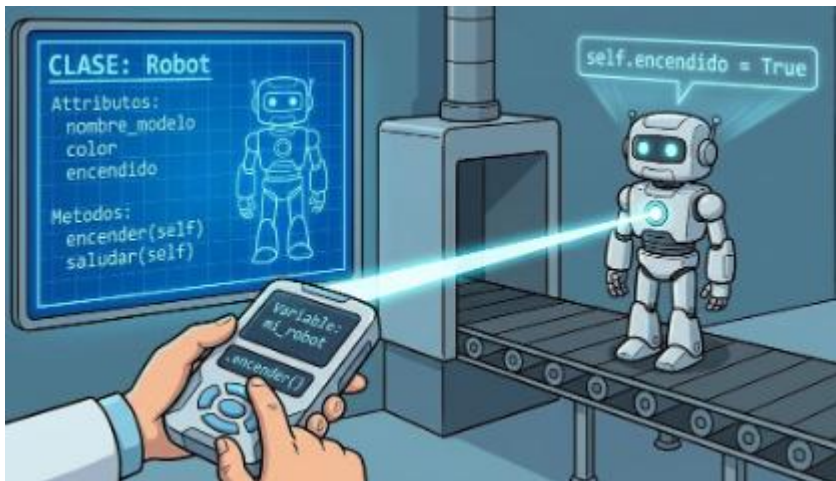
🤖 Bip... Zumbido... Iniciando sistemas.

Nuevo estado: Encendido=True

--- Intentando que salude (ahora está encendido) ---

🤖 ¡Hola! Soy un Unidad Genérica X-1 de color Gris estándar.

¡Felicidades! Has creado y controlado tu primer objeto.



El problema de nuestra fábrica actual es que solo produce clones idénticos. Todos los robots se llaman "Unidad Genérica X-1" y son grises.

Para solucionar esto, necesitamos un mecanismo que nos permita "inyectar" datos personalizados en el momento justo en que el robot cobra vida.

Ese mecanismo es un método especial y mágico llamado **Constructor**.

13.5 El Constructor `__init__`: El momento del nacimiento

En Python, el constructor es un método con un nombre reservado muy concreto: `__init__` (nota los **dos guiones bajos** al principio y al final).

Este método tiene un superpoder: **Python lo ejecuta automáticamente SIEMPRE que creas un nuevo objeto** (cuando haces `Robot()`).

La Analogía: El túnel de personalización en la fábrica

Hasta ahora, el robot salía del túnel de montaje y ya estaba listo. Ahora, con `__init__`, justo antes de salir, el robot pasa por una estación donde un operario le pregunta: "¿Cómo te vas a llamar?" y "¿De qué color te pinto?".

Cómo se usa en código

Vamos a modificar nuestro plano (la clase Robot) para incluir este método. Le añadiremos parámetros para que quien pida el robot tenga que decir el nombre y el color.

 **¡Atención! Un cambio importante:** Ya no definiremos los atributos "sueltos" al principio de la clase. Ahora los definiremos **dentro** del constructor usando self. Esto es lo que los hace verdaderamente únicos para cada objeto.

```
class Robot:
    """
    Plano de robot mejorado con personalización al nacer.
    """

    # --- EL CONSTRUCTOR MÁGICO ---
    # Se ejecuta automáticamente al hacer 'Robot(...)'.
    # El primer parámetro SIEMPRE es 'self'.
    # Los demás son los datos que pedimos para personalizar.
    def __init__(self, nombre_que_me_dan, color_elegido):
        print(f" ⚡ ¡Ha nacido un nuevo robot! (Ejecutando __init__ para {nombre_que_me_dan})")

        # --- ASIGNACIÓN DE ATRIBUTOS PERSONALIZADOS ---
        # Aquí está la CLAVE. Usamos 'self' para "pegarle" los datos al robot que está naciendo.
        # "La propiedad 'nombre' de ESTE robot (self.nombre) será igual a
        # lo que me hayan pasado en el parámetro 'nombre_que_me_dan'".
        self.nombre = nombre_que_me_dan
        self.color = color_elegido
        # También podemos establecer valores fijos iniciales.
        self.bateria = 100
```

```

        self.encendido = False
# --- MÉTODOS (Acciones) ---
def saludar(self):
    if self.encendido:
        # Ahora usamos los atributos que se definieron en el __init__
        print(f"🤖 ¡Hola! Soy {self.nombre} y soy de color {self.color}.")
    else:
        print(f"(El robot {self.nombre} no responde. Está apagado.)")

def encender(self):
    print(f"🤖 Zzzzt... {self.nombre} iniciando sistemas.")
    self.encendido = True

# --- PROGRAMA PRINCIPAL: ¡Fábrica personalizada! ---
print("--- Pedido 1 ---")
# Ahora, AL CREAR el objeto, ¡estamos OBLIGADOS a pasar los datos!
# Robot(nombre, color)
robot_rojo = Robot("Rojo-Veloz", "Rojo Ferrari")
print("\n--- Pedido 2 ---")
# Creamos otro TOTALMENTE DIFERENTE usando el MISMO plano.
robot_azul = Robot("Azul-Tranquilo", "Azul Cielo")
# Vamos a comprobar que son independientes
print("\n--- Interactuando con los robots ---")

# Encendemos solo al rojo
robot_rojo.encender()
# Les pedimos que saluden
print("\nSaluda, Rojo:")
robot_rojo.saludar()
print("\nSaluda, Azul:")
robot_azul.saludar() # Este seguirá apagado porque nadie lo encendió.

```

Salida del programa:

--- Pedido 1 ---

⚡ ¡Ha nacido un nuevo robot! (Ejecutando `__init__` para Rojo-Veloz)

--- Pedido 2 ---

⚡ ¡Ha nacido un nuevo robot! (Ejecutando `__init__` para Azul-Tranquilo)

--- Interactuando con los robots ---

🤖 Zzzzt... Rojo-Veloz iniciando sistemas.

Saluda, Rojo:

🤖 ¡Hola! Soy Rojo-Veloz y soy de color Rojo Ferrari.

Saluda, Azul:

(El robot Azul-Tranquilo no responde. Está apagado.)

¡Misión cumplida! Hemos usado un único molde (class Robot) para crear dos objetos (robot_rojo y robot_azul) que son independientes y tienen sus propios datos únicos gracias al constructor `__init__`.



Capítulo 14: Herencia - No reinventes la rueda

La herencia nos permite crear nuevas clases basadas en clases que ya existen. Es como la herencia biológica: tú heredas rasgos de tus padres (ojos, color de pelo), pero luego tienes tus propias características únicas que ellos no tienen.

14.1 La clase Padre y la clase Hija

- **Clase Padre (o Superclase):** Es la clase original, la base. (Ej: Robot).
- **Clase Hija (o Subclase):** Es la nueva clase que creamos. **Hereda automáticamente** todos los atributos y métodos del padre, y puede añadir los suyos propios o modificar los que ha heredado. (Ej: RobotVolador).

Sintaxis básica

Es sencillísimo. Al definir la nueva clase, ponemos entre paréntesis el nombre de la clase padre de la que queremos heredar.

```
# --- CLASE PADRE (La base) ---
class Robot:
    def __init__(self, nombre):
        print(f" -> [Constructor Robot] Inicializando a {nombre}...")
        self.nombre = nombre
        self.bateria = 100

    def saludar(self):
        print(f" 🤖 Soy el robot {self.nombre} y voy sobre ruedas.")
```

```
# --- CLASE HIJA (La nueva versión) ---
# ¡Fíjate en los paréntesis! Esto significa: "RobotVolador ES UN TIPO DE Robot"
class RobotVolador(Robot):
    # Aquí dentro no escribimos nada por ahora.
    # Usamos 'pass' que significa "no hagas nada aquí".
    pass
# --- PRUEBA ---
print("--- Creando un robot normal ---")
robot_tierra = Robot("R2-D2")
robot_tierra.saludar()
print("\n--- Creando un robot volador (vacío) ---")
# ¡Magia! Aunque RobotVolador está vacío, ¡funciona!
# Ha heredado el constructor __init__ y el método saludar de su padre.
robot_cielo = RobotVolador("BB-8 Volador")
robot_cielo.saludar()
```

¡Boom! Sin escribir una sola línea de lógica en RobotVolador, ya tenemos un robot funcional. Ha heredado todo de su padre.

14.2 Extendiendo y Sobrescribiendo (La verdadera potencia)

Heredar lo mismo no tiene mucha gracia. La potencia viene ahora:

1. **Extender (Añadir cosas nuevas):** El hijo puede tener métodos que el padre no tiene.
2. **Sobrescribir (Modificar el comportamiento):** El hijo puede tener un método con el **mismo nombre** que el padre, pero que haga algo diferente.

Vamos a mejorar nuestro RobotVolador.

```
class Robot:
    # (El mismo código del padre que arriba)
    def __init__(self, nombre):
        self.nombre = nombre
        self.bateria = 100
    def saludar(self):
        print(f"🤖 Soy el robot {self.nombre} y voy sobre ruedas.")
    def moverse(self):
        print(f" [Robot {self.nombre}] Moviéndome por el suelo... brrrrm...")
# --- CLASE HIJA MEJORADA ---
class RobotVolador(Robot):
    # 1. EXTENDER: Añadimos un método NUEVO y ÚNICO.
    def volar(self):
        if self.bateria > 20:
            print(f"🚀 ¡Fiuuum! {self.nombre} despegando y surcando los cielos.")
            self.bateria -= 10 # Volar gasta batería
        else:
            print(f"⚠️ {self.nombre} no tiene suficiente batería para volar.")
    # 2. SOBRESCRIBIR (Override): Cambiamos un método existente.
    # Definimos 'saludar' otra vez. Python usará ESTA versión para los robots
    # voladores.
    def saludar(self):
        print(f"🗨️ ¡Saludos desde las alturas! Soy la unidad aérea {self.nombre}.")
# --- PRUEBA ---
robot_tierra = Robot("Wall-E")
robot_cielo = RobotVolador("Eva")
print("\n--- Prueba de Saludo (Sobrescritura) ---")
robot_tierra.saludar() # Usa la versión del padre
robot_cielo.saludar() # Usa su PROPIA versión sobrescrita
print("\n--- Prueba de Movimiento (Herencia pura) ---")
```

```
robot_tierra.moverse()
robot_cielo.moverse() # El hijo también puede moverse por el suelo, lo heredó.
print("\n--- Prueba de Vuelo (Extensión) ---")
# robot_tierra.volar() # ¡ERROR! Esto fallaría, el padre no sabe volar.
robot_cielo.volar() # ¡Éxito! Método exclusivo del hijo.
```

Salida:

```
--- Prueba de Saludo (Sobrescritura) ---
🤖 Soy el robot Wall-E y voy sobre ruedas.
🚁 ¡Saludos desde las alturas! Soy la unidad aérea Eva.
--- Prueba de Movimiento (Herencia pura) ---
[Robot Wall-E] Moviéndome por el suelo... brrrrm...
[Robot Eva] Moviéndome por el suelo... brrrrm...
--- Prueba de Vuelo (Extensión) ---
🚀 ¡Fiuuum! Eva despegando y surcando los cielos.
```

14.3 El comando vital: `super()`

Hay un caso especial muy importante. ¿Qué pasa si el hijo quiere tener **su propio constructor** `__init__` para añadir atributos nuevos (como `numero_alas`), pero **también necesita** que se ejecute la lógica básica del constructor del padre (para poner el nombre y la batería)?

Si defines `__init__` en el hijo, ¡sobrescribes totalmente el del padre! El padre ya no inicializará nada.

Para solucionarlo, usamos la función mágica **`super()`**. Significa "Llama a la clase padre".

```
class RobotVolador(Robot):
    # Queremos nuestro propio constructor para pedir el número de alas
```

```

def __init__(self, nombre, numero_alas):
    print(f" -> [Constructor Hijo] Iniciando configuración de vuelo...")
    # ¡IMPORTANTE! Llamamos al constructor del PADRE primero.
    # Le pasamos los datos que él necesita (el nombre).
    # "Papá, haz tu parte del trabajo primero".
    super().__init__(nombre)
    # Ahora hacemos NUESTRA parte del trabajo exclusivo.
    self.alas = numero_alas
    print(f" -> [Constructor Hijo] Añadidas {self.alas} alas a {self.nombre}.")
def saludar(self):
    # Podemos usar super() también en métodos normales!
    # "Llama al saludo de mi padre, y luego añado lo mío".
    super().saludar()
    print(f" (...y además tengo {self.alas} alas brillantes!)")
print("\n--- Prueba con super() ---")
# Ahora el constructor pide DOS cosas: nombre y alas
drone = RobotVolador("Drone-01", 4)
print("\n--- Saludando con super() ---")
drone.saludar()

```

Salida:

```

--- Prueba con super() ---
-> [Constructor Hijo] Iniciando configuración de vuelo...
-> [Constructor Robot] Inicializando a Drone-01... <-- ¡Se ejecutó el padre!
-> [Constructor Hijo] Añadidas 4 alas a Drone-01. <-- ¡Y luego siguió el hijo!
--- Saludando con super() ---
🤖 Soy el robot Drone-01 y voy sobre ruedas. <-- Saludo del padre
(...y además tengo 4 alas brillantes!) <-- Añadido del hijo

```

HERENCIA: Reutilizar y Mejorar



¡Excelente! Ya dominas la herencia y tus robots son cada vez más complejos.

Pero ahora surge un problema. Imagina que estás trabajando en equipo. Tú has diseñado con mucho cuidado tu clase Robot para que la batería nunca pase del 100% ni baje del 0%.

Y llega tu compañero de equipo (o tú mismo con prisas un viernes por la tarde) y escribe esto en el programa principal:

```
mi_robot.bateria = 5000 # ¡Ups! ¡Batería sobrecargada!
```

```
mi_robot.bateria = -20 # ¡Ups! ¡Energía negativa!
```

¡Tu robot ha explotado! Tu lógica interna se ha roto porque **cualquiera desde fuera puede meter mano en las variables internas.**

Aquí es donde entra el tercer pilar de la POO: el **Encapsulamiento**. Se trata de proteger los datos delicados y exponer solo una forma segura de interactuar con ellos.

Capítulo 15: Encapsulamiento - Secretos y seguridad

La Analogía: El panel de control vs. el cableado interno

- **Sin Encapsulamiento:** Tu robot tiene todos los cables por fuera. Cualquiera puede llegar, cortar el cable rojo y unirlo con el azul. El resultado es impredecible y peligroso.
- **Con Encapsulamiento:** Tu robot tiene una carcasa elegante. El usuario solo ve botones bonitos: "Encender", "Volar". No sabe qué cables se mueven por dentro al pulsar el botón, y no le importa. Solo sabe que si pulsa el botón, el robot funciona de forma segura.

15.1 La filosofía de Python: "Todos somos adultos aquí"

A diferencia de otros lenguajes como Java o C++, Python **NO** tiene una forma estricta de hacer que una variable sea 100% "privada" e inaccesible desde fuera.

La filosofía de Python es: "No te voy a prohibir entrar en la sala de máquinas, pero te voy a poner carteles muy claros de que no deberías estar ahí".

Para esto, usamos **convenciones de nombres** con guiones bajos (_).

15.2 El aviso amable: Un guion bajo (_variable)

Si pones un solo guion bajo delante del nombre de un atributo o método (ej. `self._codigo_secreto`), estás poniendo un cartel de **"Solo personal autorizado"**.

Técnicamente, el programa principal *puede* acceder y modificar esa variable, pero es una señal universal entre programadores de Python que significa: **"Esta es una variable interna. Si la tocas desde fuera, es bajo tu propia responsabilidad y podrías romper algo"**.

15.3 La barrera fuerte: Doble guion bajo (__variable)

Si quieres algo más de seguridad, usas **dos guiones bajos** al principio (ej. `self.__bateria_critica`).

Esto activa un mecanismo de Python llamado *name mangling* (destrozo de nombres). Python cambia internamente el nombre de esa variable para que sea difícil (aunque no imposible) acceder a ella desde fuera por accidente. Es lo más parecido a "privado" que usamos habitualmente.

¡Vamos a arreglar nuestro Robot!

Vamos a esconder la batería real (`__bateria`) y vamos a crear métodos públicos seguros (botones del panel) para interactuar con ella. A estos métodos se les suele llamar *getters* (para obtener valor) y *setters* (para establecer valor).

```

class RobotSeguro:
    def __init__(self, nombre):
        self.nombre = nombre
        # --- ATRIBUTO PRIVADO ---
        # Usamos doble guion bajo. Desde fuera, 'mi_robot.__bateria' NO existirá.
        self.__bateria = 100
        print(f"🤖 {self.nombre} listo. Batería interna protegida.")
    # --- MÉTODOS PÚBLICOS (La interfaz segura) ---
    # GETTER: Método seguro para VER la batería
    def obtener_nivel_bateria(self):
        # Aquí dentro de la clase, Sí podemos acceder a __bateria sin problemas.
        return f"Nivel actual: {self.__bateria}%"
    # SETTER: Método seguro para MODIFICAR la batería
    def usar_bateria(self, cantidad):
        if cantidad > 0 and self.__bateria >= cantidad:
            self.__bateria -= cantidad
            print(f"-> Usados {cantidad}% de energía.")
        else:
            print(f"❌ ¡Error! No se puede usar esa cantidad de batería.")

    # Otro setter seguro
    def recargar_a_tope(self):
        print("-> Recargando sistemas al máximo...")
        self.__bateria = 100 # Aquí controlamos que nunca pase de 100

# --- PROGRAMA PRINCIPAL ---
bot = RobotSeguro("Guardián")

print("\n--- Intento de HACKEO (Acceso directo incorrecto) ---")
# Si descomentas la siguiente línea, dará un error: AttributeError
# print(bot.__bateria)

```

Incluso si intentamos asignarle un valor, no estaremos tocando la variable real interna.

```
bot.__bateria = 99999
```

```
print("He intentado hacer 'bot.__bateria = 99999'...")
```

Comprobemos si ha funcionado usando el método oficial:

```
print(f"¿Ha cambiado la batería real?: {bot.obtener_nivel_bateria()}")
```

```
print("(¡No! La variable real sigue a salvo dentro de la clase)")
```

```
print("\n--- Uso CORRECTO (A través de métodos públicos) ---")
```

Usamos los "botones" seguros que la clase nos ofrece.

```
print(bot.obtener_nivel_bateria())
```

```
bot.usar_bateria(30)
```

```
print(bot.obtener_nivel_bateria())
```

Intentamos una operación ilegal a través del método seguro

```
bot.usar_bateria(200)
```

Recargamos de forma segura

```
bot.recargar_a_tope()
```

```
print(bot.obtener_nivel_bateria())
```

Salida:



Guardián listo. Batería interna protegida.

--- Intento de HACKEO (Acceso directo incorrecto) ---

He intentado hacer 'bot.__bateria = 99999'...

¿Ha cambiado la batería real?: Nivel actual: 100%

(¡No! La variable real sigue a salvo dentro de la clase)

--- Uso CORRECTO (A través de métodos públicos) ---

Nivel actual: 100%

-> Usados 30% de energía.

Nivel actual: 70%



¡Error! No se puede usar esa cantidad de batería.

-> Recargando sistemas al máximo...

Nivel actual: 100%

¡Lo hemos conseguido! Hemos encapsulado la batería. Desde fuera, el programador solo ve métodos seguros (usar_batería, recargar). La lógica de que la batería no baje de cero ni suba de cien ahora está protegida dentro de la clase y no depende de la buena voluntad de quien use el robot.



¡Aquí comienza la recta final!

Has recorrido un camino increíble. Has pasado de no saber qué era una variable a diseñar complejos sistemas de robots con programación orientada a objetos.

Pero te voy a contar un secreto: los programadores profesionales no escriben todo el código desde cero.

Imagina que quieres construir una casa. No te pones a fabricar tus propios ladrillos, ni a forjar tus propios martillos. Vas a la ferretería y compras las herramientas y materiales que necesitas.

En Python, esa "ferretería" es gigantesca.

Capítulo 16: Librerías útiles - No reinventes la rueda (Parte 2)

Ya vimos en el Capítulo 10 qué eran los módulos y cómo instalar paquetes con pip. Ahora vamos a ver *para qué sirve realmente* todo eso.

Python es famoso porque "viene con las pilas incluidas" (la Biblioteca Estándar), pero su verdadera potencia reside en las **miles de librerías de terceros** creadas por la comunidad.

Vamos a ver dos ejemplos prácticos de cosas que serían difícilísimas de programar desde cero, pero que con la librería adecuada se hacen en 5 líneas.

16.1 Ejemplo 1: Visualización de Datos con matplotlib

El problema: Tienes una lista de números (por ejemplo, tus ahorros mes a mes) y quieres ver una gráfica bonita para entender la tendencia.

La solución: matplotlib, la librería reina para hacer gráficos científicos en 2D.

Paso previo (en tu terminal):

```
pip install matplotlib
```

El Código:

Importamos el módulo de dibujo de matplotlib y le ponemos un apodo corto 'plt'

```
import matplotlib.pyplot as plt
print("--- Generando gráfica de ahorros ---")
# 1. Los Datos
meses = ['Ene', 'Feb', 'Mar', 'Abr', 'May', 'Jun']
ahorros = [100, 150, 120, 300, 450, 420]
# 2. Crear el dibujo
plt.figure(figsize=(8, 4)) # Opcional: tamaño de la imagen

# Dibujamos una línea: eje X=meses, eje Y=ahorros
# marker='o' pone puntitos en los datos, color='green' la pinta de verde
plt.plot(meses, ahorros, marker='o', color='green', linestyle='-', linewidth=2)
# Añadimos decoración
plt.title("Mi progreso de Ahorro 2023")
plt.xlabel("Meses")
plt.ylabel("Euros ahorrados (€)")
```

```
plt.grid(True) # Añade una rejilla de fondo para leer mejor
print("¡Gráfica generada! Se abrirá en una ventana nueva...")
# 3. ¡Mostrar el resultado!
# Esto abrirá una ventana emergente con tu gráfica interactiva.
plt.show()
```

¡Boom! Con solo unas pocas líneas, Python ha dibujado una gráfica profesional con ejes, títulos y datos. ¿Te imaginas tener que programar píxel a píxel cómo dibujar esa línea?



16.2 Ejemplo 2: Hablando con Internet usando requests

El problema: Quieres que tu programa obtenga información en tiempo real de Internet. Por ejemplo, ¿qué tiempo hace en Tokio ahora mismo? ¿O quieres que te cuente un chiste aleatorio?

La solución: requests. La librería estándar de facto para que "Python navegue por internet" (técnicamente, para hacer peticiones HTTP).

Paso previo (en tu terminal):

```
pip install requests
```

El Código (Vamos a pedir un chiste a una API pública):

Una API es como una ventanilla de información para programas. Tú le pides datos a una dirección web (URL) y ella te devuelve información estructurada (normalmente en un formato llamado JSON, que Python entiende perfectamente como si fuera un diccionario).

```
import requests # Importamos el navegador para robots
print("--- Conectando con el ciberespacio para obtener un chiste ---")
# URL de una API pública y gratuita que da chistes
url_chiste = "https://official-joke-api.appspot.com/random_joke"
try:
    # 1. Hacemos la petición (como escribir la URL en el navegador y dar Enter)
    print(f'Llamando a: {url_chiste} ...')
    respuesta = requests.get(url_chiste, timeout=5) # timeout=5: si tarda más de
    5s, aborta
    # 2. Verificamos si ha ido bien (Código 200 significa OK)
    if respuesta.status_code == 200:
        print("¡Conexión establecida! Datos recibidos.\n")
```

```
# 3. Convertimos los datos recibidos (JSON) a un diccionario de Python
datos_chiste = respuesta.json()
# ¡Ya podemos usarlo como un diccionario normal!
print("-----")
print(f"🤖 Chiste tipo: {datos_chiste['type']}")
print(f"❓ Pregunta: {datos_chiste['setup']}")
print("...")
print(f"💡 Respuesta: {datos_chiste['punchline']}")
print("-----")
else:
    print(f"❌ Error en la petición. Código de estado:
{respuesta.status_code}")

except requests.exceptions.RequestException as e:
    # Esto atrapa errores de conexión (sin internet, web caída, etc.)
    print(f"❌ Error de conexión grave: {e}")
```

Salida (ejemplo, el chiste cambiará):

--- Conectando con el ciberespacio para obtener un chiste ---
 Llamando a: https://official-joke-api.appspot.com/random_joke ...
 ¡Conexión establecida! Datos recibidos.

```
-----
🤖 Chiste tipo: programming
❓ Pregunta: Why do programmers wear glasses?...
💡 Respuesta: Because they need to C#
-----
```

¡Acabas de conectar tu programa con el mundo exterior! Con esta base, podrías conectarte a la API del tiempo, a la de bolsa, a la de Twitter... las posibilidades son infinitas.

Con esto cerramos el **Capítulo**.

Has visto que Python no es solo el lenguaje, sino todo lo que la comunidad ha construido a su alrededor.

¿Estás listo para el final? En el último capítulo, haremos un resumen de todo el viaje y te daré un mapa para saber **qué hacer ahora que ya sabes programar**.

¡Lo has logrado!

Enhorabuena. De verdad. Has llegado al final de este curso intensivo.

Mira atrás un momento. ¿Recuerdas cuando ver $x = 5$ te parecía algo nuevo? ¿Cuando los bucles for te daban vueltas en la cabeza o cuando las funciones te parecían magia negra?

Ahora entiendes qué es una variable, cómo controlar el flujo de un programa, cómo estructurar datos complejos en listas y diccionarios, cómo manejar errores para que tu programa no explote, cómo guardar información para siempre en archivos, e incluso cómo diseñar sistemas complejos usando clases y objetos como un verdadero arquitecto de software. Y por si fuera poco, has aprendido a instalar y usar herramientas creadas por otros para darle superpoderes a tu código.

Ya no eres una persona que "no sabe nada de programación". **Ahora eres una persona que sabe programar en Python.** Tienes los cimientos más sólidos que podrías desear.

Capítulo Final: El Camino por Delante

La pregunta del millón es: **¿Y ahora qué?**

Este curso te ha dado el "carnet de conducir". Sabes cómo funciona el coche (Python), sabes meter las marchas (sintaxis) y conoces las normas de circulación (lógica de programación).

Pero saber conducir no te convierte en piloto de Fórmula 1, ni en camionero, ni en taxista. Ahora te toca elegir tu destino y practicar el viaje.

Aquí tienes el mapa de las rutas más populares que puedes tomar a partir de ahora:

1. El camino del Desarrollador Web

¿Quieres crear páginas web dinámicas como Instagram, Pinterest o la web de tu pizzería favorita? Python es una bestia en el servidor (backend).

- **Tu próxima herramienta:** Aprende un *framework* web como **Flask** (para empezar, minimalista) o **Django** (muy potente, con todo incluido).
- **Proyecto sugerido:** Crea un blog personal donde tú (y solo tú, con contraseña) puedas escribir y publicar artículos que cualquiera pueda leer.

2. El camino del Científico de Datos y la IA



¿Te fascinan los datos, las predicciones y la inteligencia artificial? Python es el rey absoluto en este campo.

- **Tus próximas herramientas:**
 - **Pandas y NumPy:** Para manipular y analizar tablas de datos gigantescas como si fueran hojas de Excel con esteroides.
 - **Matplotlib y Seaborn:** Para visualizar esos datos y encontrar patrones ocultos.
 - **Scikit-learn:** Para empezar con el aprendizaje automático (Machine Learning) clásico.
- **Proyecto sugerido:** Descarga un conjunto de datos público (por ejemplo, sobre precios de casas o estadísticas de Pokémon) y trata de encontrar qué factores influyen más en el precio o en la fuerza de un Pokémon, visualizándolo con gráficas.

3. El camino de la Automatización y Scripting

¿Tienes tareas aburridas y repetitivas en tu ordenador que te gustaría que se hicieran solas? Python es el mejor pegamento para unir programas y automatizar flujos de trabajo.

- **Tus próximas herramientas:**
 - **Selenium:** Para controlar el navegador web automáticamente (rellenar formularios, descargar cosas).
 - **OpenPyXL:** Para leer y escribir archivos de Excel.

- **Smtplib:** Para enviar correos electrónicos automáticamente.
- **Proyecto sugerido:** Crea un script que entre cada mañana a una web de noticias, busque los titulares que contengan una palabra clave que te interese y te envíe un resumen por correo electrónico.

4. El camino del Desarrollador de Videojuegos 🎮

¿Quieres hacer tus propios juegos? Python quizás no es para el próximo *Call of Duty*, pero es genial para aprender la lógica de los juegos en 2D.

- **Tu próxima herramienta: Pygame.** Una librería sencilla y divertida para hacer juegos retro.
- **Proyecto sugerido:** Recrea el clásico "Pong" o el "Snake" (la serpiente). Es la mejor escuela para entender bucles de juego, colisiones y puntuaciones.

Un último consejo de amigo

La programación no se aprende leyendo, se aprende haciendo. **Vas a romper cosas. Muchas veces. Y eso es bueno.**

Cada error (SyntaxError, TypeError, NameError) no es un fracaso, es una lección. La habilidad más importante de un programador no es saberse de memoria todas las funciones, sino saber leer un mensaje de error, buscar en Google, encontrar la solución en Stack Overflow o en la documentación oficial, probarla y aprender.

No tengas miedo de empezar proyectos que te parezcan grandes. Divídelos en trozos pequeños y resuelve uno a uno.

Tienes el poder de crear cosas de la nada con solo tus dedos y un teclado. Úsalo bien.

¡Feliz programación!



Sobre el autor

Diplomado en Empresariales por la Universidad de Sevilla, aficionado a la programación y a la informática en general, certificado en programación con : Java, JavaScript, Php, Android, VisualBasic, Python, Html, Scratch, AppInventor.

Podeis seguirme en:

Twitter:

<https://twitter.com/josecodetech>

Facebook:

<https://www.facebook.com/josecodetech/>

Youtube (lista de reproducción Python):

<https://www.youtube.com/@josecodetech>

GitHub:

https://github.com/josecodetech/curso_python